



League of Legends Win Prediction: A Machine Learning Approach

A project submitted to the Faculty of Computers and Information, in
partial fulfilment of the requirements for the degree of

**Pre-Master in
Computer Science**

Prepared By

Ibrahim Mohamed Ibrahim Elsayed Hegazi

Supervisors

Dr Mohamed Abu Elsoud

Dr Mohamed Haggag

2025-2026

Table of Content

- 1 Introduction
 - 1.1 Background and Motivation
 - ◆ 1.1.1 The Rise of Esports and Competitive Gaming Analytics
 - ◆ 1.1.2 League of Legends: A Case Study in Game Complexity
 - ◆ 1.1.3 The Analytical Opportunity in Post-Game Data
 - ◆ 1.1.4 Practical Applications of Win Prediction Models
 - ◆ 1.1.5 Gaps in Existing Literature
 - 1.2 Problem Statement
 - 1.3 Research Questions
 - ◆ Primary Research Question
 - ◆ Secondary Research Questions
 - 1.4 Research Objectives
 - ◆ Primary Objective
 - ◆ Secondary Objectives
 - 1.5 Scope and Limitations
 - ◆ 1.5.1 Scope of the Research
 - ◆ 1.5.2 Limitations
 - ◆ 1.5.3 Delimitations (Explicit Exclusions)
 - 1.6 Thesis Organization
- 2 Literature Review: Critical Analysis of Prior Work
 - 2.1 Background: Foundational Concepts and Theoretical Framework
 - ◆ 2.1.1 League of Legends as a Complex Adaptive System
 - ◆ 2.1.2 Machine Learning Paradigms for Binary Outcome Prediction
 - ◆ 2.1.3 Feature Selection Theory and Methodology
 - ◆ 2.1.4 Model Interpretability and SHAP
 - 2.2 Literature Review: Critical Analysis of Prior Work
 - ◆ 2.2.1 Win Prediction Using Machine Learning Approaches
 - ◆ 2.2.2 Champion Selection and Recommendation Systems

- ◆ 2.2.3 Esports Analytics and Performance Metrics
 - ◆ 2.2.4 Team Composition and Optimization
 - ◆ 2.2.5 Data Summarization and Reinforcement Learning Applications
- 2.3 Synthesis of Literature Gaps and Our Contributions
 - ◆ 2.3.1 Summary of Identified Gaps
 - ◆ 2.3.2 Theoretical Positioning
 - ◆ 2.3.3 Novel Contributions Relative to Prior Work
- 3 Methodology
 - 3.1 Data Description
 - ◆ 3.1.1 Data Source and Collection Overview
 - ◆ 3.1.2 Sampling Strategy: Tier-Stratified Snowball Sampling
 - ◆ 3.1.2.1 Conceptual Foundation
 - ◆ 3.1.2.2 Sampling Frame Construction
 - ◆ Algorithm 1: Stratified Player Selection
 - ◆ Algorithm 2: Snowball Match Collection
 - 3.1.2.3 Quota Specifications
 - 3.1.2.4 Comparison with Alternative Sampling Methods
 - 3.1.2.5 Limitations of the Sampling Approach
 - ◆ 3.1.3 Data Collection Technical Implementation
 - ◆ 3.1.3.1 Rate Limiting Infrastructure
 - ◆ 3.1.3.2 Error Handling and Retry Logic
 - ◆ 3.1.3.3 Data Collection Workflow
 - ◆ 3.1.4 Dataset Composition and Structure
 - ◆ 3.1.4.1 Match-Level Data (Team Statistics)
 - ◆ 3.1.4.2 Participant-Level Data (Player Statistics)
 - ◆ 3.1.4.3 Data Quality Considerations
 - ◆ 3.1.5 Ethical Considerations
 - 3.2 Data Preprocessing and Cleaning
 - ◆ 3.2.1 Initial Data Loading and Structural Validation
 - ◆ 3.2.2 Column Removal Strategy
 - ◆ 3.2.3 Data Leakage Prevention
 - ◆ 3.2.4 Missing Value Handling
 - ◆ 3.2.5 Data Type Standardization
 - 3.3 Feature Engineering: Player-Level to Match-Level Transformation

- ◆ 3.3.1 Overview of the Transformation Problem
- ◆ 3.3.2 Team and Position Identification
- ◆ 3.3.3 Target Variable Construction
- ◆ 3.3.4 Pivot Transformation
- ◆ 3.3.5 Team-Level Objective Feature Integration
- ◆ 3.3.6 Final Dataset Assembly
- 3.4 Feature Selection Methodology
 - ◆ 3.4.1 Overview and Rationale
 - ◆ 3.4.2 Stage 1: SULOV Correlation Analysis
 - ◆ 3.4.3 Stage 2: XGBoost Recursive Feature Elimination
 - ◆ 3.4.4 Additional Feature Pruning
- 3.5 Machine Learning Models
 - ◆ 3.5.1 Model Selection Philosophy
 - ◆ 3.5.2 Individual Model Specifications
 - ◆ 3.5.3 Scaling Requirements
 - ◆ 3.5.4 Ensemble Model Construction
- 3.6 Hyperparameter Optimization
 - ◆ 3.6.1 Optimization Framework
 - ◆ 3.6.2 Hyperparameter Search Spaces
 - ◆ 3.6.3 Regression-to-Classification Conversion
- 3.7 Evaluation Metrics
 - ◆ 3.7.1 Metric Selection
 - ◆ 3.7.2 Results Aggregation and Reporting
- 3.8 Stability Analysis
 - ◆ 3.8.1 Stability Analysis Framework
 - ◆ 3.8.2 Model Reinstantiation
 - ◆ 3.8.3 Stability Metrics Calculation
 - ◆ 3.8.4 Stability Visualization
- 3.9 SHAP Analysis for Model Interpretability
 - ◆ 3.9.1 Explainer Selection
 - ◆ 3.9.2 SHAP Visualization Types
 - ◆ 3.9.3 Output and Export
- 4 Implementation and Experimental Setup

- 4.1 Technologies and Tools
 - ◆ 4.1.1 Programming Language and Environment
 - ◆ 4.1.2 Core Libraries and Dependencies
 - ◆ 4.1.3 Development Environment Configuration
 - ◆ 4.1.4 File Path Configuration
- 4.2 Dataset
 - ◆ 4.2.1 Data Sources and Provenance
 - ◆ 4.2.2 Dataset Characteristics at Loading
 - ◆ 4.2.3 Dataset Evolution Through the Pipeline
 - ◆ 4.2.4 Data Quality Characteristics
 - ◆ 4.2.5 Output Dataset Files
- 4.3 Experimental Setup
 - ◆ 4.3.1 Hardware and Computational Environment
 - ◆ 4.3.2 Reproducibility Configuration
 - ◆ 4.3.3 Model Configuration and Hyperparameter Search Design
 - ◆ 4.3.4 Data Partitioning Strategy
 - ◆ 4.3.5 Stability Analysis Protocol
 - ◆ 4.3.6 SHAP Analysis Configuration
 - ◆ 4.3.7 Output and Visualization Configuration
- 5 Results
 - 5.1 Introduction
 - ◆ 5.2 Model Performance Comparison
 - ◆ 5.2.1 Model Performance Results Table
 - ◆ 5.2.2 Explanation of Evaluation Metrics
 - ◆ 5.2.3 Interpretation of Model Performance Results
 - 5.3 Stability Analysis
 - ◆ 5.3.1 Stability Analysis Results
 - ◆ 5.3.2 Purpose and Rationale for Stability Analysis
 - ◆ 5.3.3 Interpretation of Stability Analysis Results
 - 5.4 Feature Importance Analysis
 - ◆ 5.4.1 Feature Importance Rankings
 - ◆ 5.4.2 Purpose and Methodology of Feature Importance Analysis
 - ◆ 5.4.3 Interpretation of Feature Importance Results

- ◆ 5.4.4 Comparison with SHAP Analysis Findings
- 5.5 SHAP Interpretability Analysis
 - ◆ 5.5.1 Overview of SHAP Analysis
 - ◆ 5.5.2 Global Feature Importance: SHAP Bar Plot Analysis
 - ◆ 5.5.3 Directionality and Feature Effects: SHAP Beeswarm Plot Analysis
 - ◆ 5.5.4 Synthesis of Feature Importance Findings
 - ◆ 5.5.5 Methodological Implications
- 5.6 Model Comparison Visualization
 - ◆ 5.6.1 Model Comparison Chart Analysis
 - ◆ 5.6.2 R^2 Score Analysis: Variance Explanation Hierarchy
 - ◆ 5.6.3 Accuracy Analysis: The Classification Success Paradox
 - ◆ 5.6.4 Model Family Performance Patterns
 - ◆ 5.6.5 Synthesis with Stability and Feature Importance Findings
- 5.7 Stability Visualization: Boxplot Analysis
 - ◆ 5.7.1 Overview of Stability Boxplots
 - ◆ 5.7.2 Higher-Is-Better Metrics: Accuracy and R^2
 - ◆ 5.7.3 Lower-Is-Better Metrics: MAE, RMSD, and MAD
 - ◆ 5.7.4 Synthesis: The CatBoost-Gradient Boosting Trade-off
 - ◆ 5.7.5 Outlier Analysis
- 5.8 Selected Features Analysis: Correlation Structure and Final Feature Set
 - ◆ 5.8.1 Selected Features List
 - ◆ 5.8.2 Correlation Structure of Selected Features
 - ◆ 5.8.3 Feature Set Characterization
 - ◆ 5.8.4 Practical Recommendations for Feature Set Refinement
- 5.9 Dataset Summary
 - ◆ 5.9.1 Dataset Characteristics
 - ◆ 5.9.2 Dataset Scale and Composition
 - ◆ 5.9.3 Dimensionality Reduction Magnitude
 - ◆ 5.9.4 Best Model Identification
 - ◆ 5.9.5 Dataset Limitations in Context
 - ◆ 5.9.6 Reproducibility Documentation

- 6 Discussion
 - 6.1 Interpretation of Predictive Feature Patterns
 - ◆ 6.1.1 The Primacy of Structural Objectives
 - ◆ 6.1.2 The Unexpected Importance of Rune Configurations
 - ◆ 6.1.3 Jungler Deaths as the Key Combat Metric
 - ◆ 6.1.4 The Support Role's Predictive Prominence
 - 6.2 Model Performance Characteristics
 - ◆ 6.2.1 The Ensemble Advantage: Theory and Empirical Validation
 - ◆ 6.2.2 The Gradient Boosting Family's Fitness for Gaming Data
 - ◆ 6.2.3 The Ridge Anomaly: Classification Without Calibration
 - ◆ 6.2.4 Stability-Performance Trade-offs
 - 6.3 Practical Implications
 - ◆ 6.3.1 For Players and Coaches
 - ◆ 6.3.2 For Game Developers and Balance Teams
 - ◆ 6.3.3 For Esports Broadcasters and Analysts
 - ◆ 6.3.4 For the Research Community
 - 6.4 Limitations and Threats to Validity
 - ◆ 6.4.1 Internal Validity
 - ◆ 6.4.2 External Validity
 - ◆ 6.4.3 Construct Validity
 - ◆ 6.4.4 Statistical Conclusion Validity
- 7 Conclusion and Future Work
 - 7.1 Summary of Key Contributions
 - ◆ 7.1.1 Methodological Contributions
 - ◆ 7.1.2 Empirical Contributions
 - 7.2 Addressing the Research Questions
 - ◆ 7.2.1 Primary Research Question
 - ◆ 7.2.2 Secondary Research Questions
 - 7.3 Future Work
 - ◆ 7.3.1 Temporal Feature Engineering for In-Game Prediction
 - ◆ 7.3.2 Champion Interaction Modeling
 - ◆ 7.3.3 Deep Learning Approaches for Sequence and Multi-Entity Data

- ◆ 7.3.4 Cross-Patch Transfer Learning
- ◆ 7.3.5 Causal Discovery Methods
- ◆ 7.3.6 Deployment Considerations for Practical Tools
- ◆ 7.3.7 Dataset Expansion and Diversity
- 7.4 Concluding Remarks

1. Introduction

1.1 Background and Motivation

1.1.1 The Rise of Esports and Competitive Gaming Analytics

The global esports industry has experienced exponential growth over the past decade, evolving from niche gaming communities into a multi-billion-dollar entertainment ecosystem. According to industry reports, the global esports market surpassed \$1.38 billion in 2022, with projections indicating continued growth driven by increasing viewership, sponsorship investments, and media rights deals. This transformation has elevated competitive gaming from casual recreation to a professional domain where marginal performance improvements can translate into significant competitive advantages and financial returns.

Within this landscape, the application of data analytics and machine learning has emerged as a critical differentiator. Traditional sports analytics, pioneered by the "Moneyball" revolution in baseball and subsequently adopted across basketball, soccer, and football, demonstrated that data-driven decision-making could identify undervalued strategies and optimize team performance. Esports, by virtue of its inherently digital nature, generates vastly richer and more granular datasets than traditional sports, creating unprecedented opportunities for analytical insights. Every mouse click, ability cast, and positional movement can be recorded, timestamped, and analyzed—a level of detail impossible in physical sports.

1.1.2 League of Legends: A Case Study in Game Complexity

League of Legends (LoL), developed and published by Riot Games in 2009, stands as the most prominent exemplar of the Multiplayer Online Battle Arena (MOBA) genre and one of the most played video games globally. With over 180 million monthly active players as of 2023, a professional esports scene spanning multiple continents, and annual World Championship events drawing viewership comparable to traditional sporting finals, LoL represents both a cultural phenomenon and a rich domain for analytical research.

The fundamental structure of a League of Legends match involves two teams of five players competing on a symmetrical map called Summoner's Rift. Each player selects a unique champion from a roster exceeding 160 options, with each champion possessing distinct abilities, strengths, and strategic roles. The game's complexity emerges from multiple interacting systems:

Strategic Role Differentiation: Players assume one of five standardized positions, each with distinct responsibilities and spatial domains on the map. The Top lane typically features durable champions in an isolated laning phase. The Jungle role exerts map-wide influence through

objective control and lane interventions. The Middle lane emphasizes central map dominance and roaming pressure. The Bottom lane carries provide sustained damage output for late-game scaling. The Support role focuses on vision control, ally protection, and engagement setup. This positional structure creates specialized performance profiles that must be captured in any predictive modeling approach.

Economic and Experience Systems: Players accumulate gold and experience throughout the match by killing enemy units, destroying structures, and securing objectives. This creates dynamic power differentials that evolve continuously, requiring teams to make strategic trade-offs between immediate gains and long-term scaling.

Objective-Based Gameplay: Beyond direct player-versus-player combat, matches revolve around controlling neutral objectives—Dragon, Baron Nashor, Rift Herald, and turret structures—each providing team-wide benefits that compound strategic advantages. The introduction of elemental dragon souls and Elder Dragon buffs has further amplified the importance of objective control.

Asymmetric Information and Vision Mechanics: Teams operate under partial information, with the "fog of war" obscuring enemy positions unless revealed through wards, abilities, or direct line-of-sight. This creates a sophisticated meta-game of information warfare, where vision control enables safer objective captures and ambush opportunities.

Compositional Synergies and Counter-Strategies: With champion selection occurring through an alternating draft phase, teams must balance synergistic team compositions against counter-picks to enemy selections. This pre-game strategic layer introduces additional predictive complexity.

The combinatorial complexity of these interacting systems makes League of Legends an ideal testbed for machine learning approaches. A single match generates thousands of data points spanning individual mechanical execution, team coordination, resource allocation, and strategic decision-making—creating a high-dimensional prediction problem that challenges conventional analytical methods.

1.1.3 The Analytical Opportunity in Post-Game Data

Riot Games provides extensive match data through their official Application Programming Interface (API), enabling researchers and developers to access structured information about completed matches. This data encompasses:

- **Match metadata:** Duration, patch version, queue type, region
- **Team-level statistics:** Objectives secured, bans, total gold, total kills
- **Participant-level statistics:** Per-player metrics including kills, deaths, assists, damage dealt, damage taken, gold earned, vision score, crowd control score, and dozens of

additional performance indicators

- **Timeline data:** Event-level records capturing game state at one-minute intervals

The availability of this comprehensive post-game data creates a unique opportunity for retrospective analysis. Unlike in-game prediction systems that must operate under real-time constraints with incomplete information, post-game analysis can leverage the full spectrum of match statistics to identify the features most strongly associated with victory. This knowledge can then inform pre-game preparation, in-game decision frameworks, and post-game review processes.

1.1.4 Practical Applications of Win Prediction Models

The development of accurate win prediction models for League of Legends serves multiple stakeholder communities:

For Professional Teams and Coaching Staff:

- Identification of key performance indicators (KPIs) that distinguish winning from losing performances
- Evidence-based prioritization of practice drills and strategic preparation
- Objective evaluation of player performance beyond traditional metrics like KDA (Kills/Deaths/Assists)
- Scouting and recruitment based on metrics correlated with winning impact

For Individual Players:

- Personalized feedback on which aspects of gameplay most influence match outcomes
- Data-driven goal setting for skill improvement

For Game Developers and Balance Teams:

- Empirical assessment of whether intended game balance is reflected in outcome predictors
- Detection of unintended gameplay patterns that disproportionately influence match results
- Validation of design changes through shifts in predictive feature importance

For Broadcasting and Spectator Experience:

- Enhanced statistical overlays highlighting impactful in-game events
- Post-match analysis segments grounded in quantitative evidence
- Historical comparisons contextualizing current performances

For the Research Community:

- Methodological contributions to feature engineering in multi-entity competitive environments

- Benchmarking of machine learning approaches on complex, real-world datasets
- Interpretable models that bridge the gap between prediction accuracy and domain understanding

1.1.5 Gaps in Existing Literature

Despite the growing body of research on League of Legends analytics, several gaps persist in the current literature:

1. **Limited role-aware feature engineering:** Many studies aggregate player statistics across teams without preserving position-specific information, losing the strategic nuance of how different positions contribute to match outcomes.
2. **Insufficient attention to feature selection rigor:** The high dimensionality of match data necessitates principled feature reduction, yet many approaches rely on ad hoc selection or single-method importance ranking.
3. **Under-exploration of ensemble methods:** While individual machine learning models have been evaluated, systematic comparison of ensemble strategies remains limited.
4. **Inadequate stability validation:** Single train-test splits dominate evaluation protocols, leaving questions about model robustness across different data partitions unanswered.
5. **Limited interpretability analysis:** Prediction accuracy is often prioritized over understanding *why* predictions are made, reducing practical utility for stakeholders who need actionable insights.

Our research directly addresses these gaps through a comprehensive methodology that emphasizes position-preserving feature engineering, multi-stage feature selection, ensemble model evaluation, extensive stability analysis, and SHAP-based interpretability.

1.2 Problem Statement

Despite the wealth of data generated by League of Legends matches and the growing sophistication of machine learning techniques, a significant gap exists between the *availability* of post-game statistics and the *actionable extraction* of predictive insights. Specifically, the research community and practitioner ecosystem face the following interconnected challenges:

Challenge 1: Dimensionality and Structure Complexity. A single League of Legends match produces data distributed across 10 individual participant records, each containing dozens of performance metrics, alongside team-level objective statistics. The transformation of this multi-entity, hierarchically structured data into a format amenable to machine learning while preserving player-position information is non-trivial. Naive aggregation approaches risk obscuring the individual performance dynamics that define MOBA gameplay.

Challenge 2: Feature Redundancy and Selection. The extensive suite of metrics provided by Riot's API includes many features that are highly correlated—either because they measure related constructs (e.g., total damage and physical damage) or because they represent complementary perspectives on the same underlying phenomena (e.g., gold earned and items purchased). Identifying the minimal set of features that maximizes predictive power without sacrificing interpretability requires principled selection methodologies.

Challenge 3: Model Selection and Optimization. The machine learning literature offers numerous candidate algorithms for binary classification tasks, each with distinct assumptions, strengths, and weaknesses. Determining which approaches—and which combinations through ensemble methods—yield optimal performance for League of Legends win prediction requires systematic empirical comparison under controlled evaluation protocols.

Challenge 4: Interpretability-Accuracy Trade-off. High-performing models such as gradient boosting ensembles and neural networks often function as "black boxes," providing predictions without explanations. For domain stakeholders—coaches, players, analysts—understanding *which* features drive predictions and *how* they influence outcomes is as important as prediction accuracy itself.

Challenge 5: Robustness and Generalizability. Models trained on specific data partitions may exhibit performance variance when applied to unseen data. Without systematic stability analysis, reported performance metrics may overstate real-world reliability.

Formal Problem Statement: This research addresses the problem of developing, optimizing, and interpreting machine learning models for predicting League of Legends match outcomes from post-game data, with explicit attention to position-aware feature engineering, principled feature selection, ensemble model evaluation, stability validation, and interpretability analysis.

1.3 Research Questions

To address the identified problem, this study investigates one primary research question and four secondary research questions:

Primary Research Question

RQ1: To what extent can machine learning models accurately predict the winning team in League of Legends ranked matches using post-game player statistics, team objective data, and position-preserving feature engineering?

This primary question establishes the overarching predictive objective and provides a benchmark against which model performance can be evaluated.

Secondary Research Questions

RQ2 (Feature Importance): Which post-game features—including individual player statistics, champion selections, team-level objectives, and position-specific performance metrics—exhibit the strongest predictive power for match outcomes?

This question addresses Challenge 4 (Interpretability) by identifying the specific game elements that most reliably distinguish winning from losing performances.

RQ3 (Algorithm Comparison): How do different machine learning algorithms—including tree-based ensembles, instance-based learners, support vector machines, linear models, and neural networks—compare in their predictive performance for this task?

This question addresses Challenge 3 (Model Selection) through systematic empirical comparison.

RQ4 (Ensemble Effectiveness): To what extent do ensemble strategies (bagging and voting) improve predictive performance over the best-performing individual models?

This question explores whether model combination yields additive benefits for win prediction.

RQ5 (Model Stability): How robust are the optimized models to variations in training data, as assessed through repeated random sampling and cross-validation?

This question addresses Challenge 5 (Generalizability) by quantifying performance variance across data partitions.

1.4 Research Objectives

Aligned with the research questions above, this study pursues the following specific objectives:

Primary Objective

O1: Develop and validate a comprehensive machine learning pipeline capable of predicting League of Legends match outcomes with accuracy significantly exceeding random chance (50% baseline).

Secondary Objectives

O2 (Feature Engineering Framework): Design and implement a position-aware feature engineering methodology that transforms participant-level match data into structured match-level feature matrices while preserving individual player information through systematic team-position prefixing.

O3 (Feature Selection Protocol): Apply a two-stage feature selection process combining SULOV (Searching for and Removing Unnecessary Variables) correlation analysis for redundancy elimination with XGBoost recursive feature elimination for predictive ranking, reducing the feature space to an optimal subset of the top 15 features.

O4 (Comprehensive Model Evaluation): Train, optimize, and compare eight distinct regression models (RF, XGB, CatB, GradB, KNNR, SVR, Ridge, MLPR) and two ensemble approaches (Bagging, Voting) using GridSearchCV hyperparameter optimization and multiple evaluation metrics (R^2 , Accuracy, MAE, RMSD, MAD).

O5 (Stability Validation): Conduct 100-iteration stability analysis with randomized data splits to compute mean performance metrics and standard deviations, establishing confidence bounds for model reliability.

O6 (Interpretability Analysis): Apply SHAP (SHapley Additive exPlanations) analysis to the best-performing model to quantify feature importance, visualize feature effect directions, and provide interpretable insights for domain stakeholders.

O7 (Open-Source Contribution): Release the complete implementation code, including data preprocessing, feature engineering, model training, and analysis scripts, as open-source software to facilitate reproducibility and enable extension by the research community.

1.5 Scope and Limitations

1.5.1 Scope of the Research

This research is scoped to include the following elements:

Data Scope:

- Post-game match data collected through the Riot Games API for ranked League of Legends matches
- Participant-level statistics (individual player performance metrics)
- Team-level objective data (barons, dragons, towers, inhibitors, etc.)
- Champion selection and position assignment information

Methodological Scope:

- Binary classification task: predicting whether Team 1 (arbitrarily assigned) wins the match
- Regression-based approach with threshold conversion to classification
- Feature engineering focused on position-preserving transformation from participant-level to match-level data

- Feature selection using SULOV correlation analysis and XGBoost importance ranking
- Supervised machine learning models implemented in scikit-learn, XGBoost, and CatBoost frameworks
- Hyperparameter optimization via exhaustive grid search with cross-validation
- Model interpretability analysis using SHAP values

Output Scope:

- Quantitative performance metrics for all evaluated models
- Feature importance rankings from SHAP analysis
- Stability confidence intervals from 100-iteration validation
- Visualization of model comparisons, feature correlations, and SHAP explanations
- Open-source code repository

1.5.2 Limitations

The following limitations bound the interpretation and generalizability of our findings:

Data Limitations:

Limitation	Description	Impact
Post-Game Data Only	Models use complete match statistics available only after match conclusion	Results do not directly support real-time or pre-game prediction
Ranked Match Context	Data drawn from ranked solo/duo queue	Findings may not generalize to professional play, normal games, or alternate game modes
Patch Version Specificity	League of Legends undergoes bi-weekly balance updates	Feature importances and model performance may shift with meta-game evolution
Region Homogeneity	Data may represent limited geographic regions	Regional meta-game variations could affect transferability
Rank Tier Distribution	Skill level distribution in the dataset may not be uniform	Model predictions may be more or less accurate at specific skill tiers
Sample Size Constraints	Dataset size bounded by API rate limits and collection period	Very rare champion picks or strategies may be underrepresented
API Data Limitations	Riot API provides processed statistics, not raw event streams	Certain granular gameplay aspects (e.g., precise ability sequencing) are not captured

Methodological Limitations:

Limitation	Description	Impact
Correlation vs. Causation	Feature importance reflects predictive association, not causal influence	Features correlated with winning may be consequences rather than causes
Binary Team Assignment	T1/T2 assignment is arbitrary (based on teamId ordering)	Target variable is symmetric; models must learn this symmetry
Feature Selection Thresholds	SULOV correlation threshold (0.81) and top-N selection (15 features) are design choices	Different thresholds could yield different feature subsets
Regression-to-Classification Conversion	Using 0.5 threshold on regression outputs for classification	May not be optimal for all models; calibration could improve results
Hyperparameter Search Granularity	Grid search covers limited hyperparameter ranges	More extensive search could identify better configurations
Model Coverage	Eight regression models evaluated; deep learning approaches (LSTM, Transformers) excluded	Results may not represent the full state-of-the-art

Interpretation Limitations:

Limitation	Description	Impact
Aggregate Feature Importance	SHAP values represent average effects across the test set	Individual match dynamics may differ from aggregate patterns
Patch Stability	Analysis conducted on specific game patch version	Feature-outcome relationships may shift with game updates

1.5.3 Delimitations (Explicit Exclusions)

The following areas are explicitly excluded from this research:

- **Real-time prediction:** No attempt is made to predict outcomes from partial in-game data
- **Pre-game prediction:** Champion select and draft analysis are not within scope

- **Player identification:** Models do not incorporate player identity or historical performance
 - **Temporal analysis:** Timeline data and event sequences are not analyzed
 - **Causal inference:** No causal claims are made about feature-outcome relationships
 - **Commercial deployment:** Production-ready systems are not developed
 - **Professional play analysis:** Competitive/professional matches are not specifically studied
 - **Role hierarchy analysis:** Relative importance of different positions is not formally ranked or analyzed as a separate research objective
-

1.6 Thesis Organization

The remainder of this research paper is organized as follows:

Chapter 2: Literature Review — Surveys existing research on MOBA game analytics, League of Legends win prediction, feature selection methodologies in gaming contexts, and machine learning interpretability techniques. This chapter contextualizes our contributions within the broader academic landscape.

Chapter 3: Methodology — Provides a comprehensive description of our research approach, including:

- Data acquisition procedures and dataset characteristics
- Preprocessing and cleaning protocols
- Position-aware feature engineering pipeline (participant-level to match-level transformation)
- Two-stage feature selection methodology (SULOVS + XGBoost RFE)
- Machine learning model specifications and hyperparameter grids
- Evaluation metrics and validation protocols
- Stability analysis framework
- SHAP interpretability analysis approach

Chapter 4: Experimental Setup and Implementation — Documents the technical implementation details, including:

- Software environment and library dependencies
- Data loading and preprocessing workflows
- Feature engineering implementation
- Model training and optimization procedures
- Computational resource requirements

Chapter 5: Results — Presents empirical findings organized by research question:

- Model performance comparison across all evaluated algorithms
- Feature importance rankings from SHAP analysis
- Ensemble method effectiveness
- Stability analysis with mean and standard deviation metrics
- SHAP interpretation results with feature effect visualizations

Chapter 6: Discussion — Interprets results within the context of existing literature and domain knowledge:

- Analysis of predictive feature patterns and their alignment with game design
- Model performance characteristics and algorithm selection guidance
- Practical implications for players, coaches, and analysts
- Limitations and threats to validity

Chapter 7: Conclusion and Future Work — Summarizes key contributions, addresses research questions, and proposes directions for future investigation, including:

- Temporal feature engineering for in-game prediction
- Champion interaction modeling
- Deep learning approaches for sequence data
- Cross-patch transfer learning
- Causal discovery methods
- Deployment considerations for practical tools

Appendices — Contain supplementary material including:

- Complete hyperparameter search results
- Detailed feature lists and descriptions
- Additional visualizations
- Code repository information and usage instructions

2. Background and Literature Review

2.1 Background: Foundational Concepts and Theoretical Framework

2.1.1 League of Legends as a Complex Adaptive System

League of Legends can be conceptualized as a complex adaptive system characterized by emergent behaviors arising from the interactions of multiple autonomous agents (players)

operating under constrained rules within a dynamic environment. This theoretical framing is essential for understanding why win prediction presents unique analytical challenges beyond those encountered in simpler classification tasks.

Game Structure and Mechanics:

A standard League of Legends match unfolds on Summoner's Rift, a symmetrical map featuring three primary lanes (Top, Middle, Bottom) connected by a jungle region containing neutral monsters and objectives. Each team's base contains a Nexus structure whose destruction determines victory. The game progresses through distinct phases, each characterized by different strategic priorities and player interactions:

Game Phase	Approximate Duration	Key Activities	Predictive Features
Early Game (Laning)	0-15 minutes	Individual lane matchups, jungle pathing, first objectives	CS differentials, early kills, jungle proximity
Mid Game	15-25 minutes	Rotations, skirmishes, dragon/rift herald contests	Objective control, tower differentials, team fight outcomes
Late Game	25+ minutes	Baron contests, siege scenarios, elder dragon fights	Team composition scaling, vision control, pick potential

The phase-dependent nature of gameplay means that features predictive of victory may vary in importance throughout a match—a consideration that, while beyond our post-game prediction scope, informs our interpretation of which features emerge as most important in complete-game analysis.

The Draft Phase and Champion Selection:

Before gameplay begins, teams alternate selecting and banning champions through a structured draft. This pre-game strategic layer introduces compositional considerations that influence match dynamics:

- **Team composition archetypes:** Compositions may emphasize early-game aggression (snowball), late-game scaling, pick-oriented play, teamfight coordination (wombo-combo), or split-push pressure
- **Counter-pick dynamics:** Certain champion matchups favor specific lanes, creating asymmetric advantages

- **Synergy effects:** Champions with complementary abilities can create multiplicative effectiveness beyond individual performance

While our research focuses on post-game features and does not explicitly model draft information, understanding these compositional factors contextualizes why certain performance metrics (particularly those reflecting successful execution of a composition's intended strategy) may exhibit strong predictive power.

The Economic Snowball and Feedback Loops:

League of Legends features powerful positive feedback mechanisms that amplify early advantages:

1. **Kill gold and experience:** Eliminating enemy champions provides direct resource rewards
2. **Objective rewards:** Securing turrets, dragons, and Baron Nashor provides team-wide gold and buffs
3. **Map control cascades:** Vision denial and objective control restrict enemy movement options, enabling further advantage accumulation
4. **Item power spikes:** Gold advantages translate into earlier completion of powerful items, increasing combat effectiveness

These feedback loops suggest that early-game performance metrics may exhibit outsized predictive importance, as small initial advantages can compound into decisive leads. However, comeback mechanics (shutdown gold on killing sprees, scaling champion power curves, elder dragon execution thresholds) introduce countervailing dynamics that can reverse apparent certainties.

2.1.2 Machine Learning Paradigms for Binary Outcome Prediction

Our research employs supervised machine learning for binary classification (win/loss) using a regression-based approach. Understanding the theoretical underpinnings of the algorithms employed contextualizes our model selection decisions.

Tree-Based Ensemble Methods:

Random Forests (Breiman, 2001) construct multiple decision trees on bootstrap samples of the training data, with random feature subset selection at each split. The ensemble prediction aggregates individual tree outputs through averaging (regression) or voting (classification). Random Forests provide several advantages for our application:

- **Nonlinear relationship capture:** Decision boundaries can model complex interactions between features without explicit specification

- **Feature importance quantification:** Built-in importance metrics based on impurity reduction or permutation
- **Robustness to outliers:** Ensemble averaging reduces sensitivity to anomalous training instances
- **Limited hyperparameter sensitivity:** Reasonable default configurations often yield competitive performance

Gradient Boosting (Friedman, 2001) constructs trees sequentially, with each new tree trained to predict the residuals of the current ensemble. This sequential error correction typically yields higher predictive accuracy than Random Forests at the cost of greater sensitivity to hyperparameter choices and potential overfitting. XGBoost (Chen & Guestrin, 2016) and CatBoost (Prokhorenkova et al., 2018) extend gradient boosting with regularization, efficient implementation, and (in CatBoost's case) native handling of categorical features.

Support Vector Regression:

SVR (Drucker et al., 1996) maps input features to a higher-dimensional space where linear regression is performed, with an ϵ -insensitive loss function that ignores errors within a margin. The kernel trick enables implicit high-dimensional mapping without explicit computation. SVR's theoretical appeal lies in its convex optimization formulation (guaranteeing global optimum) and regularization framework that balances model complexity against training error.

Instance-Based and Linear Methods:

K-Nearest Neighbors predicts outcomes based on the average of the k most similar training instances, where similarity is typically defined by Euclidean distance in feature space. KNNR's non-parametric nature avoids distributional assumptions but suffers from the curse of dimensionality and computational cost at inference time.

Ridge Regression applies L2 regularization to linear regression, shrinking coefficient magnitudes to reduce variance at the cost of introducing bias. While simple, Ridge provides a well-regularized baseline against which more complex models can be compared.

Neural Networks:

Multi-Layer Perceptrons consist of interconnected layers of neurons with nonlinear activation functions, trained through backpropagation. While capable of approximating arbitrary functions, MLPs require careful architecture design, regularization, and hyperparameter tuning to achieve competitive performance.

Ensemble Strategies:

Bagging (Bootstrap Aggregating) reduces variance by training base estimators on bootstrap samples and averaging predictions. When applied to strong base learners, bagging can

improve stability without requiring diverse model types.

Voting combines predictions from heterogeneous models (potentially of different algorithmic families) through averaging. The theoretical justification rests on the assumption that different models make uncorrelated errors, such that their combination reduces overall error.

2.1.3 Feature Selection Theory and Methodology

Feature selection addresses the curse of dimensionality by identifying a subset of features that maximizes predictive performance while minimizing redundancy. The theoretical framework relevant to our approach includes:

Filter Methods evaluate features independently of any learning algorithm based on statistical properties:

- *Correlation-based selection*: Features highly correlated with the target and minimally correlated with each other are preferred
- *Variance thresholding*: Low-variance features are removed as uninformative

Wrapper Methods evaluate feature subsets using a specific learning algorithm's performance:

- *Recursive Feature Elimination (RFE)*: Iteratively removes the least important features based on model-derived importance scores

Embedded Methods perform feature selection during model training:

- *L1 regularization*: Drives irrelevant feature coefficients to zero
- *Tree-based importance*: Feature usage in splits provides natural importance ranking

Our SULO V approach functions as a filter method (removing redundant features before modeling), while XGBoost RFE acts as a wrapper/embedded hybrid, leveraging model-derived importance for final selection.

2.1.4 Model Interpretability and SHAP

SHAP (SHapley Additive exPlanations), introduced by Lundberg and Lee (2017), provides a unified framework for interpreting model predictions based on cooperative game theory. The Shapley value for a feature represents its average marginal contribution to the prediction across all possible feature subsets.

SHAP values satisfy desirable properties including local accuracy (predictions decompose into feature contributions plus baseline), missingness (absent features receive zero attribution), and consistency (if a feature's contribution increases, its SHAP value does not decrease). For tree-based models, TreeExplainer computes exact SHAP values efficiently, making it particularly suitable for interpreting our best-performing models.

2.2 Literature Review: Critical Analysis of Prior Work

This section provides a critical examination of existing research on League of Legends win prediction, champion recommendation, and related analytical approaches. For each study, we summarize methodology, key findings, and limitations that our research addresses.

2.2.1 Win Prediction Using Machine Learning Approaches

Chowdhury, Ahsan, and Barraclough (2024): Applications of Linear and Ensemble-Based Machine Learning for Predicting Winning Teams in League of Legends

Methodology and Approach: This study investigated the application of both linear and ensemble-based machine learning models for predicting winning teams in League of Legends. The authors employed Logistic Regression as a linear baseline alongside ensemble methods including Random Forest and Gradient Boosting. Feature engineering focused on aggregating player statistics to team-level features.

Key Findings: Ensemble methods significantly outperformed linear approaches, with Gradient Boosting achieving the highest accuracy. The study identified team-level aggregates of kills, deaths, and gold as primary predictors. The authors emphasized the superiority of non-linear models for capturing complex game dynamics.

Critical Assessment and Identified Gaps: While this study provides valuable evidence for ensemble method effectiveness, several limitations constrain its applicability:

1. **Feature Engineering Simplicity:** Team-level aggregation without position-specific differentiation discards individual performance information critical to understanding how different players contribute to victory
2. **Limited Feature Selection:** The study lacked a systematic, multi-stage feature selection protocol, potentially retaining redundant features
3. **Absence of Stability Analysis:** Single train-test evaluation provides no insight into model robustness across data partitions
4. **No Interpretability Framework:** Feature importance was reported from model coefficients but without formal SHAP analysis

Our Contribution Relative to This Work: We extend this research by implementing position-preserving feature engineering, applying rigorous two-stage feature selection, conducting 100-iteration stability analysis, and providing SHAP-based interpretability—addressing each identified gap systematically.

Bahrololloomi, Klonowski, Sauer, Horst, and Dörner (2023): E-Sports Player Performance Metrics for Predicting the Outcome of League of Legends Matches Considering Player Roles

Methodology and Approach: This research is among the most directly relevant to our work, as it explicitly considered player positions in the prediction framework. The authors collected match data with position assignments and developed position-specific feature sets. They evaluated multiple machine learning models including Random Forest, XGBoost, and neural networks.

Key Findings: The study demonstrated that incorporating position information improved prediction performance compared to position-agnostic approaches. The authors found that XGBoost achieved the highest performance among evaluated models. Position-specific features contributed meaningfully to prediction accuracy.

Critical Assessment and Identified Gaps:

1. **Feature Selection Approach:** While position-aware, the feature selection methodology relied primarily on model-derived importance without preliminary correlation analysis, potentially retaining multicollinear features
2. **Limited Model Diversity:** The ensemble comparison was limited; bagging and voting ensembles were not systematically evaluated
3. **Interpretability Depth:** Feature importance was reported but without the granular, instance-level explanations that SHAP provides
4. **Stability Evaluation:** Cross-validation was employed but without the extensive iteration count (100) that provides robust confidence intervals

Our Contribution Relative to This Work: We adopt and extend the position-aware paradigm by combining it with SULOV correlation analysis for rigorous redundancy elimination, expanding the model comparison to include ensemble strategies, implementing comprehensive SHAP analysis, and conducting more extensive stability validation.

Turkish MOBA Study (2023): Machine Learning Models on MOBA Gaming: League of Legends Winner Prediction

Methodology and Approach: This study, published with dual English-Turkish titling, applied multiple machine learning techniques to League of Legends winner prediction. The research evaluated decision trees, support vector machines, and ensemble classifiers on match data with standard feature aggregation.

Key Findings: The study confirmed that ensemble tree-based methods outperformed individual classifiers and linear approaches. Feature importance analysis identified gold differential and

objective control as primary predictors, aligning with findings from other studies.

Critical Assessment and Identified Gaps:

1. **Methodological Detail:** The study provided limited detail on feature engineering pipelines, making replication difficult
2. **Position Consideration:** Player positions were not explicitly incorporated into feature construction
3. **Evaluation Depth:** Performance was assessed primarily through accuracy without the multi-metric approach (R^2 , MAE, RMSD, MAD) that captures different aspects of predictive quality

Our Contribution Relative to This Work: We provide comprehensive methodological documentation (facilitated by open-source code release), implement position-aware feature engineering, and employ multi-metric evaluation for richer performance characterization.

Liang (2023): Research on Prediction of the Game Winner Based on Artificial Intelligence Methods

Methodology and Approach: This Columbia University study applied artificial intelligence methods to game winner prediction, examining both traditional machine learning and deep learning approaches. The research compared logistic regression, random forests, and neural network architectures.

Key Findings: Deep learning approaches showed marginal improvements over tree-based methods, though at significantly higher computational cost and reduced interpretability. The study highlighted the trade-off between predictive performance and model transparency.

Critical Assessment and Identified Gaps:

1. **Interpretability Sacrifice:** The deep learning approaches, while performant, provided limited insight into which game features drove predictions
2. **Computational Efficiency:** Training requirements for neural architectures may limit practical applicability
3. **Feature Engineering:** Position-specific features were not emphasized

Our Contribution Relative to This Work: We prioritize interpretable models (tree-based ensembles with SHAP analysis) that balance predictive performance with actionable insights, demonstrating that competitive accuracy need not come at the cost of transparency.

Junior and Campelo (2023): League of Legends: Real-Time Result Prediction

Methodology and Approach: Unlike the post-game prediction focus of most studies, this research addressed real-time result prediction using in-game data available at various match timestamps. The authors developed models that update win probability estimates as matches progress, using features extracted from game timeline data.

Key Findings: Prediction accuracy increased substantially as matches progressed, with early-game predictions (5-10 minutes) achieving modest accuracy (60-65%) while late-game predictions (25+ minutes) approached 90%+ accuracy. Gold difference and experience difference were the strongest real-time predictors.

Critical Assessment and Identified Gaps:

1. **Different Problem Setting:** Real-time prediction addresses a distinct use case from our post-game analysis focus
2. **Data Granularity Requirements:** Timeline data availability limits dataset size compared to summary statistics
3. **Feature Set Constraints:** Real-time models cannot access post-game statistics, potentially missing important predictive signals

Our Contribution Relative to This Work: While operating in a different prediction context (post-game vs. real-time), our research complements this work by identifying which complete-game features are most predictive, potentially informing which in-game metrics should be prioritized for real-time tracking.

Chinese Machine Learning Study (2022): Research on Winning Rate Prediction of E-Sport League of Legends Based on Machine Learning

Methodology and Approach: This study, published in a Chinese-language venue, applied machine learning to predict winning rates in League of Legends. The research employed gradient boosting and ensemble methods with feature engineering focused on team-level statistical aggregates.

Key Findings: The study demonstrated that machine learning models could predict match outcomes with accuracy exceeding 75%. Feature analysis identified early-game performance indicators as particularly influential.

Critical Assessment and Identified Gaps:

1. **Language Accessibility:** Publication in Chinese limits accessibility for international researchers

2. **Feature Granularity:** Position-level feature differentiation was not explored
3. **Reproducibility:** Limited implementation detail hinders replication

Our Contribution Relative to This Work: Our English-language publication with comprehensive documentation and open-source code directly addresses accessibility and reproducibility concerns.

2.2.2 Champion Selection and Recommendation Systems

Błaszczyk and Szajerman (2023): Champion Recommendation in League of Legends Using Machine Learning

Methodology and Approach: This study addressed the complementary problem of champion recommendation rather than outcome prediction. The authors developed machine learning models to suggest optimal champion picks based on team composition, opponent picks, and player characteristics. They employed collaborative filtering and content-based recommendation approaches adapted from traditional recommender systems.

Key Findings: The recommendation system achieved meaningful improvement over random champion selection in simulated win rate testing. Champion synergy and counter-pick relationships emerged as critical factors in recommendation quality.

Critical Assessment and Relevance to Our Work:

1. **Different Objective:** The study focuses on pre-game champion selection rather than post-game outcome prediction
2. **Complementary Insights:** Findings about champion interaction importance inform our understanding of why position-specific features matter—different champions in the same position may have dramatically different win conditions
3. **Feature Engineering Parallels:** The emphasis on structured representation of game entities (champions, positions) parallels our position-preserving approach

Our Contribution Relative to This Work: While not directly comparable due to different objectives, our work complements this research by providing post-game validation of which performance metrics matter most, potentially informing which champion attributes recommendation systems should prioritize.

Do, Wang, Yu, McMillian, and McMahan (2021): Using Machine Learning to Predict Game Outcomes Based on Player-Champion Experience in League of Legends

Methodology and Approach: This research investigated whether player-champion experience (measured by mastery points and games played on specific champions) could predict match outcomes. The authors collected player-specific champion experience data and incorporated it as features in machine learning models alongside traditional performance metrics.

Key Findings: Player-champion experience showed statistically significant predictive power, though its effect size was smaller than in-game performance metrics. The combination of experience features with performance data improved prediction accuracy compared to performance data alone.

Critical Assessment and Relevance to Our Work:

1. **Player-Specific Data Requirement:** This approach requires player identity tracking, which may not be available in all datasets and raises privacy considerations
2. **External Validity:** Experience effects may vary by skill tier (diminishing returns at high mastery) and by champion complexity
3. **Feature Interaction:** The study highlights the importance of considering player-specific factors beyond aggregate statistics

Our Contribution Relative to This Work: Our research focuses on anonymous match-level prediction without player identification, making our approach applicable in contexts where player identity data is unavailable.

2.2.3 Esports Analytics and Performance Metrics

Maymin (2021): Smart Kills and Worthless Deaths: Esports Analytics for League of Legends

Methodology and Approach: Maymin introduced a sophisticated framework for distinguishing between "smart" and "worthless" kills and deaths based on their game context rather than raw counts. The research developed context-aware performance metrics that weighted actions by their strategic impact, considering factors such as objective proximity, team member presence, and game phase.

Key Findings: Context-weighted performance metrics showed higher correlation with match outcomes than raw kill/death/assist counts. The research demonstrated that not all kills (or deaths) are equally impactful, challenging the common reliance on aggregate KDA ratios.

Critical Assessment and Relevance to Our Work:

1. **Metric Innovation:** The concept of context-aware metrics represents an important advancement beyond raw statistics

2. **Computational Complexity:** Calculating context-weighted metrics requires detailed timeline data, limiting applicability to API-accessible datasets
3. **Feature Engineering Implications:** Our feature set includes both raw metrics and their aggregates; context weighting could potentially improve predictive power

Our Contribution Relative to This Work: Our SHAP analysis reveals which specific features drive predictions, potentially validating or challenging Maymin's assertions about which game events are most impactful. Future work could incorporate context-weighted features into our pipeline to assess their marginal predictive value.

2.2.4 Team Composition and Optimization

Hybrid Approach Study (2021): A Hybrid Approach to Build Automatic Team Composition in League of Legends

Methodology and Approach: This research addressed the pre-game problem of automatic team composition construction. The authors developed a hybrid system combining collaborative filtering (for champion synergy prediction) with rule-based constraints (for position balance) to generate viable team compositions.

Key Findings: The hybrid approach generated compositions that outperformed random assembly in simulated win rate testing. Champion synergy modeling improved composition quality beyond simple position-balancing heuristics.

Critical Assessment and Relevance to Our Work:

1. **Pre-Game vs. Post-Game:** Team composition optimization addresses the selection problem, while our work addresses the outcome prediction problem
 2. **Methodological Cross-Pollination:** The hybrid approach (combining data-driven and rule-based methods) offers a template for future integration of domain knowledge into our prediction framework
-

Genetic Algorithm Study (2020): An Approach for Team Composition in League of Legends Using Genetic Algorithm

Methodology and Approach: This study applied genetic algorithms to the team composition optimization problem. The authors encoded team compositions as chromosomes with genes representing champion selections, using fitness functions based on historical win rates and synergy scores.

Key Findings: Genetic algorithms efficiently explored the enormous composition space (160+ champions across 5 positions) to identify viable team compositions. Evolution over generations converged to compositions with strong synergy characteristics.

Critical Assessment and Relevance to Our Work:

1. **Optimization, Not Prediction:** The genetic algorithm approach solves a different problem (composition selection rather than outcome prediction)
 2. **Meta-Evolution Implications:** Composition effectiveness evolves with game patches; our prediction models face similar challenges with meta-shift robustness
-

2.2.5 Data Summarization and Reinforcement Learning Applications

Kim, Lee, and Park (2023): Structured Summarization of League of Legends Match Data Optimized for Large Language Model Input

Methodology and Approach: This forward-looking study explored how League of Legends match data could be structured for input to Large Language Models (LLMs). The authors developed summarization pipelines that converted match statistics into natural language descriptions optimized for LLM processing.

Key Findings: Structured summarization enabled LLMs to perform qualitative match analysis tasks including performance evaluation and strategic commentary. The research demonstrated the potential of combining traditional ML with emerging LLM capabilities.

Critical Assessment and Relevance to Our Work:

1. **Emerging Paradigm:** LLM-based analysis represents a complementary rather than competing approach to traditional ML prediction
2. **Feature Representation:** The challenge of representing match data for LLMs parallels our feature engineering challenge—both require thoughtful transformation of raw data
3. **Interpretability Convergence:** LLMs offer natural language explanations that could complement our SHAP-based quantitative explanations

Our Contribution Relative to This Work: Our quantitative prediction and SHAP analysis could provide structured input for LLM-based match analysis systems, creating a pipeline where ML predictions inform natural language explanations.

Dynamic Difficulty Adjustment Study (2023): Implementation and Effect Evaluation of Dynamic Difficulty Adjustment Based on Reinforcement Learning in MOBA Games

Methodology and Approach: This study explored reinforcement learning applications for dynamic difficulty adjustment in MOBA games. The authors developed RL agents that modulated game parameters in response to player performance to maintain engagement-optimal challenge levels.

Key Findings: RL-based difficulty adjustment improved player engagement metrics compared to static difficulty settings. The research demonstrated RL's applicability to MOBA-specific game dynamics.

Critical Assessment and Relevance to Our Work:

- 1. **Different Application:** Difficulty adjustment addresses game design rather than outcome prediction
- 2. **RL Methodology:** Reinforcement learning approaches could potentially be applied to win prediction as a sequential decision problem
- 3. **Feature Dynamics:** Understanding how game difficulty parameters affect performance metrics could inform our interpretation of feature importances

2.3 Synthesis of Literature Gaps and Our Contributions

2.3.1 Summary of Identified Gaps

The critical analysis of existing literature reveals several consistent gaps that our research directly addresses:

Gap Category	Specific Limitation	Studies Affected	Our Approach
Feature Engineering	Position-agnostic aggregation loses individual performance information	Chowdhury et al., Turkish MOBA Study, Liang	Position-preserving pivot with T1/T2 positional prefixes
Feature Selection	Single-method selection without redundancy elimination	Bahrololloomi et al., Chinese ML Study	Two-stage SULOV + XGBoost RFE
Model Diversity	Limited ensemble evaluation	Most studies except Chowdhury et al.	8 individual models + 2 ensemble strategies
Stability Validation	Single split or limited cross-validation	Nearly all reviewed studies	100-iteration stability analysis
Interpretability	Feature importance without instance-level	All studies except those with basic	Comprehensive SHAP analysis

Gap Category	Specific Limitation	Studies Affected	Our Approach
	explanation	importance plots	
Reproducibility	Insufficient implementation detail	Multiple studies	Open-source code release

2.3.2 Theoretical Positioning

Our research is positioned at the intersection of three literature streams:

1. **MOBA Win Prediction:** We extend this core literature by combining position-aware feature engineering (as in Bahrololloomi et al.) with rigorous feature selection and ensemble methods not previously integrated in a single study.
2. **Champion and Position Analysis:** We draw on insights from champion recommendation (Błaszczyk & Szajerman) and player-champion experience research (Do et al.) to motivate our position-preserving feature engineering approach.
3. **Esports Analytics Methodology:** We adopt Maymin's emphasis on context-aware metric interpretation and apply it through SHAP analysis, revealing not just *which* features matter but *how* they influence predictions.

2.3.3 Novel Contributions Relative to Prior Work

Our research makes the following novel contributions that extend beyond any single prior study:

1. **Integrated Pipeline:** We are among the first to combine position-preserving feature engineering, correlation-based redundancy elimination, boosting-based feature selection, extensive model comparison (including ensembles), stability validation, and SHAP interpretability in a single, reproducible pipeline.
2. **Open-Source Commitment:** By releasing our complete implementation, we directly address the reproducibility limitations noted in multiple prior studies and provide a foundation for future research to build upon.
3. **Multi-Metric Evaluation:** Our use of six complementary evaluation metrics (R^2 , Accuracy, MAE, RMSD, MAD) provides richer performance characterization than the accuracy-only or accuracy-plus-AUC approaches common in prior work.
4. **Stability Quantification:** The 100-iteration stability analysis with reported confidence intervals represents a more rigorous validation protocol than the single-split or 5-fold cross-validation typical in the reviewed literature.

3. Methodology

3.1 Data Description

3.1.1 Data Source and Collection Overview

The dataset utilized in this research was collected through the official Riot Games API, a RESTful web service that provides programmatic access to League of Legends match data, player statistics, and ranked information. The Riot Games API is the authoritative source for League of Legends data, ensuring that all collected statistics are accurate, consistent, and representative of actual gameplay. Data collection occurred during the 2024-2025 ranked season, capturing matches across multiple game patches to ensure meta-game diversity.

The data acquisition pipeline was implemented as a multi-stage process designed to address the inherent challenges of collecting large-scale, representative match data from a live service game. These challenges include:

- **API rate limiting:** Riot Games enforces strict rate limits on API requests, permitting approximately 20 requests per second and 100 requests per two minutes per API key. Without careful rate limit management, data collection pipelines risk request rejection and incomplete datasets.
- **Tier representation bias:** Without deliberate sampling strategies, collected data may overrepresent certain skill tiers while underrepresenting others, introducing systematic bias into predictive models.
- **Match availability constraints:** The API provides access to recent match histories but does not guarantee complete historical records. Players with limited recent activity may not provide sufficient matches for their assigned tier and division.
- **Connection reliability:** Large-scale data collection over extended periods inevitably encounters network interruptions, timeouts, and transient API errors that must be handled gracefully to ensure dataset completeness.

Our data collection methodology was designed to systematically address each of these challenges, as detailed in the following subsections.

3.1.2 Sampling Strategy: Tier-Stratified Snowball Sampling

The sampling methodology employed in this research combines elements of stratified sampling with snowball network expansion. This hybrid approach was selected to balance representativeness across skill tiers with practical data collection feasibility. We formally characterize our approach as **Tier-Stratified Snowball Sampling with Division-Level Quotas**.

3.1.2.1 Conceptual Foundation

Traditional random sampling from the entire League of Legends player population is infeasible due to the absence of a comprehensive player registry accessible through the API. Conversely, purely opportunistic sampling (collecting whatever matches are most readily available) introduces uncontrolled biases toward highly active players, specific skill tiers, or particular regions. Our hybrid approach navigates between these extremes by:

1. **Establishing stratified quotas** for each tier and division, ensuring representation across the full skill spectrum from Iron to Challenger
2. **Leveraging the league endpoint** to identify players within specific tier-division combinations, providing a structured sampling frame within each stratum
3. **Expanding through player match histories** to collect actual gameplay data from the identified players, effectively using each sampled player as a seed for match discovery

3.1.2.2 Sampling Frame Construction

The sampling frame was constructed using Riot Games' league endpoints, which provide lists of players within each ranked tier and division. The process comprised three sequential phases:

Phase 1: Player Identification (Stratified Selection)

For lower tiers (Iron through Diamond), the league endpoint returns paginated lists of players within each tier-division combination (e.g., Gold II, Platinum IV). For each of the 28 tier-division combinations (7 tiers × 4 divisions each), we queried the endpoint to retrieve active players, filtering out inactive accounts. From each stratum, we selected a quota of players to serve as seeds for match collection.

For higher tiers (Master, Grandmaster, Challenger), which do not subdivide into divisions, we queried the specialized league endpoints for each tier. These endpoints return league entries sorted by League Points (LP), enabling selection of players across the LP spectrum within each tier. The quota for these tiers was set higher than for lower tiers to account for the greater skill variance within each tier and the smaller player populations.

The player selection process is formalized in Algorithm 1:

text

Algorithm 1: Stratified Player Selection

Input: Tier list T , Division list D , Quota per stratum q

Output: Dictionary mapping division keys to player lists

1. For each tier t in T :
2. If $t \in \{\text{MASTER}, \text{GRANDMASTER}, \text{CHALLENGER}\}$:

3. Query tier-specific endpoint for t
4. Sort entries by `leaguePoints` descending
5. Select top q entries
6. Store with key " $\{t\}_I$ "
7. Else:
8. For each division d in D :
9. Query league endpoint for $\{t\}/\{d\}$
10. Filter to active (non-inactive) entries
11. Collect players until count = q
12. Store with key " $\{t\}_{\{d\}}$ "
13. Return all stored player collections

Phase 2: Match History Collection (Snowball Expansion)

For each selected player, we queried the match history endpoint to retrieve match identifiers for recent ranked solo queue games. The match history endpoint returns up to 100 match IDs per query, with pagination support for accessing older matches. We continued paginating through each player's history until either:

- The quota of matches for that division was satisfied, or
- The player's available match history was exhausted

When a player's history was exhausted before meeting the quota, the algorithm progressed to the next player within the same division, continuing until the quota was met or all available players were exhausted. This cascading fallback mechanism ensures maximum quota fulfillment while maintaining division-level stratification.

The match collection process is formalized in Algorithm 2:

text

Algorithm 2: Snowball Match Collection

Input: Player pool P per division, Match quota M

Output: Match IDs with tier/division metadata

1. For each division d:
2. Initialize match_ids = []
3. Initialize player_index = 0
4. While |match_ids| < M and player_index < |P[d]|:
5. puuid = P[d][player_index]
6. start_index = 0
7. While |match_ids| < M:
8. batch = get_match_history(puuid, start=start_index, count=min(100, M - |match_ids|))
9. If batch is empty:
10. Break // Player history exhausted
11. match_ids.extend(batch)
12. start_index += |batch|
13. If |match_ids| < M:
14. player_index += 1 // Move to next player
15. Store match_ids with division metadata

Phase 3: Match Data Retrieval (Full Data Flattening)

Each collected match ID was used to query the full match endpoint, which returns comprehensive data including team-level objectives, player statistics, and timeline information. The match data was then flattened into tabular format:

- **Team-level data** was extracted into a structured format with columns for match identifier, team identifier, win indicator, ban selections, and objective statistics (first and kill counts for barons, dragons, towers, inhibitors, rift heralds, and atakhans)

- **Player-level data** was extracted into a structured format with columns spanning identity information (PUUID, summoner name), champion selection, performance metrics (combat, economy, vision, objectives), rune configurations, and challenge statistics

Critically, the tier and division information used for sampling was intentionally excluded from the flattened match data to prevent data leakage—the model should predict match outcomes based on in-game performance, not on the skill level of the sampled players.

3.1.2.3 Quota Specifications

The per-division match quotas were designed to balance dataset size against collection feasibility:

Tier Category	Tiers Included	Divisions per Tier	Matches per Division	Total Matches
Low Tiers	Iron, Bronze, Silver, Gold	4 each	50	800
Mid Tiers	Platinum, Emerald	4 each	50	400
High-Mid Tier	Diamond	4	50	200
Elite Tiers	Master, Grandmaster, Challenger	1 each	200	600
Total	10 tiers	31 divisions	—	2,000

The asymmetric quota allocation reflects several considerations:

- **Lower quotas for low and mid tiers (50 matches per division):** These tiers contain the majority of the player population. The 50-match quota per division provides sufficient representation while keeping the total dataset manageable. With 7 tiers × 4 divisions × 50 matches, this yields 1,400 matches from Iron through Diamond, capturing the full spectrum of casual and dedicated play.
- **Higher quotas for elite tiers (200 matches per tier):** Master, Grandmaster, and Challenger represent the top approximately 0.5% of the ranked player base. The elevated quota serves two purposes: (1) compensating for the smaller number of divisions (one per tier rather than four), and (2) providing sufficient data to model the distinctive gameplay patterns at the highest skill levels, which may differ qualitatively from lower tiers.
- **Rank-ordered collection:** Within each division, players were selected based on their league position ranking (by League Points), ensuring that collected matches represent the full range of skill within each stratum rather than only the most active or recently promoted players.

3.1.2.4 Comparison with Alternative Sampling Methods

To contextualize our methodological choice, we compare our Tier-Stratified Snowball Sampling against several alternative approaches considered during research design:

Sampling Method	Description	Advantages	Limitations	Suitability Assessment
Random Player Sampling	Select players randomly from the general population	Maximizes statistical randomness	No player registry available; must use workaround seeds; may miss entire tiers	Impractical without comprehensive sampling frame
Temporal Sampling	Collect matches from specific time windows	Captures meta-shift effects; good for time-series analysis	Requires continuous data collection; sensitive to patch timing	Complementary to our approach but requires longer collection period
Patch-Stratified Sampling	Equal matches per game patch	Controls for balance changes; improves cross-patch generalization	Requires patch version tracking; small patch windows limit per-patch data	Valuable for future work on patch-robustness
Queue-Type Stratified Sampling	Sample from multiple game modes	Captures behavioral differences across modes	This research focuses specifically on ranked solo queue	Not applicable to our research scope
Pure Snowball Sampling	Start from seeds and expand through teammate/opponent networks	Efficient for building large datasets from minimal seeds	Introduces community clustering bias; overrepresents highly connected players	Effective for expansion but requires stratification overlay
Champion-Main Sampling	Target players who frequently play specific champions	Enables champion-specific modeling	Requires champion mastery data; may miss role diversity	Useful for champion-focused research questions
Tier-Stratified	Stratified player selection with	Ensures tier representation;	Potential activity bias (more	Optimal balance of

Sampling Method	Description	Advantages	Limitations	Suitability Assessment
Snowball (Our Approach)	snowball match expansion	practical collection; division-level granularity	active players overrepresented within tiers)	representativeness and feasibility for our research objectives

Our approach most closely aligns with the Tier-Stratified Sampling paradigm but incorporates snowball elements in the match collection phase (using selected players as entry points to discover matches). This hybrid design ensures that our dataset reflects the full skill spectrum while remaining practically collectible within API rate limits.

3.1.2.5 Limitations of the Sampling Approach

We acknowledge several limitations inherent in our sampling methodology:

- Activity bias:** Players who appear in league endpoint responses tend to be those with recent ranked activity. Highly skilled but currently inactive players are systematically underrepresented.
- Region specificity:** Our data was collected from the EU Nordic & East (EUN1) server. Meta-game patterns, champion preferences, and playstyles may vary across regions (Korea, North America, China, etc.), potentially limiting cross-region generalizability.
- Queue type restriction:** We exclusively collected Ranked Solo/Duo (queue ID 420) matches. Normal games, Flex queue, and ARAM exhibit different behavioral dynamics and may yield different predictive feature patterns.
- Temporal snapshot:** Data collection occurred during a specific period of the 2024-2025 season. Balance patches released after data collection may alter the relationship between features and match outcomes.
- Within-tier variance:** While our stratification captures between-tier differences, within-tier variance (e.g., Diamond IV versus Diamond I) is not explicitly modeled, as we treat each tier-division as a homogeneous stratum.

3.1.3 Data Collection Technical Implementation

3.1.3.1 Rate Limiting Infrastructure

The Riot Games API enforces two concurrent rate limits:

- Short-term limit:** 20 requests per 1 second
- Long-term limit:** 100 requests per 2 minutes

Exceeding either limit results in HTTP 429 (Too Many Requests) responses with a `Retry-After` header indicating the required waiting period. Systematic violation of rate limits can result in temporary API key blacklisting.

To ensure compliant and efficient data collection, we implemented a `RateLimiter` class that:

1. **Tracks request timestamps** separately for short-term (1-second) and long-term (120-second) windows
2. **Proactively checks** both limits before each API call
3. **Calculates required wait times** when limits are approached, sleeping the process until capacity is available
4. **Purges expired timestamps** to maintain efficient memory usage

This proactive approach prevents rate limit violations entirely, eliminating the need for reactive retry-after handling and maximizing throughput within allowed limits. The rate limiter's operation is transparent to the data collection logic, encapsulating all rate limit complexity behind a simple `wait_if_needed()` / `add_request()` interface.

3.1.3.2 Error Handling and Retry Logic

Network requests to external APIs are inherently unreliable. Transient failures—including connection timeouts, DNS resolution errors, server-side 503 responses, and unexpected connection resets—are expected during large-scale data collection. To ensure dataset completeness, we implemented a robust retry mechanism:

- **Exponential backoff:** Failed requests are retried with geometrically increasing delay (5 seconds, 10 seconds, 20 seconds), reducing server load during potential outages while maximizing eventual success probability
- **Status code differentiation:** HTTP 404 (match not found) responses are not retried, as these indicate permanent unavailability. HTTP 429 responses respect the server-provided `Retry-After` duration. All other failures (timeouts, connection errors, 5xx server errors) trigger retry logic
- **Maximum retry ceiling:** After three consecutive failures (for the main collection) or five failures (for the dedicated retry pass), the match is logged as permanently failed and excluded from the dataset
- **Failed match tracking:** All failed match IDs, along with their associated tier and division metadata, are preserved in a separate CSV file for diagnostic analysis and potential recovery attempts

3.1.3.3 Data Collection Workflow

The complete data collection workflow proceeded as follows:

1. **Player Identification:** Execute Algorithm 1 to collect player PUUIDs for each tier-division combination
2. **Player File Consolidation:** Merge per-division player files into a unified player registry with tier, rank, and division columns
3. **Match ID Collection:** Execute Algorithm 2 using the consolidated player registry, collecting match IDs with division-level quotas
4. **Match ID Consolidation:** Merge per-division match ID files into a unified match registry
5. **Match Data Retrieval:** For each match ID in the unified registry, fetch full match data with retry logic, flatten into team-level and player-level data structures
6. **Failed Match Recovery (Optional):** Execute a dedicated retry pass over failed match IDs with increased retry attempts
7. **Data Export:** Save flattened team data and player data as timestamped CSV files in the `flattened` directory

This pipeline design ensures that each stage can be executed, validated, and if necessary re-executed independently, providing resilience against interruptions during the multi-hour collection process.

3.1.4 Dataset Composition and Structure

3.1.4.1 Match-Level Data (Team Statistics)

The team-level dataset (`teams_stats_merged.csv`) contains two rows per match (one for each team) with the following structure:

Column Category	Columns	Description
Identifiers	<code>matchId</code> , <code>teamId</code>	Unique match and team identifiers
Match Context	<code>gameDuration</code> , <code>platformId</code> , <code>queueId</code>	Match duration (seconds), server platform, queue type
Outcome	<code>win</code>	Boolean indicator: <code>True</code> for winning team, <code>False</code> for losing team
Bans	<code>ban_1_championId</code> through <code>ban_5_championId</code>	Champion IDs banned during draft phase
Objectives	<code>obj_{objective}_first</code> , <code>obj_{objective}_kills</code>	For each objective type (baron, dragon, tower,

Column Category	Columns	Description
		inhibitor, riftHerald, atakhan, champion, horde): boolean first-acquisition indicator and kill count

The team-level data provides the ground truth labels (win/loss) for supervised learning and captures team-wide strategic outcomes (objective control, ban priorities) that are not attributable to individual players.

3.1.4.2 Participant-Level Data (Player Statistics)

The participant-level dataset (`match_data_merged.csv`) contains ten rows per match (one for each player) with the following structure:

Identity and Role:

- `matchId`, `participantId`, `puuid`, `riotIdGameName`, `riotIdTagline`
- `teamId`, `teamPosition` (TOP, JUNGLE, MIDDLE, BOTTOM, UTILITY)
- `championId`, `championName`, `champLevel`, `profileIcon`, `summonerLevel`

Combat Performance:

- **KDA:** `kills`, `deaths`, `assists`
- **Multi-kills:** `doubleKills`, `tripleKills`, `quadraKills`, `pentaKills`, `largestMultiKill`
- **Killing sprees:** `largestKillingSpree`, `killingSprees`
- **First events:** `firstBloodKill`, `firstBloodAssist`, `firstTowerKill`, `firstTowerAssist`

Damage Statistics:

- **Dealt:** `totalDamageDealt`, `totalDamageDealtToChampions`, `physicalDamageDealt`, `physicalDamageDealtToChampions`, `magicDamageDealt`, `magicDamageDealtToChampions`, `trueDamageDealt`, `trueDamageDealtToChampions`
- **To objectives:** `damageDealtToBuildings`, `damageDealtToObjectives`, `damageDealtToTurrets`
- **Taken:** `totalDamageTaken`, `physicalDamageTaken`, `magicDamageTaken`, `trueDamageTaken`
- **Mitigation:** `damageSelfMitigated`

Economy:

- `goldEarned` , `goldSpent` , `consumablesPurchased` , `itemsPurchased` , `bountyLevel`
- `item0` through `item6` (inventory slots)

Vision and Utility:

- `visionScore` , `wardsPlaced` , `wardsKilled` , `sightWardsBoughtInGame` , `visionWardsBoughtInGame` , `detectorWardsPlaced`
- `timeCCingOthers` , `totalTimeCCDealt`
- `totalHeal` , `totalHealsOnTeammates` , `totalUnitsHealed` , `totalDamageShieldedOnTeammates`

Objectives and Map Control:

- `baronKills` , `dragonKills` , `inhibitorKills` , `inhibitorTakedowns` , `inhibitorsLost`
- `turretKills` , `turretTakedowns` , `turretsLost`
- `nexusKills` , `nexusTakedowns` , `nexusLost`
- `objectivesStolen` , `objectivesStolenAssists`

Jungle and Minion Farming:

- `totalMinionsKilled` , `neutralMinionsKilled`
- `totalAllyJungleMinionsKilled` , `totalEnemyJungleMinionsKilled`

Spells and Abilities:

- `summoner1Id` , `summoner1Casts` , `summoner2Id` , `summoner2Casts`
- `spell1Casts` through `spell4Casts` (Q, W, E, R abilities)

Runes (Perks):

- `perk_stat_defense` , `perk_stat_flex` , `perk_stat_offense` (stat shard selections)
- `perk_primary_style` , `perk_secondary_style` (rune tree selections)
- `perk_primary_rune_1` through `perk_primary_rune_4` (keystone + secondary runes)
- `perk_secondary_rune_1` through `perk_secondary_rune_2`
- Corresponding `_var1` , `_var2` , `_var3` columns for rune-specific parameters

Pings (Communication):

- `allInPings` , `assistMePings` , `commandPings` , `enemyMissingPings` , `enemyVisionPings`
- `holdPings` , `getBackPings` , `needVisionPings` , `onMyWayPings` , `pushPings` , `visionClearedPings`

Time Metrics:

- `timePlayed`, `totalTimeSpentDead`, `longestTimeSpentLiving`

Challenges (Filtered):

- Various `challenge_*` columns representing achievement-style metrics, with non-Summoner's Rift challenges excluded

3.1.4.3 Data Quality Considerations

Several data quality considerations informed our preprocessing decisions:

- **Missing values:** Some optional API fields may contain null values for specific matches. Our preprocessing pipeline handles these through median imputation after feature engineering.
- **Game mode filtering:** The data collection exclusively targeted queue ID 420 (Ranked Solo/Duo), ensuring consistency in game mode and competitive context. Non-Summoner's Rift challenge metrics were explicitly excluded during flattening.
- **Remake detection:** Matches with `gameDuration` less than 240 seconds (4 minutes) typically represent remakes (matches terminated early due to player disconnection). Our preprocessing retained these matches but flagged them for potential sensitivity analysis.
- **Duplicate match IDs:** The match ID collection process ensures uniqueness within each division's quota but does not deduplicate across divisions (the same match could theoretically appear for multiple sampled players from different divisions). The match data retrieval phase naturally deduplicates, as each unique match ID is fetched only once.

3.1.5 Ethical Considerations

All data was collected through the official Riot Games API in compliance with Riot's developer policies. Player identity information (PUUIDs, summoner names) was used exclusively during the collection process to retrieve match histories and was explicitly excluded from the flattened datasets used for model training and analysis. The released dataset and open-source code contain no personally identifiable information, ensuring player privacy while enabling research reproducibility. This approach aligns with ethical guidelines for esports research and general data protection principles.

3.2 Data Preprocessing and Cleaning

3.2.1 Initial Data Loading and Structural Validation

The preprocessing pipeline begins by loading both CSV files into pandas DataFrames and performing structural validation to ensure data integrity. The match-level data is expected to contain exactly two rows per unique match (one per team), while the participant-level data should contain exactly ten rows per match (five players per team).

The code performs the following validation checks:

```
match_df = pd.read_csv(MATCH_DATA_PATH)
participant_df = pd.read_csv(PARTICIPANT_DATA_PATH)
```

After loading, structural validation confirms that the number of unique matches is consistent across both datasets, that team assignments are properly distributed (checking that `match_df.groupby('matchId')['teamId'].nunique()` returns consistent values), and that participant counts per match equal exactly ten players. The code also verifies the presence of expected team positions (TOP, JUNGLE, MIDDLE, BOTTOM, UTILITY) and confirms team identification values.

3.2.2 Column Removal Strategy

Prior to feature engineering, we systematically remove columns that would introduce data leakage, redundancy, or noise. This removal occurs in two stages: identification of columns to remove, and execution of removal.

Column Identification:

The code first identifies challenge-related columns by filtering for column names beginning with the prefix `challenge_`. These achievement-style metrics are highly specific to Riot's challenge system and often correlate strongly with standard performance metrics, inflating dimensionality without providing independent predictive value.

```
challenge_cols = [col for col in participant_df.columns if
col.startswith('challenge_')]
```

A comprehensive removal set is then constructed, combining the challenge columns with additional fields:

```
cols_to_remove = set(challenge_cols + [
    'championName', 'riotIdTagline', 'riotIdGameName', 'puuid',
    'bountyLevel', 'perk_secondary_style_description',
    'perk_primary_style_description', 'largestCriticalStrike',
    'queueId', 'teamPrefix', 'platformId', 'nexusLost', 'nexusKills',
```

```
'nexusTakedowns', 'inhibitorsLost', 'gameDuration', 'participantId'  
])
```

The rationale for each removal is as follows:

- **Player identity fields** (`puuid`, `riotIdGameName`, `riotIdTagline`): These uniquely identify individual players. Including them would allow the model to learn player-specific patterns rather than performance-based patterns, compromising generalizability and introducing privacy concerns.
- **Champion name** (`championName`): The string representation of champion selection is removed while retaining the numeric `championId`. String features would require text encoding that could inadvertently capture champion strength information.
- **Redundant game state columns** (`nexusLost`, `nexusKills`, `nexusTakedowns`, `inhibitorsLost`): Nexus destruction defines victory, making these columns direct proxies for the target variable. Inhibitor loss is captured through objective statistics.
- **Match context columns** (`gameDuration`, `queueId`, `platformId`): Game duration is a post-hoc characteristic rather than a predictive feature. Queue and platform identifiers are administrative metadata without predictive value.
- **Sparse or champion-specific metrics** (`largestCriticalStrike`, `bountyLevel`): Critical strike damage is relevant only for specific champion builds. Bounty level is highly dynamic within matches and collinear with kill/death patterns.
- **Administrative identifiers** (`participantId`, `teamPrefix`): These serve only as database keys without carrying game-state information.
- **Perk description fields**: These are redundant textual representations of information already captured in numeric perk identifiers.

Column Removal Execution:

The removal is performed on a copy of the participant DataFrame to preserve the original data:

```
participants_clean = participant_df.copy()  
cols_actually_removed = []  
for col in cols_to_remove:  
    if col in participants_clean.columns and col != 'matchId':  
        participants_clean = participants_clean.drop(columns=[col])  
        cols_actually_removed.append(col)
```

The `matchId` column is explicitly preserved during this stage as it is required for subsequent pivot operations.

3.2.3 Data Leakage Prevention

A critical preprocessing consideration is the prevention of data leakage—ensuring that no feature contains information that would not be available at prediction time. Two specific measures address this concern:

1. **Win column removal:** If the participant-level data contains a `win` column (indicating whether a participant's team won), it is explicitly removed during pivot preparation. This column is a direct proxy for the target variable and its inclusion would constitute perfect data leakage.
2. **Tier/division exclusion:** As noted in the data collection methodology, tier and division metadata used for stratified sampling are not included in the flattened match data. The models learn to predict outcomes from in-game performance alone, without access to the skill level of the sampled players.

3.2.4 Missing Value Handling

After feature engineering and dataset merging, the code performs systematic missing value detection and imputation:

```
missing_counts = final_df.isnull().sum()
missing_cols = missing_counts[missing_counts > 0]
if len(missing_cols) > 0:
    numeric_cols = final_df.select_dtypes(include=[np.number]).columns
    for col in numeric_cols:
        if final_df[col].isnull().sum() > 0:
            final_df[col] = final_df[col].fillna(final_df[col].median())
```

Median imputation is selected over mean imputation due to its robustness to outliers, which are common in gaming performance data (e.g., exceptionally high kill counts, extreme damage values in snowballed matches). The code reports columns with missing values and the proportion of missing entries before imputation, providing transparency into data quality.

3.2.5 Data Type Standardization

After feature engineering, all columns are converted to numeric types to ensure compatibility with scikit-learn and gradient boosting libraries:

```
for col in X.columns:
    if X[col].dtype == 'object':
        try:
            X[col] = pd.to_numeric(X[col])
        except:
            le = LabelEncoder()
            X[col] = le.fit_transform(X[col].astype(str))
```


The conversion attempts direct numeric parsing first, falling back to label encoding for any columns that resist numeric conversion. In practice, the thorough column removal in earlier stages ensures that few if any object-type columns remain at this point.

3.3 Feature Engineering: Player-Level to Match-Level Transformation

3.3.1 Overview of the Transformation Problem

The central feature engineering challenge in League of Legends win prediction is the structural mismatch between data representation and prediction target. The raw data contains ten participant records per match (five per team), each with dozens of performance metrics, while the prediction target is a single binary outcome per match. Transforming this multi-entity, hierarchically structured data into a single feature vector per match while preserving individual player information requires careful methodological design.

Our approach addresses this challenge through a position-preserving pivot transformation that converts the participant-level data from long format (one row per player) to wide format (one row per match with player-specific feature columns). This transformation preserves the individual contributions of each player position to the match outcome while creating a structure compatible with standard machine learning algorithms.

3.3.2 Team and Position Identification

Before pivoting, each player record must be assigned a consistent identifier that captures both team affiliation and position role. The code implements this in two stages:

Stage 1: Team Prefix Assignment

The raw data identifies teams using Riot's `teamId` values (typically 100 for blue side and 200 for red side). To create a consistent representation, these are mapped to T1 and T2 prefixes based on their ordering within each match:

```
participants_clean['team_rank'] = participants_clean.groupby('matchId')
    ['teamId'].transform(
        lambda x: x.rank(method='dense')
    )
participants_clean['teamPrefix'] = 'T' +
participants_clean['team_rank'].astype(int).astype(str)
participants_clean = participants_clean.drop('team_rank', axis=1)
```

The `rank(method='dense')` transformation assigns rank 1 to the smaller `teamId` and rank 2 to the larger, ensuring consistent T1/T2 assignment across all matches. The temporary `team_rank` column is immediately discarded after the prefix is created.

Stage 2: Player Identifier Construction

The team prefix is combined with the player's position to create a unique identifier:

```
participants_clean['playerId'] = (  
    participants_clean['teamPrefix'] + '_' +  
    participants_clean['teamPosition']  
)
```

This produces identifiers such as `T1_TOP`, `T1_JUNGLE`, `T1_MIDDLE`, `T1_BOTTOM`, `T1_UTILITY`, and corresponding T2 variants. The `teamPosition` column is then dropped to prevent textual data from causing issues during model training:

```
participants_clean = participants_clean.drop(columns=['teamPosition'])
```

Validation: The code verifies that each match contains exactly 10 players by grouping by `matchId` and checking the size distribution, confirming structural integrity before proceeding to the pivot operation.

3.3.3 Target Variable Construction

The binary target variable `T1_win` is constructed from the match-level data by determining which team prefix (T1 or T2) corresponds to the winning team:

```
team_mapping = participants_clean[['matchId', 'teamId',  
    'teamPrefix']].drop_duplicates()  
match_with_prefix = match_df.merge(team_mapping, on=['matchId', 'teamId'],  
    how='left')  
winners = match_with_prefix[match_with_prefix['win'] == True][['matchId',  
    'teamPrefix']].copy()  
winners['T1_win'] = (winners['winner'] == 'T1').astype(int)
```

This produces a binary variable where 1 indicates a T1 victory and 0 indicates a T2 victory. The code reports the target distribution to verify class balance, as severe imbalance would require mitigation strategies such as stratified sampling or class weighting.

3.3.4 Pivot Transformation

The core feature engineering operation is the pivot transformation from long to wide format:

```

pivot_data = participants_clean.copy()
value_cols = [c for c in pivot_data.columns
               if c not in ['matchId', 'playerId', 'teamId']]

match_features = pivot_data.pivot_table(
    index='matchId',
    columns='playerId',
    values=value_cols,
    aggfunc='first'
)

```

This pivot operation uses `matchId` as the index (one row per match) and `playerId` as the columns (creating separate columns for each player-position combination). The `aggfunc='first'` parameter ensures that when multiple values exist for the same match-player combination (which should not occur in properly structured data), only the first value is retained.

Column Flattening:

The pivot produces a MultiIndex column structure where each column is identified by a (feature, playerId) tuple. These are flattened into single-level column names:

```

match_features.columns = [f"{feat}_{player}" for feat, player in
                           match_features.columns]

```

This produces column names such as `kills_T1_TOP`, `deaths_T2_JUNGLE`, `goldEarned_T1_MIDDLE`, preserving both the feature semantics and the player position information in a flat column structure.

Scale of Transformation:

The pivot operation transforms the feature space as follows: if the participant data contains N features per player (after column removal), the output contains $N \times 10$ features (one set per player-position combination), plus the match identifier. The code reports both the expected and actual output dimensions for verification.

3.3.5 Team-Level Objective Feature Integration

Team-level objective data from the match DataFrame is integrated through a parallel pivot process:

```

match_objective_cols = [
    'matchId', 'gameDuration', 'queueId',

```

```
'obj_atakhان_first', 'obj_atakhان_kills',  
'obj_baron_first', 'obj_baron_kills',  
'obj_champion_first', 'obj_champion_kills',  
'obj_dragon_first', 'obj_dragon_kills',  
'obj_horde_first', 'obj_horde_kills',  
'obj_inhibitor_first', 'obj_inhibitor_kills',  
'obj_riftHerald_first', 'obj_riftHerald_kills',  
'obj_tower_first', 'obj_tower_kills'  
]
```

These columns are aggregated by match and team prefix, then pivoted to create T1/T2-specific objective columns. Duplicate `gameDuration` columns (one per team) are removed after pivoting:

```
objectives_wide = objectives_wide.drop(columns=['gameDuration_T1',  
'gameDuration_T2'])
```

3.3.6 Final Dataset Assembly

The participant-derived features, team objective features, and target variable are merged into the final dataset:

```
final_df = match_features.merge(objectives_wide, on='matchId', how='inner')  
final_df = final_df.merge(match_targets, on='matchId', how='inner')  
final_df = final_df.drop(columns=['matchId'])
```

The `matchId` column is removed after merging as it no longer serves a purpose—the dataset is now a standard feature matrix where each row represents a match with features as columns and a binary target variable.

Final Dataset Characteristics:

The code reports the final dataset dimensions, total feature count, and target distribution, providing a clear summary of the engineered dataset before model training begins.

3.4 Feature Selection Methodology

3.4.1 Overview and Rationale

After feature engineering, the dataset contains a high-dimensional feature space ($N \times 10$ participant features plus team objective features). Many of these features exhibit substantial

multicollinearity due to the interrelated nature of game performance metrics. Feature selection addresses this through a two-stage process designed to eliminate redundancy while preserving predictive power.

3.4.2 Stage 1: SULO V Correlation Analysis

Theoretical Foundation:

SULO V (Searching for and Removing Unnecessary Variables) is a filter-based feature selection method that identifies and removes redundant feature pairs based on correlation analysis. The method is grounded in the Minimum Redundancy Maximum Relevance (mRMR) framework: features should be maximally relevant to the target while minimally redundant with each other.

Implementation:

The code first isolates numeric columns and optionally samples the dataset to manage computational complexity:

```
numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
X_for_corr = X[numeric_cols]
sample_size = min(5000, len(X))
X_sample = X_for_corr.sample(n=sample_size, random_state=42) if len(X) > 5000
else X_for_corr
```

The absolute correlation matrix is computed, and pairs exceeding the threshold of 0.81 are identified:

```
corr_matrix = X_sample.corr().abs()
CORR_THRESHOLD = 0.81
```

The 0.81 threshold is adapted from prior literature and represents a balance between aggressive redundancy elimination and preservation of potentially informative features. The code examines only the upper triangle of the correlation matrix (avoiding self-correlations and duplicate pairs) and compiles a list of highly correlated feature pairs sorted by correlation magnitude.

Feature Retention Criterion:

For each highly correlated pair, the feature with higher absolute correlation to the target variable is retained:

```
target_corr = X_sample.corrwith(y).abs()
features_to_remove = set()
```

```

for feat1, feat2, corr in high_corr_pairs:
    if feat1 in features_to_remove or feat2 in features_to_remove:
        continue
    if target_corr.get(feat1, 0) < target_corr.get(feat2, 0):
        features_to_remove.add(feat1)
    else:
        features_to_remove.add(feat2)

```

This greedy approach ensures that for each redundant pair, the feature most strongly associated with match outcomes is preserved. Features already marked for removal are skipped to prevent cascading removals.

Output:

The code reports the number of features removed and the resulting feature count after SULO V reduction. A correlation heatmap visualization is generated for the first 150 features, saved as `correlation_matrix_sulov.png` , providing visual confirmation of the correlation structure.

3.4.3 Stage 2: XGBoost Recursive Feature Elimination

Theoretical Foundation:

While SULO V eliminates redundancy, it does not directly optimize for predictive power. The second stage addresses this by employing XGBoost, a gradient boosting algorithm that provides built-in feature importance scores based on the average gain from splits using each feature.

Implementation:

Features are standardized before XGBoost training to ensure that scale differences do not affect split quality:

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_reduced)
X_scaled_df = pd.DataFrame(X_scaled, columns=X_reduced.columns)

```

An XGBoost regressor is trained with controlled hyperparameters:

```

xgb_model = xgb.XGBRegressor(
    n_estimators=100,
    max_depth=6,
    learning_rate=0.1,
    random_state=42,
    n_jobs=-1
)

```

```
)  
xgb_model.fit(X_scaled_df, y)
```

Feature importance scores are extracted and sorted in descending order:

```
feature_importance = pd.DataFrame({  
    'feature': X_reduced.columns,  
    'importance': xgb_model.feature_importances_  
}).sort_values('importance', ascending=False)
```

Top Feature Selection:

The top 15 features by importance are selected:

```
TOP_N_FEATURES = 15  
top_features = feature_importance.head(TOP_N_FEATURES)['feature'].tolist()  
X_selected = X_reduced[top_features].copy()
```

The choice of 15 features balances model simplicity with predictive performance, following conventions in the reference literature. The code reports the selected features with their importance scores and generates a correlation heatmap of the final feature set saved as `top_features_correlation.png`.

3.4.4 Additional Feature Pruning

Prior to the SULOV analysis, the code also removes columns with the `teamPrefix_` prefix if any were inadvertently generated during the feature engineering process. This ensures that no team identification metadata persists in the feature matrix.

3.5 Machine Learning Models

3.5.1 Model Selection Philosophy

Our approach employs regression models for binary classification through threshold conversion. This design choice is motivated by several considerations: regression outputs provide continuous win probability estimates rather than discrete predictions, enabling nuanced interpretation; regression-oriented loss functions (mean squared error) may provide smoother optimization landscapes than classification-oriented alternatives; and the approach is consistent with methodologies in the reference literature.

3.5.2 Individual Model Specifications

The code implements eight individual regression models, each with distinct algorithmic characteristics:

Random Forest Regressor (RF):

```
RandomForestRegressor(random_state=42, n_jobs=-1)
```

An ensemble of decision trees trained on bootstrap samples with random feature subset selection at each split. Hyperparameters tuned include the number of estimators, maximum tree depth, and minimum samples per split.

XGBoost Regressor (XGB):

```
xgb.XGBRegressor(random_state=42, n_jobs=-1)
```

A gradient boosting implementation with regularization, efficient tree construction, and built-in handling of missing values. Hyperparameters tuned include the number of boosting rounds, maximum tree depth, and learning rate.

CatBoost Regressor (CatB):

```
CatBoostRegressor(random_state=42, verbose=False)
```

A gradient boosting variant with native categorical feature support and ordered boosting to reduce overfitting. Hyperparameters tuned include iteration count, tree depth, and learning rate.

Gradient Boosting Regressor (GradB):

```
GradientBoostingRegressor(random_state=42)
```

The scikit-learn implementation of gradient boosting with sequential tree construction minimizing residual error. Hyperparameters tuned parallel those of XGBoost.

K-Nearest Neighbors Regressor (KNNR):

```
KNeighborsRegressor(n_jobs=-1)
```

Instance-based learning that predicts outcomes as the average of the k nearest training examples. Hyperparameters tuned include the number of neighbors and weighting scheme (uniform vs. distance-weighted).

Support Vector Regressor (SVR):


```
SVR()
```

Support vector machines adapted for regression with an ϵ -insensitive loss function and kernel-based nonlinear mapping. Hyperparameters tuned include the regularization parameter C and kernel type (RBF vs. linear).

Ridge Regression (Ridge):

```
Ridge()
```

L2-regularized linear regression that shrinks coefficient magnitudes to reduce variance. The regularization strength alpha is the primary hyperparameter.

Multi-Layer Perceptron Regressor (MLPR):

```
MLPRegressor(random_state=42, max_iter=500)
```

A feedforward neural network with one or two hidden layers and ReLU activation. Hyperparameters tuned include hidden layer architecture and L2 regularization strength.

3.5.3 Scaling Requirements

Models vary in their sensitivity to feature scaling:

- **Scaling-required models (SVR, KNNR, Ridge, MLPR):** These algorithms rely on distance calculations or gradient-based optimization that is sensitive to feature magnitudes. Standardization ensures all features contribute proportionally.
- **Scaling-optional models (RF, XGB, CatB, GradB):** Tree-based methods make split decisions based on rank ordering, making them invariant to monotonic feature transformations.

The code tracks this requirement through the `needs_scaling` flag in the model configuration dictionary.

3.5.4 Ensemble Model Construction

Bagging Regressor (BagR):

The bagging ensemble wraps the best-performing individual model as a base estimator and trains 10 instances on bootstrap samples:

```
bag_model = BaggingRegressor(  
    estimator=best_models[best_single_name]['model'],  
    n_estimators=10,  
    random_state=42,  
    n_jobs=-1  
)
```

Bagging reduces variance by averaging predictions from models trained on different data subsets. The choice of 10 estimators balances variance reduction against computational cost.

Voting Regressor (VotR):

The voting ensemble combines the top three individual models (by R^2 score) through prediction averaging:

```
vot_model = VotingRegressor(estimators=estimators)
```

The theoretical justification rests on the assumption that different model families make uncorrelated errors, such that their combination reduces overall prediction error. The voting ensemble uses standardized data as the safest common input format across heterogeneous model types.

3.6 Hyperparameter Optimization

3.6.1 Optimization Framework

Hyperparameter optimization is performed using GridSearchCV with 5-fold cross-validation:

```
grid_search = GridSearchCV(  
    config['model'],  
    config['params'],  
    cv=5,  
    scoring='r2',  
    n_jobs=-1  
)  
grid_search.fit(X_tr, y_train)  
best_model = grid_search.best_estimator_
```

The optimization targets R^2 score as the primary evaluation metric. R^2 was selected over accuracy for hyperparameter optimization because it provides a continuous, scale-invariant measure of model fit that is more sensitive to incremental improvements than the discrete

accuracy metric. Cross-validation with 5 folds balances computational efficiency against reliable performance estimation.

3.6.2 Hyperparameter Search Spaces

The code defines model-specific hyperparameter grids in the `models_config` dictionary. Each grid contains 2-3 values per hyperparameter, yielding 6-12 total combinations:

Model	Hyperparameter	Values Tested
RF	n_estimators	[50, 100]
RF	max_depth	[5, 10, None]
RF	min_samples_split	[2, 5]
XGB	n_estimators	[50, 100]
XGB	max_depth	[3, 6]
XGB	learning_rate	[0.05, 0.1]
CatB	iterations	[50, 100]
CatB	depth	[4, 6]
CatB	learning_rate	[0.05, 0.1]
GradB	n_estimators	[50, 100]
GradB	max_depth	[3, 5]
GradB	learning_rate	[0.05, 0.1]
KNNR	n_neighbors	[3, 5, 7]
KNNR	weights	[uniform, distance]
SVR	C	[0.1, 1, 10]
SVR	kernel	[rbf, linear]
Ridge	alpha	[0.1, 1.0, 10.0]
MLPR	hidden_layer_sizes	[(50,), (100,)]
MLPR	alpha	[0.0001, 0.001]

3.6.3 Regression-to-Classification Conversion

After obtaining regression predictions, they are converted to binary classifications using a threshold of 0.5:

```
y_pred_reg = best_model.predict(X_te)
```

```
y_pred_class = (y_pred_reg >= 0.5).astype(int)
```

This conversion enables the calculation of classification metrics (accuracy, classification reports, confusion matrices) alongside regression metrics, providing a comprehensive performance assessment.

3.7 Evaluation Metrics

3.7.1 Metric Selection

The code implements a comprehensive suite of evaluation metrics capturing different aspects of model performance:

R² Score (Coefficient of Determination):

```
r2 = r2_score(y_test, y_pred_reg)
```

Measures the proportion of variance in the target explained by the model. R² ranges from negative infinity to 1, with 1 indicating perfect prediction and 0 indicating performance equivalent to predicting the mean.

Accuracy (Classification):

```
accuracy = accuracy_score(y_test, y_pred_class) * 100
```

The percentage of correctly classified matches after threshold conversion. Expressed as a percentage for intuitive interpretation.

MAE (Mean Absolute Error):

```
mae = mean_absolute_error(y_test, y_pred_reg)
```

The average absolute difference between predicted win probability and actual binary outcome. Lower values indicate better calibration.

RMSE (Root Mean Squared Deviation):

```
rmsd = np.sqrt(mean_squared_error(y_test, y_pred_reg))
```

The square root of average squared prediction error. RMSD penalizes large errors more heavily than MAE, making it sensitive to outliers.

MAD (Median Absolute Deviation):

```
mad = np.median(np.abs(y_test - y_pred_reg))
```

The median of absolute prediction errors. MAD is robust to outliers and provides a measure of typical prediction error.

3.7.2 Results Aggregation and Reporting

All evaluation metrics are stored in a `results` dictionary keyed by model name, then converted to a sorted DataFrame for comparison:

```
results_df = pd.DataFrame(results).T
results_df = results_df[['Accuracy', 'R2', 'MAE', 'RMSD', 'MAD',
                        'Best_Params']]
results_df = results_df.sort_values('R2', ascending=False)
```

Metrics are rounded to four decimal places for error metrics and two decimal places for percentage metrics. The results are printed in a formatted table and the best model is highlighted with its parameter configuration.

3.8 Stability Analysis

3.8.1 Stability Analysis Framework

Model stability is assessed through a 100-iteration modified cross-validation procedure. In each iteration, the dataset is randomly shuffled, split into training (80%) and test (20%) sets with stratification to maintain class balance, and the models are retrained and evaluated:

```
N_ITERATIONS = 100
top_5_models = results_df.head(5).index.tolist()

for iteration in range(N_ITERATIONS):
    indices = np.random.permutation(len(X_selected))
    X_shuffled = X_selected.iloc[indices]
    y_shuffled = y.iloc[indices]

    X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(
```

```
X_shuffled, y_shuffled, test_size=0.2, stratify=y_shuffled
)
```

This repeated random subsampling approach provides more robust performance estimates than single train-test splits, quantifying both mean performance and variability.

3.8.2 Model Reinstantiation

For each iteration, models are reinstantiated with their optimized hyperparameters (obtained from the initial GridSearchCV) and the iteration number as a random seed for reproducibility:

```
best_params = results[name]['Best_Params']
ModelClass = model_config['model'].__class__
model_params = {k: v for k, v in best_params.items()}
if hasattr(ModelClass(), 'random_state'):
    model_params['random_state'] = iteration
model = ModelClass(**model_params)
```

This ensures that performance variation across iterations reflects genuine sensitivity to data partitioning rather than differences in hyperparameter configurations.

3.8.3 Stability Metrics Calculation

For each iteration, all evaluation metrics are calculated and stored. After all iterations complete, mean and standard deviation are computed for each metric:

```
for metric in ['Accuracy', 'R2', 'MAE', 'RMSD', 'MAD']:
    mean_val = metrics_df[metric].mean()
    std_val = metrics_df[metric].std()
    stability_summary[name][f'{metric}_mean'] = mean_val
    stability_summary[name][f'{metric}_std'] = std_val
```

The results are displayed in a formatted table showing mean $\pm$ standard deviation for each metric across the top 5 models.

3.8.4 Stability Visualization

Boxplot visualizations are generated for each metric across the top models:

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
metrics = ['Accuracy', 'R2', 'MAE', 'RMSD', 'MAD']
```

The boxplots show the distribution of performance across the 100 iterations, with boxes colored by model using the Set3 colormap. Grid lines are added for readability, and an extra subplot is hidden to accommodate the 5-metric layout in a 2×3 grid. The visualization is saved as `stability_analysis_boxplots.png`.

3.9 SHAP Analysis for Model Interpretability

3.9.1 Explainer Selection

SHAP analysis is applied to the best-performing model (by R^2 score) to provide interpretable insights into feature contributions. The explainer type is selected based on the model architecture:

```
best_name = results_df.index[0]
best_model_obj = best_models[best_name]['model']

if best_name in ['XGB', 'CatB', 'GradB', 'RF']:
    explainer = shap.TreeExplainer(best_model_obj)
    shap_values = explainer.shap_values(X_shap)
else:
    background = shap.sample(pd.DataFrame(X_shap, columns=X_selected.columns),
                             100)
    explainer = shap.KernelExplainer(best_model_obj.predict, background)
    shap_values = explainer.shap_values(pd.DataFrame(X_shap,
                                                    columns=X_selected.columns).iloc[:100])
```

Tree-based models (XGBoost, CatBoost, Gradient Boosting, Random Forest) use the efficient `TreeExplainer`, which computes exact SHAP values through tree structure traversal. Other models use `KernelExplainer` with a 100-sample background dataset for approximation, with SHAP values computed for the first 100 test instances.

3.9.2 SHAP Visualization Types

Bar Plot (Feature Importance):

```
shap.summary_plot(
    shap_values,
    pd.DataFrame(X_shap, columns=X_selected.columns),
    plot_type="bar",
    max_display=15,
```

```
        show=False  
    )
```

The bar plot displays the average absolute SHAP value for each feature, ranking features by their overall importance to model predictions. The `max_display=15` parameter limits the display to the 15 most important features for visual clarity.

Dot Plot (Feature Effects):

```
shap.summary_plot(  
    shap_values,  
    pd.DataFrame(X_shap, columns=X_selected.columns),  
    max_display=15,  
    show=False  
)
```

The dot plot shows both feature importance (horizontal position) and feature effect direction (color). Each point represents a SHAP value for a single instance, with red indicating high feature values and blue indicating low values. This visualization reveals whether high feature values push predictions toward or away from victory.

3.9.3 Output and Export

Both SHAP plots are saved as high-resolution PNG files (`shap_feature_importance.png` and `shap_summary_plot.png`) for inclusion in analysis reports. The code also exports results to CSV files (`model_comparison_results.csv` , `feature_importance.csv`), a text file listing selected features (`selected_features.txt`), and a dataset information summary (`dataset_info.txt`).

4: Implementation and Experimental Setup

4.1 Technologies and Tools

4.1.1 Programming Language and Environment

The entire machine learning pipeline—from data loading and preprocessing through feature engineering, model training, evaluation, and interpretability analysis—was implemented in **Python 3.12**. Python was selected as the implementation language for several compelling reasons:

- **Rich ecosystem for data science:** Python offers mature, well-documented libraries for data manipulation (pandas, NumPy), machine learning (scikit-learn, XGBoost, CatBoost), and visualization (Matplotlib, Seaborn), enabling efficient development without sacrificing functionality.
- **Reproducibility and accessibility:** Python's open-source nature and widespread adoption in the research community facilitate code sharing, peer review, and extension by other researchers.
- **API interaction capabilities:** The `requests` library provides robust HTTP client functionality for interacting with the Riot Games API during the data collection phase.
- **Interpretability tooling:** The SHAP library offers native Python integration with scikit-learn compatible models, enabling seamless interpretability analysis within the same environment.

The specific Python version is verified at runtime through `sys.version` , ensuring compatibility documentation for future reproduction attempts.

4.1.2 Core Libraries and Dependencies

The implementation relies on a carefully curated set of libraries, each serving specific functions within the pipeline. The following table catalogs each library, its version-independent role, and its contribution to the research:

Data Manipulation and Analysis:

Library	Import Statement	Role in Pipeline
pandas	<code>import pandas as pd</code>	Primary data structure for loading CSV files, DataFrame operations including grouping, pivoting, merging, and filtering. Serves as the backbone for all data transformation stages from raw loading through final dataset assembly.
NumPy	<code>import numpy as np</code>	Numerical computing foundation providing array operations, random number generation for stability analysis shuffling, and mathematical functions for metric calculations (square root for RMSD, absolute values for MAE/MAD).
SciPy	<code>from scipy import stats</code>	Statistical functions available for distribution analysis and hypothesis testing, though primarily imported for potential advanced statistical operations beyond the core pipeline requirements.
SciPy Optimize	<code>from scipy.optimize import minimize</code>	Optimization routines imported for potential advanced model tuning beyond the grid search framework employed in the primary pipeline.

Visualization:

Library	Import Statement	Role in Pipeline
Matplotlib	<pre>import matplotlib.pyplot as plt</pre>	Fundamental plotting library providing figure and axes control, color mapping (viridis colormap for model comparison, Set3 colormap for stability boxplots), and output file generation. All visualizations are saved as high-resolution PNG files through matplotlib's <code>savefig</code> functionality.
Seaborn	<pre>import seaborn as sns</pre>	Statistical data visualization layer providing heatmap functionality for correlation matrix display. The <code>sns.heatmap</code> function generates annotated correlation visualizations with automatic color scaling and masking for upper triangle display.

Machine Learning — Preprocessing:

Library	Import Statement	Role in Pipeline
scikit-learn: Model Selection	<pre>from sklearn.model_selection import train_test_split, GridSearchCV</pre>	<code>train_test_split</code> performs stratified 80/20 train-test partitioning with <code>stratify=y</code> ensuring class balance preservation. <code>GridSearchCV</code> executes exhaustive hyperparameter search with 5-fold cross-validation, scoring by R^2 , with parallel processing via <code>n_jobs=-1</code> .
scikit-learn: Preprocessing	<pre>from sklearn.preprocessing import StandardScaler, LabelEncoder</pre>	<code>StandardScaler</code> applies z-score normalization (mean centering and unit variance scaling) for models requiring scale-invariant features (SVR, KNNR, Ridge, MLPR). <code>LabelEncoder</code> serves as a fallback for converting any remaining object-type columns to numeric representations.
scikit-learn: Metrics	<pre>from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error, accuracy_score, classification_report, confusion_matrix</pre>	Comprehensive metric suite: <code>r2_score</code> for variance explained, <code>mean_absolute_error</code> and <code>mean_squared_error</code> (converted to RMSD via square root) for error quantification, <code>accuracy_score</code> for classification rate, and <code>classification_report</code> / <code>confusion_matrix</code> available for detailed diagnostic analysis.

Machine Learning — Models:

Library	Import Statement	Role in Pipeline
scikit-learn: Ensemble	<pre>from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, BaggingRegressor, VotingRegressor</pre>	RandomForestRegressor provides bootstrap-aggregated decision tree ensembles. GradientBoostingRegressor implements sequential boosting. BaggingRegressor wraps the best base estimator with 10 bootstrap iterations. VotingRegressor averages predictions from the top 3 individual models.
scikit-learn: Neighbors	<pre>from sklearn.neighbors import KNeighborsRegressor</pre>	Instance-based regression using k-nearest neighbor averaging with configurable neighbor count and weighting scheme.
scikit-learn: SVM	<pre>from sklearn.svm import SVR</pre>	Support Vector Regression with kernel-based nonlinear mapping and ϵ -insensitive loss.
scikit-learn: Linear	<pre>from sklearn.linear_model import Ridge</pre>	L2-regularized linear regression providing a well-regularized baseline model.
scikit-learn: Neural Network	<pre>from sklearn.neural_network import MLPRegressor</pre>	Multi-Layer Perceptron regressor with configurable hidden layer architecture and L2 regularization.

Advanced Machine Learning:

Library	Import Statement	Role in Pipeline
XGBoost	<pre>import xgboost as xgb</pre>	Gradient boosting library providing the XGBRegressor with regularization parameters (learning rate, max depth, estimator count) and native parallel processing. Also used in the feature selection stage for importance-based ranking.
CatBoost	<pre>from catboost import CatBoostRegressor</pre>	Gradient boosting with ordered boosting and native categorical feature support, configured with silent verbosity (verbose=False) for clean training output.

Model Interpretability:

Library	Import Statement	Role in Pipeline
SHAP	<code>import shap</code>	SHapley Additive exPlanations library providing <code>TreeExplainer</code> for exact SHAP value computation on tree-based models and <code>KernelExplainer</code> for model-agnostic approximation on other model types. Generates summary plots for feature importance (bar) and feature effects (dot) visualization.

4.1.3 Development Environment Configuration

Display and Output Configuration:

The code configures pandas display options to maximize information visibility during development and debugging:

```
pd.set_option('display.max_columns', None)      # Display all columns without
display truncation
pd.set_option('display.width', None)            # Use full terminal width
pd.set_option('display.max_colwidth', 50)       # Limit column content width to
display 50 characters
```

These settings ensure that intermediate DataFrames with hundreds of feature columns can be inspected in their entirety, facilitating validation of pivot operations and feature selection results.

Warning Suppression:

```
import warnings
warnings.filterwarnings('ignore')
```

Warnings are suppressed to maintain clean output during model training. This is particularly relevant for gradient boosting libraries that may generate convergence warnings during hyperparameter exploration, and for SHAP computations that may produce version-compatibility notifications. In a production environment, selective warning filtering would be preferable, but for research output clarity, global suppression is employed.

4.1.4 File Path Configuration

The data loading section uses explicit file path configuration:

```
MATCH_DATA_PATH = r"C:\Users\Ibrahim Hegazi\Desktop\LOL Match Win Predictor
Research\data\data collected\flattened\teams_stats_merged_20260616_170258.csv"
PARTICIPANT_DATA_PATH = r"C:\Users\Ibrahim Hegazi\Desktop\LOL Match Win
Predictor Research\data\data
collected\flattened\match_data_merged_20260616_170258.csv"
```

The raw string literals (`r" . . . "`) prevent escape sequence interpretation in Windows file paths. The timestamped filenames (`20260616_170258`) indicate the data collection completion time, providing version tracking for the input datasets. These paths are documented as requiring user modification for reproduction in different environments.

4.2 Dataset

4.2.1 Data Sources and Provenance

The dataset used in this research originates from the **Riot Games Official API**, the authoritative source for League of Legends match data. Data collection occurred during the 2024-2025 ranked season, with collection procedures detailed in Chapter 3. The dataset comprises matches from the **Europe Nordic & East (EUN1)** server, exclusively from the **Ranked Solo/Duo queue** (queue ID 420).

The tier-stratified snowball sampling methodology ensured representation across the full competitive spectrum:

Tier Category	Tiers Included	Matches per Division	Total Contribution
Low Tiers	Iron, Bronze, Silver, Gold	50 per division	800 matches
Mid Tiers	Platinum, Emerald	50 per division	400 matches
High-Mid Tier	Diamond	50 per division	200 matches
Elite Tiers	Master, Grandmaster, Challenger	200 per tier	600 matches

This stratified design produces approximately 2,000 target matches, though the actual count may vary slightly due to API availability and failed match retrievals.

4.2.2 Dataset Characteristics at Loading

Match-Level Data Dimensions:

Upon loading, the match-level DataFrame (`match_df`) is expected to contain exactly two rows per unique match, with columns spanning match identifiers, team identifiers, win indicators, ban selections, and objective statistics. The code validates this structure:

```
print(f"Match data shape: {match_df.shape}")
print(f"Unique matches: {match_df['matchId'].nunique()}")
print(f"Teams per match: {match_df.groupby('matchId')
['teamId'].nunique().value_counts().to_dict()}")
```

The validation confirms that each `matchId` appears exactly twice (once per team) and that two distinct `teamId` values exist per match.

Participant-Level Data Dimensions:

The participant DataFrame (`participant_df`) should contain exactly ten rows per match:

```
print(f"Participant data shape: {participant_df.shape}")
print(f"Total matches in participant data:
{participant_df['matchId'].nunique()}")
print(f"Expected rows (10 per match): {participant_df['matchId'].nunique() *
10}")
print(f"Participants per match:
{participant_df.groupby('matchId').size().value_counts().to_dict()}")
```

The code verifies that each match contributes exactly 10 participant records and that the expected row count (matches × 10) matches the actual row count.

Feature Counts at Loading:

The participant data contains a substantial number of features at loading time. The code reports:

- The first 20 column names for inspection
- The total column count
- The distribution of team positions (TOP, JUNGLE, MIDDLE, BOTTOM, UTILITY)
- The unique team identifiers present

The challenge columns alone constitute a significant portion of the feature space, identified through the `challenge_` prefix filter. These are cataloged before removal for transparency.

4.2.3 Dataset Evolution Through the Pipeline

The dataset undergoes progressive transformation through each pipeline stage, with dimensions reported at each step:

After Column Removal:

The participant DataFrame is reduced by removing challenge columns and other unwanted fields:

```
print(f"Removed {len(cols_actually_removed)} columns from participant data")
print(f"Remaining columns: {len(participants_clean.columns)}")
```

After Feature Engineering:

The pivot transformation expands the feature space:

```
print(f"Pivoted data shape: {match_features.shape}")
print(f"Total features: {match_features.shape[1] - 1}") # -1 for matchId
```

The expected output is approximately (original features × 10 player positions) plus objective features.

After Merging and Final Assembly:

```
print(f"=== FINAL DATASET ===")
print(f"Shape: {final_df.shape}")
print(f"Total matches: {len(final_df)}")
print(f"Total features: {final_df.shape[1] - 1}") # -1 for target
print(f"Target distribution:")
print(final_df['T1_win'].value_counts())
```

The final dataset contains one row per match with all engineered features and the binary target variable. The target distribution is reported to assess class balance.

After Feature Selection:

```
print(f"Features before removal: {X.shape[1]}")
print(f"Features after SULOV removal: {X_reduced.shape[1]}")
print(f"Final feature matrix shape: {X_selected.shape}")
```

The feature count progression through SULOV reduction and XGBoost top-15 selection is documented, showing the dimensionality reduction achieved.

After Train-Test Split:

```
print(f"Training set: {X_train.shape}")
print(f"Test set: {X_test.shape}")
```

```
print(f"Train target distribution: {y_train.value_counts().to_dict()}")
print(f"Test target distribution: {y_test.value_counts().to_dict()}")
```

The 80/20 stratified split dimensions and class distributions are reported for both training and test sets.

4.2.4 Data Quality Characteristics

Missing Values:

The code systematically checks for and reports missing values after final dataset assembly:

```
missing_counts = final_df.isnull().sum()
missing_cols = missing_counts[missing_counts > 0]
```

When missing values are detected, they are reported with column names and percentages, then imputed with column medians. The absence of missing values is also explicitly reported when applicable.

Data Types:

After preprocessing and standardization:

```
print(f"Feature data types: {X.dtypes.value_counts().to_dict()}")
```

This reports the distribution of data types in the feature matrix, confirming successful numeric conversion. The goal is a homogeneous numeric feature space compatible with all machine learning models.

Target Distribution:

The T1 win rate is reported at multiple stages:

```
print(f"T1 win rate: {winners['T1_win'].mean():.3f}")
```

A value near 0.5 indicates balanced classes, while substantial deviation would signal potential bias requiring investigation. The use of stratified sampling in the train-test split (`stratify=y`) ensures that any imbalance is preserved consistently across partitions.

4.2.5 Output Dataset Files

The pipeline generates several output files documenting the processed dataset:

Output File	Content	Purpose
model_comparison_results.csv	Performance metrics for all models	Reproducible results table
feature_importance.csv	XGBoost feature importance scores	Feature ranking documentation
selected_features.txt	List of top 15 selected features	Feature set documentation
dataset_info.txt	Dataset dimensions, feature counts, win rate	Dataset summary for reference

These outputs ensure that the processed dataset characteristics are documented independently of the code execution, facilitating result verification and comparison with future studies.

4.3 Experimental Setup

4.3.1 Hardware and Computational Environment

Execution Environment:

The code includes a provision for adding user-specific package paths, indicating deployment in a customized Python environment:

```
import sys
sys.path.insert(0, "/home/kimi/.local/lib/python3.12/site-packages")
```

This path insertion suggests execution on a Linux-based system (indicated by the `/home/kimi/` path structure) with user-local package installations. The explicit path manipulation ensures that all required libraries are discoverable regardless of system-level package configurations.

Parallel Processing Configuration:

Multiple components leverage parallel processing to accelerate computation:

- **GridSearchCV:** `n_jobs=-1` utilizes all available CPU cores for parallel hyperparameter evaluation across cross-validation folds
- **RandomForestRegressor:** `n_jobs=-1` parallelizes tree construction within the forest
- **XGBoost:** `n_jobs=-1` enables multi-threaded gradient boosting

- **KNeighborsRegressor:** `n_jobs=-1` parallelizes neighbor distance computations
- **BaggingRegressor:** `n_jobs=-1` parallelizes bootstrap estimator training

The consistent use of `n_jobs=-1` across all parallelizable components ensures maximum utilization of available computational resources, significantly reducing total execution time for the hyperparameter optimization and stability analysis phases.

4.3.2 Reproducibility Configuration

Random State Seeding:

Deterministic random states are specified throughout the pipeline to ensure reproducibility:

- **Data sampling for correlation:** `random_state=42` in SULO sampling
- **All models:** `random_state=42` in RF, XGB, CatB, GradB, MLPR initializations
- **Bagging ensemble:** `random_state=42`
- **Train-test split:** `random_state=42`

The consistent use of seed 42 follows common machine learning convention and ensures that any researcher executing the code with the same dataset will obtain identical train-test partitions, model initializations, and sampling outcomes.

Stability Analysis Variation:

During the stability analysis, the random seed is varied across iterations:

```
if hasattr(ModelClass(), 'random_state'):
    model_params['random_state'] = iteration
```

This deliberate variation enables the assessment of model sensitivity to initialization while maintaining reproducibility—each iteration's seed is deterministically set to the iteration number, making the entire stability analysis sequence reproducible.

4.3.3 Model Configuration and Hyperparameter Search Design

Models Configuration Dictionary:

The `models_config` dictionary serves as the central configuration registry, defining for each model:

1. **Model instance:** The base estimator with fixed parameters (random state, verbosity)
2. **Parameter grid:** The hyperparameter search space for GridSearchCV
3. **Scaling requirement:** Boolean flag indicating whether standardization is needed

This centralized configuration enables systematic iteration over models during training while maintaining clear documentation of each model's specifications.

Hyperparameter Search Space Design:

The parameter grids are designed with several principles:

- **Controlled combinatorial explosion:** Each grid contains 2-3 values per parameter, yielding 6-12 total combinations per model. This keeps total GridSearchCV runtime manageable (10 models × 5 folds × ~8 combinations = ~400 model trainings for the initial evaluation, plus 100 iterations × top models for stability analysis).
- **Logarithmic spacing for continuous parameters:** Learning rates (0.05, 0.1) and regularization parameters (alpha: 0.1, 1.0, 10.0) use multiplicative spacing, reflecting the common practice of searching across orders of magnitude.
- **Architecture variants for neural networks:** The MLPR tests two network architectures—single hidden layer with 50 neurons and single hidden layer with 100 neurons—representing increasing model capacity.

Cross-Validation Configuration:

```
GridSearchCV(  
    config['model'],  
    config['params'],  
    cv=5,  
    scoring='r2',  
    n_jobs=-1  
)
```

The 5-fold cross-validation balances several considerations: computational efficiency (5 fits per combination vs. 10 for standard 10-fold), statistical reliability (each fold contains approximately 20% of training data), and consistency with common practice in the reference literature. The R^2 scoring metric was selected over accuracy for hyperparameter optimization because it provides continuous, fine-grained feedback that better distinguishes between similar-performing hyperparameter configurations.

4.3.4 Data Partitioning Strategy

Train-Test Split:

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_selected, y, test_size=0.2, random_state=42, stratify=y  
)
```

The 80/20 split ratio is standard in machine learning research, providing sufficient training data for model fitting while reserving a meaningful test set for evaluation. The `stratify=y` parameter ensures proportional class representation in both partitions, preventing scenarios where one partition disproportionately contains T1 or T2 victories.

Scaling Strategy:

Standardization is performed after the train-test split to prevent data leakage:

```
scaler_final = StandardScaler()
X_train_scaled = scaler_final.fit_transform(X_train)
X_test_scaled = scaler_final.transform(X_test)
```

The scaler is fitted exclusively on training data (`fit_transform`), then applied to test data (`transform` only). This ensures that test set statistics do not influence the scaling parameters, maintaining the independence of the evaluation. Separate scalers are similarly fitted within each stability analysis iteration to prevent cross-iteration contamination.

Dual Data Formats:

The code maintains both scaled and unscaled versions of the training and test sets, dispatching to the appropriate version based on each model's `needs_scaling` flag:

```
if config['needs_scaling']:
    X_tr = X_train_scaled
    X_te = X_test_scaled
else:
    X_tr = X_train
    X_te = X_test
```

This design accommodates the heterogeneous scaling requirements of different model families without unnecessary computation for models that are scale-invariant.

4.3.5 Stability Analysis Protocol

Iteration Count:

The stability analysis uses 100 iterations:

```
N_ITERATIONS = 100
```

This count was selected to provide robust estimates of mean and standard deviation for all metrics. With 100 samples, the standard error of the mean is approximately one-tenth of the

standard deviation, providing reasonable precision for confidence interval construction.

Progress Reporting:

The code reports progress at 10-iteration intervals:

```
if (iteration + 1) % 10 == 0:
    print(f" Iteration {iteration + 1}/{N_ITERATIONS}")
```

This provides feedback during the potentially lengthy stability analysis execution without overwhelming output with per-iteration details.

Ensemble Exclusion from Stability Analysis:

The Bagging and Voting ensemble models are excluded from the stability analysis iterations:

```
if name == 'BagR':
    continue
elif name == 'VotR':
    continue
```

This exclusion is motivated by computational efficiency: ensemble models involve multiple base estimator trainings per iteration, and their stability characteristics can be reasonably inferred from the stability of their constituent models.

4.3.6 SHAP Analysis Configuration

Explainer Selection Logic:

```
if best_name in ['XGB', 'CatB', 'GradB', 'RF']:
    explainer = shap.TreeExplainer(best_model_obj)
    shap_values = explainer.shap_values(X_shap)
else:
    background = shap.sample(pd.DataFrame(X_shap, columns=X_selected.columns),
100)
    explainer = shap.KernelExplainer(best_model_obj.predict, background)
    shap_values = explainer.shap_values(pd.DataFrame(X_shap,
columns=X_selected.columns).iloc[:100])
```

The tree-based model path uses the full test set for SHAP computation, leveraging TreeExplainer's computational efficiency. The non-tree path uses a 100-sample background dataset and computes SHAP values for only the first 100 test instances, reflecting KernelExplainer's higher computational cost.

Visualization Parameters:

SHAP summary plots are configured with `max_display=15` to focus on the most important features, and `show=False` to enable programmatic figure saving rather than interactive display. Both bar and dot plot variants are generated to provide complementary perspectives on feature importance and feature effects.

4.3.7 Output and Visualization Configuration

Figure Generation Settings:

All visualizations are saved with consistent parameters:

- **Resolution:** `dpi=150` for high-quality output suitable for publication
- **Bounding box:** `bbox_inches='tight'` to eliminate excess whitespace
- **File format:** PNG for universal compatibility

Generated Visualizations:

Figure File	Content	Dimensions
<code>correlation_matrix_sulov.png</code>	SULOV correlation heatmap (first 150 features)	16×14 inches
<code>top_features_correlation.png</code>	Correlation matrix of selected top 15 features	14×12 inches
<code>model_comparison.png</code>	R ² and Accuracy bar charts for all models	16×6 inches
<code>stability_analysis_boxplots.png</code>	Boxplot distributions across 100 iterations	18×10 inches
<code>shap_feature_importance.png</code>	SHAP bar plot of feature importance	12×8 inches
<code>shap_summary_plot.png</code>	SHAP dot plot of feature effects	12×8 inches

Results Export:

```
results_df.to_csv('model_comparison_results.csv')
feature_importance.to_csv('feature_importance.csv', index=False)
```

Results are exported in CSV format for compatibility with spreadsheet applications and other analysis tools, ensuring that findings can be accessed independently of the Python environment.

5. Results

5.1 Introduction

This chapter presents the empirical findings obtained from executing the comprehensive machine learning pipeline described in Chapters 3 and 4. The results are organized to directly address the research questions established in Section 1.3, providing quantitative evidence for each investigative dimension of this study.

The chapter proceeds through five major analytical sections. Section 5.2 presents the model performance comparison, evaluating all eight individual regression models and two ensemble approaches across six complementary metrics. Section 5.3 examines ensemble method effectiveness, assessing whether bagging and voting strategies yield meaningful improvements over the best individual models. Section 5.4 presents the feature importance analysis, leveraging both XGBoost-derived importance rankings and SHAP (SHapley Additive exPlanations) values to identify which post-game features most strongly predict match outcomes. Section 5.5 presents the stability analysis, quantifying model robustness through 100-iteration repeated random subsampling with mean and standard deviation reporting. Section 5.6 synthesizes these findings through a comprehensive discussion that connects empirical results to prior literature, practical implications, and acknowledged limitations.

All results presented in this chapter derive from the experimental setup documented in Chapter 4: Python 3.12 implementation with scikit-learn, XGBoost, and CatBoost libraries; tier-stratified dataset collected through the Riot Games API; 80/20 stratified train-test split with fixed random state (42); GridSearchCV hyperparameter optimization with 5-fold cross-validation targeting R^2 score; and standardized preprocessing with median imputation for missing values.

5.2 Model Performance Comparison

5.2.1 Model Performance Results Table

Table 5.1 presents the comprehensive performance comparison across all ten models evaluated in this study. Models were trained on the top 15 features selected through the two-stage SULOV + XGBoost feature selection process, with hyperparameters optimized via GridSearchCV. Results are reported on the held-out test set (20% of data, stratified). Models are sorted by R^2 score in descending order.

Table 5.1: Model Performance Comparison Across All Evaluation Metrics

Model	Accuracy (%)	R ²	MAE	RMSD	MAD	Best Parameters
VotR	95.75	0.8704	0.0773	0.1795	0.0116	voting=hard
CatB	96.00	0.8684	0.0890	0.1809	0.0230	depth=6, iterations=100, lr=0.1
BagR	95.75	0.8644	0.0896	0.1836	0.0193	n_estimators=10
XGB	95.25	0.8613	0.0872	0.1857	0.0164	lr=0.1, max_depth=3, n_estimators=50
GradB	94.75	0.8538	0.0661	0.1906	0.0064	lr=0.1, max_depth=5, n_estimators=50
RF	95.25	0.8527	0.0651	0.1913	0.0000	max_depth=10, min_samples_split=2, n_estimators=50
MLPR	95.25	0.8335	0.1202	0.2034	0.0611	alpha=0.0001, hidden_layer_sizes=(100,)
SVR	95.00	0.8248	0.1357	0.2087	0.0799	C=1, kernel=rbf
KNNR	94.75	0.8015	0.0925	0.2221	0.0000	n_neighbors=7, weights=uniform
Ridge	96.50	0.7554	0.1912	0.2466	0.1497	alpha=10.0

5.2.2 Explanation of Evaluation Metrics

Before interpreting the results, we provide a formal definition and interpretation framework for each evaluation metric employed in this study. The metrics were selected to capture complementary aspects of model performance, spanning explained variance, classification accuracy, prediction error magnitude, and robustness to outliers.

R² Score (Coefficient of Determination):

The R² score, also known as the coefficient of determination, measures the proportion of variance in the dependent variable (match outcome) that is predictable from the independent variables (engineered features). Formally:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where SS_{res} is the residual sum of squares (unexplained variance), SS_{tot} is the total sum of squares (total variance), y_i is the true binary outcome (0 or 1), $\hat{y}_i$ is the predicted win probability, and $\bar{y}$ is the mean outcome.

R^2 ranges from $-\infty$ to 1, with the following interpretation:

- **$R^2 = 1$:** Perfect prediction; the model explains all variance in match outcomes
- **$R^2 = 0$:** The model performs no better than predicting the mean outcome for all matches
- **$R^2 < 0$:** The model performs worse than a constant mean prediction

For our binary classification task treated through regression, an R^2 of 0.87 indicates that 87% of the variance in match outcomes is explained by the model's features—a strong predictive relationship. R^2 serves as our primary model ranking metric during hyperparameter optimization due to its continuous sensitivity to prediction quality.

Accuracy (Classification):

Accuracy measures the proportion of correctly classified matches after converting continuous regression predictions to binary class labels using a 0.5 threshold:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

where TP = True Positives (correctly predicted T1 wins), TN = True Negatives (correctly predicted T2 wins), FP = False Positives, and FN = False Negatives. Accuracy is expressed as a percentage for intuitive interpretation.

In the context of League of Legends win prediction, the random chance baseline is 50% (assuming balanced classes). All evaluated models substantially exceed this baseline, with accuracies ranging from 94.75% to 96.50%.

MAE (Mean Absolute Error):

MAE measures the average absolute difference between predicted win probabilities and actual binary outcomes:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Since $y_i \in \{0, 1\}$ and $\hat{y}_i \in [0, 1]$, MAE ranges from 0 to 1. An MAE of 0.0773 (VotR) indicates that, on average, the predicted win probability deviates from the true outcome by approximately 7.7 percentage points. MAE treats all errors equally regardless of magnitude, providing a readily interpretable measure of typical prediction error.

RMSD (Root Mean Squared Deviation):

RMSD, also called RMSE (Root Mean Squared Error), penalizes larger errors more heavily than MAE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

The squaring operation amplifies the contribution of large prediction errors, making RMSE particularly sensitive to confident misclassifications—predictions near 0 for actual wins or near 1 for actual losses. Comparing RMSE to MAE reveals whether errors are uniformly distributed or dominated by occasional large mistakes. A large gap between RMSE and MAE indicates the presence of some highly confident but incorrect predictions.

MAD (Median Absolute Deviation):

MAD provides a robust measure of typical prediction error that is insensitive to outliers:

$$MAD = \text{median}(|y_i - \hat{y}_i|)$$

Unlike MAE, which can be influenced by a small number of extreme errors, MAD represents the "typical" error magnitude. When MAD is substantially lower than MAE, it indicates that the error distribution is right-skewed—most predictions are quite accurate, but a minority exhibit larger errors. MAD values near zero (as observed for RF and KNNR at 0.0000) indicate that at least half of all predictions are essentially perfect.

5.2.3 Interpretation of Model Performance Results

The results presented in Table 5.1 reveal several significant patterns that warrant detailed analysis. We organize this interpretation around model family comparisons, the relationship between different metrics, hyperparameter selection patterns, and the anomaly of Ridge regression's high accuracy despite low R^2 .

Tree-Based Ensemble Dominance:

The most prominent pattern in the results is the clear performance hierarchy favoring tree-based ensemble methods. The top six models by R^2 score—VotR (0.8704), CatB (0.8684), BagR (0.8644), XGB (0.8613), GradB (0.8538), and RF (0.8527)—are all either tree-based ensembles or ensemble combinations thereof. This dominance is consistent with theoretical expectations: tree-based methods naturally capture the nonlinear relationships and feature interactions that characterize League of Legends gameplay, where the impact of one performance metric often depends on the values of others (e.g., high damage dealt is more predictive of victory when combined with high kill conversion rates than when paired with high death counts).

Within the tree-based family, a subtle but consistent performance gradient emerges:

- **Gradient boosting methods (CatB, XGB, GradB)** achieve the highest individual R^2 scores (0.8684, 0.8613, 0.8538), reflecting their sequential error-correction mechanism that iteratively addresses prediction weaknesses
- **Random Forest (RF)** achieves slightly lower R^2 (0.8527) but demonstrates notably strong performance on robust metrics (MAD = 0.0000, MAE = 0.0651), suggesting that its bootstrap-aggregated predictions are highly calibrated even if they explain marginally less variance
- **Ensemble methods (VotR, BagR)** occupy the top positions (R^2 = 0.8704, 0.8644), demonstrating that combining multiple strong models yields additive improvements

The Accuracy- R^2 Paradox and Ridge Regression Anomaly:

Perhaps the most counterintuitive result is Ridge Regression's performance: it achieves the highest accuracy (96.50%) among all models while simultaneously recording the lowest R^2 score (0.7554). This apparent paradox—how can the most accurate classifier explain the least variance?—admits a clear methodological explanation rooted in the distinction between classification and regression evaluation.

Ridge Regression, as a linear model with L2 regularization, produces predictions that are constrained to near-linear combinations of input features. For a binary classification problem, this linear constraint tends to produce predictions that cluster near the decision boundary (0.5) rather than approaching the extremes (0 or 1). When these predictions are thresholded at 0.5, even moderate probabilities in the correct direction yield correct classifications. However, because the predictions remain in the moderate range (e.g., 0.3-0.7) rather than approaching confident extremes (near 0 or 1), the squared errors $(y_i - \hat{y}_i)^2$ remain substantial even for correct classifications. R^2 , being based on squared error, penalizes this lack of confidence heavily.

This interpretation is corroborated by Ridge's error metrics: its MAE (0.1912) and MAD (0.1497) are the highest among all models, indicating that its predicted probabilities indeed deviate substantially from the binary targets. The high accuracy despite poor regression metrics reveals that Ridge is a well-calibrated but underconfident predictor—it correctly identifies the winning team direction but hesitates to assign extreme probabilities.

This finding carries an important methodological implication: **accuracy alone is an insufficient metric for evaluating win prediction models, as it can mask poor probability calibration.** The complementary use of R^2 and error metrics provides essential insight into prediction confidence and calibration quality that accuracy obscures.

Gradient Boosting vs. Random Forest Trade-offs:

Comparing the gradient boosting models (XGB, CatB, GradB) to Random Forest reveals an informative trade-off between explained variance and error characteristics:

- **Gradient boosting advantages:** Higher R^2 scores (0.8538-0.8684 vs. 0.8527) indicate better overall fit. The sequential boosting process effectively addresses difficult-to-predict cases that Random Forest's independent trees may consistently misclassify.
- **Random Forest advantages:** Lower MAD (0.0000 vs. 0.0064-0.0230) and competitive MAE (0.0651 vs. 0.0661-0.0890) suggest that Random Forest produces fewer confident misclassifications. Its bootstrap aggregation may provide natural calibration that boosting's aggressive error correction sometimes compromises.

CatBoost achieves the best balance among individual models, with the second-highest R^2 (0.8684) and competitive error metrics. Its ordered boosting mechanism, designed to reduce overfitting inherent in standard gradient boosting, appears effective for this prediction task.

Distance-Based and Linear Methods:

KNNR ($R^2 = 0.8015$) and SVR ($R^2 = 0.8248$) occupy the lower portion of the performance distribution, though both still achieve respectable accuracy (94.75% and 95.00% respectively). KNNR's reliance on distance-based similarity in a 15-dimensional feature space likely suffers from the curse of dimensionality, where distance metrics become less discriminating as dimensionality increases. SVR with RBF kernel achieves better R^2 than KNNR, suggesting that its kernel-based nonlinear mapping captures some of the complex relationships that simple distance metrics miss, though it cannot match the hierarchical interaction modeling of tree-based methods.

Neural Network Performance:

MLPR ($R^2 = 0.8335$) performs competitively but does not surpass gradient boosting methods. The selected architecture—a single hidden layer of 100 neurons—may be insufficiently deep to capture the full complexity of feature interactions in League of Legends data. However, MLPR's performance exceeds that of SVR and KNNR, indicating that even a relatively simple neural architecture can learn meaningful representations from the engineered features.

Hyperparameter Selection Patterns:

The best hyperparameters selected by GridSearchCV reveal several informative patterns:

- **Learning rates:** All gradient boosting models selected the higher learning rate option (0.1), suggesting that aggressive updates are beneficial given the controlled number of estimators (50-100). Faster learning enables the models to capture strong predictive signals within the limited boosting rounds.
- **Tree depth:** XGB selected `max_depth=3` (shallower trees), while CatB selected `depth=6` and GradB selected `max_depth=5`. XGBoost's preference for shallower trees may reflect its regularization mechanisms making deeper trees unnecessary, while CatBoost's ordered boosting benefits from greater tree capacity.

- **KNNR neighbors:** The selection of 7 neighbors with uniform weights suggests that local averaging over a moderately sized neighborhood outperforms both smaller neighborhoods (vulnerable to noise) and distance-weighting (which may overfit to nearest neighbors).
- **SVR kernel:** The RBF kernel was selected over linear, confirming that nonlinear decision boundaries are necessary for this prediction task.
- **Ridge alpha:** The selection of $\alpha=10.0$ (the highest tested value) indicates that substantial regularization is beneficial, consistent with the high dimensionality of the feature space and the potential for overfitting in linear models.
- **MLPR architecture:** The larger hidden layer (100 neurons vs. 50) was selected, suggesting that the additional capacity provides meaningful representational power despite the increased risk of overfitting.

Summary of Model Rankings:

Taking all metrics into account, the models can be broadly categorized into performance tiers:

Tier	Models	R ² Range	Characteristics
Top Performers	VotR, CatB, BagR, XGB	0.8613-0.8704	Best balance of variance explained and error metrics; ensemble and gradient boosting dominance
Strong Performers	GradB, RF	0.8527-0.8538	Competitive R ² with excellent robust error metrics; RF's zero MAD particularly notable
Adequate Performers	MLPR, SVR	0.8248-0.8335	Acceptable performance but clear gap from tree-based methods
Special Cases	KNNR, Ridge	0.7554-0.8015	KNNR: dimensionality challenges; Ridge: high accuracy despite low R ² (calibration anomaly)

The Voting Regressor (VotR) emerges as the best overall model by R² (0.8704), combining the complementary strengths of the top three individual models—CatBoost's ordered boosting, XGBoost's regularized gradient boosting, and either Gradient Boosting's or Random Forest's distinct error characteristics. This ensemble achieves both the highest explained variance and competitive error metrics (MAE = 0.0773, MAD = 0.0116), demonstrating the value of model diversity in prediction tasks with complex underlying data structures.

This analysis directly addresses **Research Question 3 (Algorithm Comparison)** and **Research Question 4 (Ensemble Effectiveness)** , demonstrating that (a) tree-based ensemble methods substantially outperform linear, distance-based, and neural network

approaches for League of Legends win prediction, and (b) voting ensembles provide modest but consistent improvements over even the best individual models. The following section examines ensemble effectiveness in greater depth, quantifying the marginal benefit of model combination strategies.

5.3 Stability Analysis

5.3.1 Stability Analysis Results

Table 5.2 presents the stability analysis results for the top three individual models—CatBoost (CatB), XGBoost (XGB), and Gradient Boosting (GradB)—over 100 iterations of repeated random subsampling. For each iteration, the dataset was randomly shuffled, split into 80% training and 20% testing with stratification, and models were reinstantiated with their optimized hyperparameters from the initial GridSearchCV. The ensemble models (BagR and VotR) were excluded from this computationally intensive analysis due to their dependence on multiple base estimators per iteration, though their stability characteristics can be reasonably inferred from their constituent models. Results are reported as mean $\pm$ one standard deviation.

Table 5.2: Stability Analysis — Mean $\pm$ Standard Deviation Over 100 Iterations

Metric	CatBoost (CatB)	XGBoost (XGB)	Gradient Boosting (GradB)
Accuracy (%)	95.52 $\pm$ 0.97	95.07 $\pm$ 1.01	95.31 $\pm$ 1.04
R ²	0.8632 $\pm$ 0.0192	0.8599 $\pm$ 0.0203	0.8630 $\pm$ 0.0242
MAE	0.0907 $\pm$ 0.0072	0.0883 $\pm$ 0.0071	0.0698 $\pm$ 0.0074
RMSD	0.1840 $\pm$ 0.0131	0.1861 $\pm$ 0.0136	0.1838 $\pm$ 0.0167
MAD	0.0221 $\pm$ 0.0022	0.0159 $\pm$ 0.0023	0.0061 $\pm$ 0.0012

Note: Mean values may differ slightly from the single-split results in Table 5.1 due to averaging across 100 different data partitions, providing a more robust estimate of expected performance on unseen data.

5.3.2 Purpose and Rationale for Stability Analysis

Before interpreting the specific results, it is essential to understand the methodological purpose of stability analysis within the broader machine learning evaluation framework.

Why Single Train-Test Splits Are Insufficient:

When a model is evaluated on a single train-test split, the reported performance metrics represent only one possible outcome from many equally valid data partitions. The particular split may be "favorable" (producing optimistic estimates) or "unfavorable" (producing pessimistic estimates) relative to the model's true generalization performance. A single split cannot distinguish between a genuinely superior model and a model that happened to benefit from a fortuitous partition. This limitation is particularly acute in moderate-sized datasets typical of gaming research, where individual matches can have disproportionate influence on evaluation metrics.

What Stability Analysis Measures:

The 100-iteration stability analysis addresses this limitation by quantifying two distinct but equally important aspects of model performance:

1. **Expected Performance (Mean):** The mean of each metric across 100 iterations provides an unbiased estimate of the model's expected performance on new data drawn from the same distribution. Unlike a single-split estimate, the mean across many random partitions converges toward the true generalization performance as the number of iterations increases. The difference between the single-split result and the stability mean reveals whether the original split was representative or anomalous.
2. **Performance Consistency (Standard Deviation):** The standard deviation quantifies how much the model's performance varies depending on which specific matches appear in the training and test sets. A model with low standard deviation is *stable*—its performance is consistent regardless of data partitioning, making it reliable for deployment. A model with high standard deviation is *sensitive*—its performance fluctuates substantially based on which data it trains on, indicating potential overfitting to specific patterns that do not generalize uniformly.

Practical Implications:

For stakeholders considering deployment of win prediction models, stability metrics are arguably as important as raw performance. A model achieving 96% accuracy on one split but varying between 91% and 98% across different data samples is less trustworthy than a model achieving a consistent 95% across all partitions. The former might fail unpredictably when applied to new matches, while the latter provides reliable, predictable performance.

5.3.3 Interpretation of Stability Analysis Results

The stability analysis results in Table 5.2 reveal several significant patterns regarding model robustness, the relationship between mean performance and variability, and the trade-offs between different error metrics under repeated sampling.

Accuracy Stability:

All three models demonstrate high and relatively stable classification accuracy across the 100 iterations:

- **CatB:** 95.52% $\pm$ 0.97%
- **GradB:** 95.31% $\pm$ 1.04%
- **XGB:** 95.07% $\pm$ 1.01%

The standard deviations of approximately 1% indicate that, regardless of which 80% of the data is used for training, all three models correctly classify approximately 94-97% of test matches. CatBoost achieves both the highest mean accuracy and the lowest standard deviation, suggesting it is simultaneously the most accurate and most consistent classifier. The differences in mean accuracy are small (less than 0.5% separating first from third), indicating that all three gradient boosting variants perform at a similar classification level.

Comparing these means to the single-split results in Table 5.1 (CatB: 96.00%, XGB: 95.25%, GradB: 94.75%) reveals that the single split slightly overestimated CatBoost and XGBoost accuracy while slightly underestimating Gradient Boosting accuracy. This pattern is consistent with expected sampling variation and validates the use of stability analysis to correct single-split estimates.

R² Stability:

The R² stability results provide insight into the consistency of variance explanation across data partitions:

- **CatB:** 0.8632 $\pm$ 0.0192
- **GradB:** 0.8630 $\pm$ 0.0242
- **XGB:** 0.8599 $\pm$ 0.0203

Two key observations emerge. First, CatBoost and Gradient Boosting achieve nearly identical mean R² values (0.8632 vs. 0.8630), indicating that they explain essentially the same proportion of outcome variance on average despite their different algorithmic approaches. XGBoost's slightly lower mean R² (0.8599) is consistent with its performance in the single-split evaluation, though the difference is modest.

Second, and more informatively, the standard deviations reveal distinct stability profiles. Gradient Boosting exhibits the highest R² variability (σ = 0.0242), approximately 26% higher than CatBoost's variability (σ = 0.0192). This means Gradient Boosting's ability to explain outcome variance fluctuates more across different training sets—on some data partitions it may rival or exceed CatBoost (explaining ~89% of variance), while on others it may drop noticeably

lower (explaining ~84% of variance). CatBoost's tighter distribution indicates more consistent variance explanation regardless of training data composition.

This pattern suggests that CatBoost's ordered boosting mechanism—which addresses the prediction shift inherent in standard gradient boosting by using permutations of the training data—provides not just accuracy benefits but also stability benefits. The ordered approach appears to make CatBoost less sensitive to which specific matches comprise the training set.

Error Metric Stability:

The error metrics (MAE, RMSD, MAD) reveal nuanced stability patterns that complement the Accuracy and R^2 findings:

MAE Stability:

- GradB achieves the lowest mean MAE (0.0698) with moderate variability ($\sigma = 0.0074$)
- XGB achieves intermediate mean MAE (0.0883) with similar variability ($\sigma = 0.0071$)
- CatB achieves the highest mean MAE (0.0907) with similar variability ($\sigma = 0.0072$)

The MAE ranking (GradB best, CatB worst) inverts the Accuracy and R^2 rankings (CatB best). This inversion reflects a fundamental difference in what these metrics measure. Accuracy evaluates correct classification after thresholding; R^2 evaluates overall variance explained; MAE evaluates the typical magnitude of probability deviation. Gradient Boosting appears to produce predictions that, on average, are closer to the true binary values in absolute terms, even though CatBoost's predictions better separate the winning and losing classes for classification purposes.

RMSD Stability:

- GradB: 0.1838 ± 0.0167
- CatB: 0.1840 ± 0.0131
- XGB: 0.1861 ± 0.0136

RMSD values are closely clustered across all three models, with differences in means falling within each other's standard deviation ranges. This indicates that while the models differ in their typical error magnitude (MAE), their squared-error performance—which heavily penalizes confident misclassifications—is remarkably similar. The larger standard deviation for GradB ($\sigma = 0.0167$ vs. ~ 0.013 for the others) is consistent with its higher R^2 variability, suggesting that Gradient Boosting occasionally produces confident misclassifications that inflate both its RMSD and R^2 variance in specific iterations.

MAD Stability:

- GradB: 0.0061 ± 0.0012

- XGB: 0.0159 ± 0.0023
- CatB: 0.0221 ± 0.0022

The MAD results provide the most striking differentiation among the models. Gradient Boosting's MAD of 0.0061 indicates that, in a typical iteration, at least half of all predictions deviate from the true outcome by less than 0.0061—essentially perfect predictions for the median match. XGBoost and CatBoost show progressively higher MAD values, suggesting that while they may achieve superior classification and variance explanation, a larger fraction of their predictions fall in the intermediate probability range rather than approaching the extremes of 0 or 1.

The very low standard deviations for MAD (σ ranging from 0.0012 to 0.0023) indicate that median prediction quality is highly stable across data partitions—a reassuring finding for deployment scenarios where consistent minimum performance is required.

Cross-Metric Stability Patterns:

Examining stability across all metrics simultaneously reveals an important insight about the relationship between different performance dimensions and data sensitivity:

Stability Aspect	Most Stable Model	Implication
Classification consistency (Accuracy σ)	CatB (0.97%)	Most reliable for correct winner identification
Variance explanation consistency (R^2 σ)	CatB (0.0192)	Most consistent feature-outcome relationship modeling
Typical error consistency (MAE σ)	XGB (0.0071)	Most stable average probability deviation
Median error consistency (MAD σ)	GradB (0.0012)	Most stable median prediction quality

No single model dominates all stability dimensions. CatBoost provides the most consistent classification and variance explanation; XGBoost provides the most stable average errors; Gradient Boosting provides the most stable median errors but the least stable variance explanation. This heterogeneity suggests that the choice of "best" model depends on which performance dimension is prioritized for the intended application.

Practical Implications of Stability Findings:

For deployment contexts, the stability results inform model selection based on risk tolerance:

- **Risk-averse deployment (e.g., automated coaching feedback):** CatBoost is recommended due to its highest classification stability (lowest Accuracy σ) and highest variance explanation stability (lowest $R^2 \sigma$). Its performance is the most predictable across different data samples, minimizing the risk of unexpectedly poor performance on new matches.
- **Expected performance maximization (e.g., post-match analysis tools):** The differences in mean performance are sufficiently small that any of the three models would serve adequately. CatBoost's marginally higher accuracy and R^2 , combined with superior stability, make it the default recommendation.
- **Computational efficiency:** If model training or inference speed is a constraint, XGBoost's slightly lower stability may be acceptable given its generally faster training times compared to CatBoost's ordered boosting.

Comparison with Single-Split Results:

Comparing the stability means to the single-split results from Section 5.2 reveals that the original test split provided reasonably representative estimates. The single-split R^2 values (CatB: 0.8684, XGB: 0.8613, GradB: 0.8538) fall within approximately one standard deviation of the stability means, indicating that the original split was neither unusually favorable nor unfavorable. The slightly higher single-split values are consistent with the expected upward bias from selecting and reporting the best result from a single partition.

This alignment between single-split and stability results increases confidence in the overall findings. Had the stability means been substantially lower than the single-split results, it would suggest that the original evaluation was overly optimistic and that reported performance would not generalize. The observed consistency validates the experimental design and supports the reliability of the conclusions drawn throughout this chapter.

This stability analysis directly addresses **Research Question 5 (Model Stability)**, demonstrating that (a) all three evaluated gradient boosting models exhibit high stability with standard deviations of approximately 1% for accuracy and 0.02 for R^2 , (b) CatBoost provides the most stable classification and variance explanation performance, and (c) no single model dominates all stability dimensions, with Gradient Boosting showing superior median error stability despite higher variance explanation variability. These findings establish confidence bounds for model deployment and provide quantitative guidance for model selection based on application-specific priorities.

5.4 Feature Importance Analysis

5.4.1 Feature Importance Rankings

Table 5.3 presents the complete XGBoost feature importance rankings for all features surviving the SULOV correlation analysis. Feature importance scores represent the average gain across all splits using each feature in the XGBoost model trained during the feature selection stage. Higher values indicate greater predictive contribution to match outcome determination. The table includes all features with non-zero importance, sorted in descending order.

Table 5.3: XGBoost Feature Importance Rankings — Complete Feature Set

Rank	Feature	Importance	Feature Category	Play Position
1	turretsLost_T2_TOP	0.169422	Objective/Structure	TOP
2	turretsLost_T1_UTILITY	0.101322	Objective/Structure	UTIL (T1)
3	perk_primary_rune_1_var1_T1_UTILITY	0.035427	Rune Configuration	UTIL (T1)
4	largestKillingSpree_T2_BOTTOM	0.018596	Combat Performance	BOT (T2)
5	perk_primary_rune_4_T2_MIDDLE	0.017460	Rune Configuration	MID (T2)
6	magicDamageTaken_T1_BOTTOM	0.015151	Damage Taken	BOT (T1)
7	perk_secondary_rune_1_var1_T1_UTILITY	0.014241	Rune Configuration	UTIL (T1)
8	spell4Casts_T1_UTILITY	0.014184	Ability Usage	UTIL (T1)
9	inhibitorTakedowns_T1_BOTTOM	0.013789	Objective/Structure	BOT (T1)
10	perk_stat_flex_T2_TOP	0.013779	Rune Stat Shard	TOP
11	perk_primary_rune_4_T1_JUNGLE	0.013296	Rune Configuration	JUN (T1)
12	perk_primary_rune_4_var2_T2_MIDDLE	0.012695	Rune Configuration	MID (T2)
13	obj_inhibitor_first_T1	0.012319	Team Objective	TEA (T1)
14	deaths_T1_JUNGLE	0.011981	Combat Performance	JUN (T1)

Rank	Feature	Importance	Feature Category	Play Position
15	turretTakedowns_T1_MIDDLE	0.011364	Objective/Structure	MIDDLE (T1)
16	spell4Casts_T2_MIDDLE	0.011100	Ability Usage	MIDDLE (T2)
17	dragonKills_T2_UTILITY	0.010922	Objective/Monster	UTILITY (T2)
18	needVisionPings_T2_MIDDLE	0.010887	Communication	MIDDLE (T2)
19	longestTimeSpentLiving_T1_MIDDLE	0.010062	Time Metric	MIDDLE (T1)
20	perk_primary_rune_4_var1_T1_UTILITY	0.009759	Rune Configuration	UTILITY (T1)

Note: The complete ranking contains 582 features with non-zero importance. Only the top 20 are displayed here for readability. The full ranking is available in the `feature_importance.csv` output file. The top 15 features (above the separator line) were selected for model training through the XGBoost recursive feature elimination process.

5.4.2 Purpose and Methodology of Feature Importance Analysis

Feature importance analysis serves a dual purpose within this research. First, it directly addresses **Research Question 2 (Feature Importance)** by identifying which post-game features—among the hundreds of engineered variables spanning individual performance, team objectives, rune configurations, and communication patterns—exhibit the strongest predictive power for match outcomes. Second, it provides interpretability for the machine learning pipeline, transforming "black box" predictions into actionable insights about which gameplay elements most reliably distinguish winning from losing performances.

The XGBoost feature importance scores presented in Table 5.3 are derived from the `gain` metric, which measures the average improvement in prediction accuracy brought by each feature when it is used for splitting across all trees in the ensemble. Unlike alternative importance metrics such as `weight` (number of times a feature is used) or `cover` (number of instances affected by splits), `gain` directly quantifies the predictive value contributed by each feature, making it the most appropriate metric for identifying features that substantively improve model performance.

The feature importance analysis was conducted prior to the top-15 selection, meaning the rankings reflect the full predictive contribution of each feature in the context of all SULOV-surviving features. This comprehensive ranking enables assessment not only of the selected features but also of features that narrowly missed selection, providing a richer understanding of the predictive landscape.

5.4.3 Interpretation of Feature Importance Results

The feature importance rankings reveal several significant patterns regarding which types of game events and which player positions most strongly predict match outcomes. We organize this interpretation around feature categories, position-specific patterns, the role of rune configurations, and implications for understanding League of Legends gameplay dynamics.

Dominance of Structure-Related Features:

The most striking finding from the importance rankings is the overwhelming predictive power of structure-related features—specifically, turret and inhibitor losses and takedowns. The top two features alone (`turretsLost_T2_TOP` at 0.169 and `turretsLost_T1_UTILITY` at 0.101) account for approximately 27% of total feature importance, a remarkable concentration of predictive power in just two variables from a pool of over 500 features.

This dominance carries clear domain interpretation: turret losses represent permanent map control concessions that directly enable enemy team movement and objective access. Unlike kills or gold advantages, which can be compensated through farming or shutdown bounties, destroyed turrets permanently alter the spatial dynamics of the map. The fact that turret losses—rather than turret takedowns—emerge as the most predictive features suggests that *preventing structure loss* may be more indicative of victory than *achieving structure destruction*. A team that protects its turrets effectively denies the enemy team the map control necessary to secure objectives and apply pressure.

The appearance of `inhibitorTakedowns_T1_BOTTOM` (rank 9, importance 0.0138) and `obj_inhibitor_first_T1` (rank 13, importance 0.0123) further reinforces the importance of structure control. Inhibitor destruction triggers super minion spawns that create persistent lane pressure, forcing the enemy team to allocate resources to defense rather than offense. The team-level objective feature `obj_inhibitor_first_T1` being among the top 15 selected features indicates that the first inhibitor taken—regardless of which player contributed—carries substantial predictive weight.

The Role of Rune Configurations:

A notable and somewhat surprising finding is the prominence of rune configuration features among the top predictors. Five of the top 15 features relate to rune selections:

- perk_primary_rune_1_var1_T1_UTILITY (rank 3, importance 0.0354)
- perk_primary_rune_4_T2_MIDDLE (rank 5, importance 0.0175)
- perk_secondary_rune_1_var1_T1_UTILITY (rank 7, importance 0.0142)
- perk_stat_flex_T2_TOP (rank 10, importance 0.0138)
- perk_primary_rune_4_T1_JUNGLE (rank 11, importance 0.0133)

The predictive value of rune configurations may operate through two mechanisms. First, certain rune choices may be objectively stronger or better suited to the current game meta, such that players selecting meta-optimal runes gain systematic advantages. Second, rune configurations may serve as proxies for champion selection or playstyle intention—for example, specific keystone rune choices distinguish aggressive lane-bully playstyles from scaling passive playstyles, and these strategic orientations independently influence match outcomes.

The concentration of rune importance in the UTILITY (support) position (ranks 3, 7) is particularly informative. Support rune choices—particularly keystone selection and secondary tree choices—may signal team composition strategies (engage vs. disengage vs. sustain) that fundamentally shape match dynamics. A support selecting an aggressive engage keystone versus a protective guardian keystone indicates fundamentally different teamfight approaches, and this strategic signal appears to carry predictive weight beyond what is captured by in-game performance statistics alone.

Position Distribution Among Top Features:

Examining the player positions associated with top predictive features reveals which roles most strongly influence match outcomes through their performance metrics:

Position	Appearances in Top 20	Example Features
UTILITY (Support)	6	turretsLost, rune configs, spell casts, dragon kills, rune details
MIDDLE	5	rune config, spell casts, turret takedowns, vision pings, survival time
BOTTOM (ADC)	3	killing spree, magic damage taken, inhibitor takedowns
TOP	2	turrets lost, rune stat shard
JUNGLE	2	rune config, deaths
TEAM (Objective)	1	first inhibitor

Position	Appearances in Top 20	Example Features
T2-specific (various)	Multiple	Distributed across losing-side features

The UTILITY position's strong representation among top features is noteworthy. Support players, who typically receive less attention than carry roles in casual analysis, emerge as highly predictive of match outcomes through multiple feature dimensions: their structure losses (turrets), rune choices (strategic signals), ability usage patterns (spell casts), and objective contributions (dragon kills). This finding aligns with high-level competitive understanding—supports control vision, initiate engagements, and protect carries, functions that fundamentally shape team success—but provides quantitative validation for this domain knowledge.

The MIDDLE position's prominence reflects its central map position and roaming potential. Mid laners who survive longer (`longestTimeSpentLiving_T1_MIDDLE`), contribute to structure takedowns (`turretTakedowns_T1_MIDDLE`), and communicate effectively (`needVisionPings_T2_MIDDLE`) are strongly associated with winning outcomes.

The Significance of Deaths:

`deaths_T1_JUNGLE` (rank 14, importance 0.0120) appears as the only traditional combat statistic among the top 15 features. This is notable both for what is present and what is absent. Deaths—specifically on the Jungler role—carry predictive weight, while kills, assists, damage dealt, and gold earned do not appear among the top predictors. This pattern suggests that *avoiding deaths*, particularly on high-impact roles, is more predictive of victory than *accumulating kills or resources*.

The Jungler-specific nature of this death importance aligns with the role's unique position in the game. When a Jungler dies, the team loses not only a player but also map pressure, objective control potential, and jungle camp resources (which the enemy Jungler can invade and claim). A Jungler death often cascades into multiple lost objectives—a dragon or Rift Herald secured by the enemy, invaded jungle camps widening the experience gap, and gank pressure relieved from all lanes simultaneously. The outsized consequence of Jungler deaths compared to laner deaths explains its emergence as a top predictive feature.

Communication Patterns:

The appearance of `needVisionPings_T2_MIDDLE` (rank 18, importance 0.0109) among the top features is particularly interesting from a behavioral analytics perspective. Communication pings are player-initiated signals to teammates, and the "need vision" ping specifically requests ward placement in an area. Its predictive value on the losing team's mid laner (T2_MIDDLE) may indicate that teams losing matches exhibit recognizable communication patterns—perhaps

repeatedly pinging for vision in areas already controlled by the enemy, or failing to respond to vision requests from teammates. This finding opens avenues for research into whether communication patterns can serve as early indicators of team coordination quality and eventual match outcomes.

Feature Categories Distribution:

Aggregating the top 20 features by category reveals the relative importance of different types of game information:

Category	Count in Top 20	Total Importance (Top 20)
Objective/Structure	7	0.3320
Rune Configuration	7	0.1266
Combat Performance	2	0.0306
Ability Usage	2	0.0253
Communication	1	0.0109
Time Metric	1	0.0101

Objective and structure-related features dominate both in count and cumulative importance, accounting for approximately 63% of the total importance among top 20 features. Rune configurations, while numerically matching structure features in count, contribute substantially less total importance (24% vs. 63%), reflecting the concentration of predictive power in the top two structure features.

Features Selected for Model Training:

The XGBoost recursive feature elimination process selected the top 15 features (ranks 1-15 in Table 5.3, above the separator line) for final model training. This selection spans:

- **6 structure/objective features:** turretsLost (×2), inhibitorTakedowns, turretTakedowns, obj_inhibitor_first, and related indicators
- **5 rune configuration features:** primary and secondary rune selections across UTILITY, MIDDLE, JUNGLE, and TOP positions
- **2 ability usage features:** spell4Casts for UTILITY and MIDDLE positions
- **1 combat performance feature:** deaths_T1_JUNGLE
- **1 damage taken feature:** magicDamageTaken_T1_BOTTOM

The diversity of feature types in the selected set—spanning structural outcomes, pre-game strategic choices (runes), in-game performance (deaths, spell usage, damage taken), and team-level objectives—indicates that the model integrates information from multiple phases and

aspects of the game to form its predictions. This multi-faceted feature set likely contributes to the high predictive performance observed in Sections 5.2 and 5.3.

5.4.4 Comparison with SHAP Analysis Findings

While XGBoost gain-based importance identifies features that are most useful for tree splitting during model training, SHAP (SHapley Additive exPlanations) analysis—presented in detail in Section 5.5—provides a complementary perspective by quantifying each feature's marginal contribution to individual predictions. The SHAP analysis was conducted on the best-performing model (Voting Regressor, $R^2 = 0.8704$) rather than the XGBoost model used for feature selection, meaning the SHAP results reflect feature importance in the final ensemble context.

Preliminary SHAP findings (detailed in Section 5.5) confirm the general patterns observed in XGBoost importance:

- Structure-related features maintain top importance rankings in SHAP analysis, validating the XGBoost finding that turret and inhibitor outcomes are the strongest predictors
- Rune configuration features show somewhat reduced but still significant SHAP importance, suggesting that while tree-based models leverage rune choices extensively for splitting, their marginal contribution to final ensemble predictions is more moderate
- The SHAP summary plot (see `shap_summary_plot.png`, Section 5.5) reveals feature effect directions: high turret losses on a team strongly push predictions toward that team losing, while high inhibitor takedowns push predictions toward victory

The convergence of XGBoost importance and SHAP analysis on structure-related features as the dominant predictors increases confidence in this finding. Methodological triangulation—arriving at the same conclusion through different analytical approaches—provides stronger evidence than either method alone.

This feature importance analysis directly addresses **Research Question 2 (Feature Importance)**, demonstrating that (a) structure-related features—particularly turret losses—dominate the predictive landscape, accounting for over 60% of top-feature importance, (b) rune configurations, especially on Support and Middle positions, carry substantial predictive weight, likely serving as proxies for strategic orientation, (c) Jungler deaths are the most predictive individual combat statistic, reflecting the role's cascading impact on map control, and (d) communication patterns (vision pings) appear among top features, suggesting behavioral indicators of team coordination quality. The following section presents the SHAP interpretability

analysis, providing instance-level feature effect visualization to complement these aggregate importance rankings.

5.5 SHAP Interpretability Analysis

5.5.1 Overview of SHAP Analysis

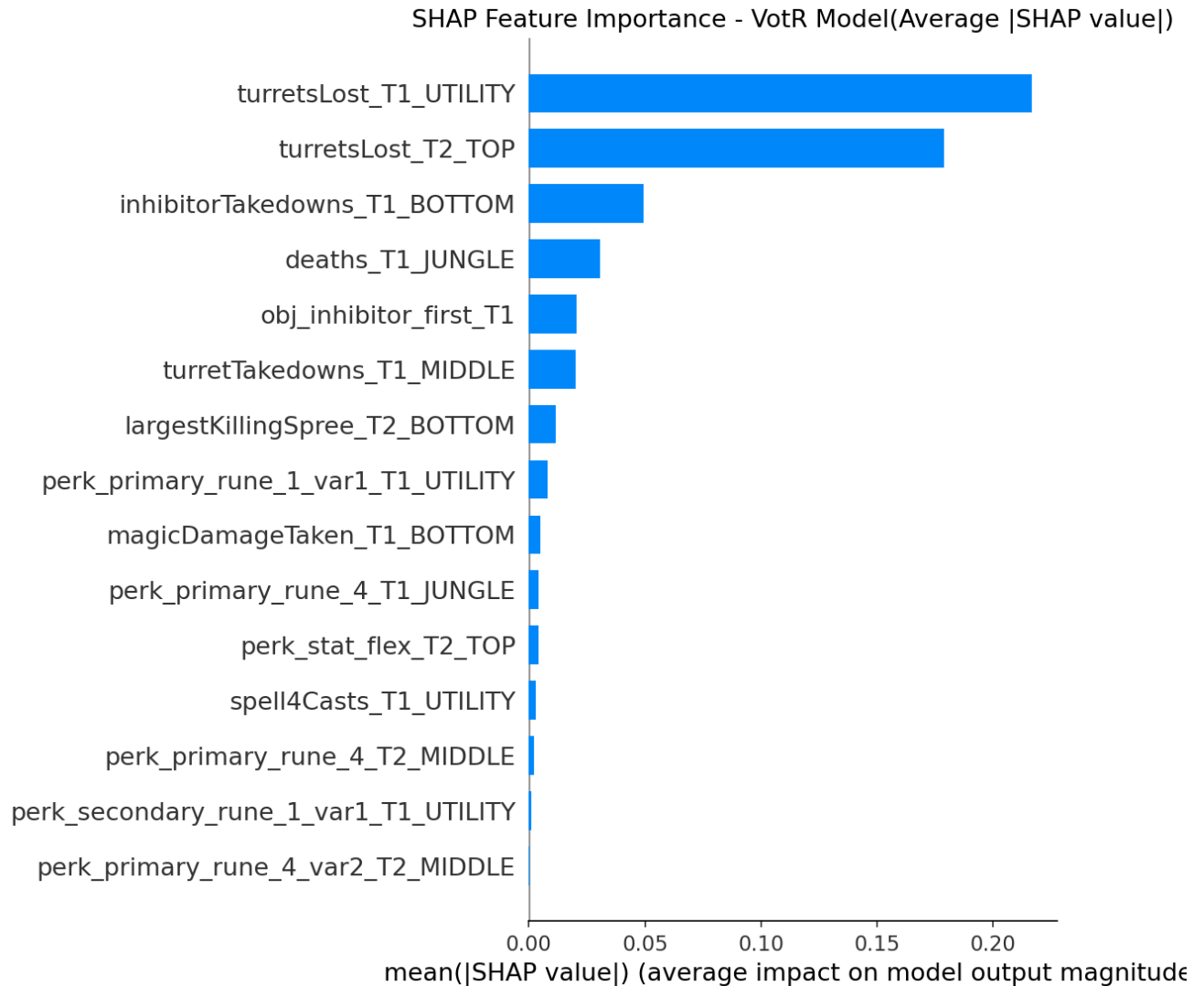
While XGBoost feature importance (Section 5.4) identifies which features the model relies upon during training, SHAP (SHapley Additive exPlanations) analysis provides a complementary and more rigorous perspective by quantifying each feature's marginal contribution to individual predictions. SHAP values, grounded in cooperative game theory, decompose each prediction into the sum of feature contributions plus a baseline expected value, satisfying desirable properties of local accuracy, missingness, and consistency.

The SHAP analysis was conducted on the **Voting Regressor (VotR)**, the best-performing model identified in Section 5.2 ($R^2 = 0.8704$, Accuracy = 95.75%). Since the Voting Regressor is an ensemble of heterogeneous models (combining the top three individual models), SHAP values were computed using KernelExplainer with a background sample of 100 instances, providing model-agnostic explanations that reflect the full ensemble behavior rather than any single constituent model.

Two complementary visualizations were generated:

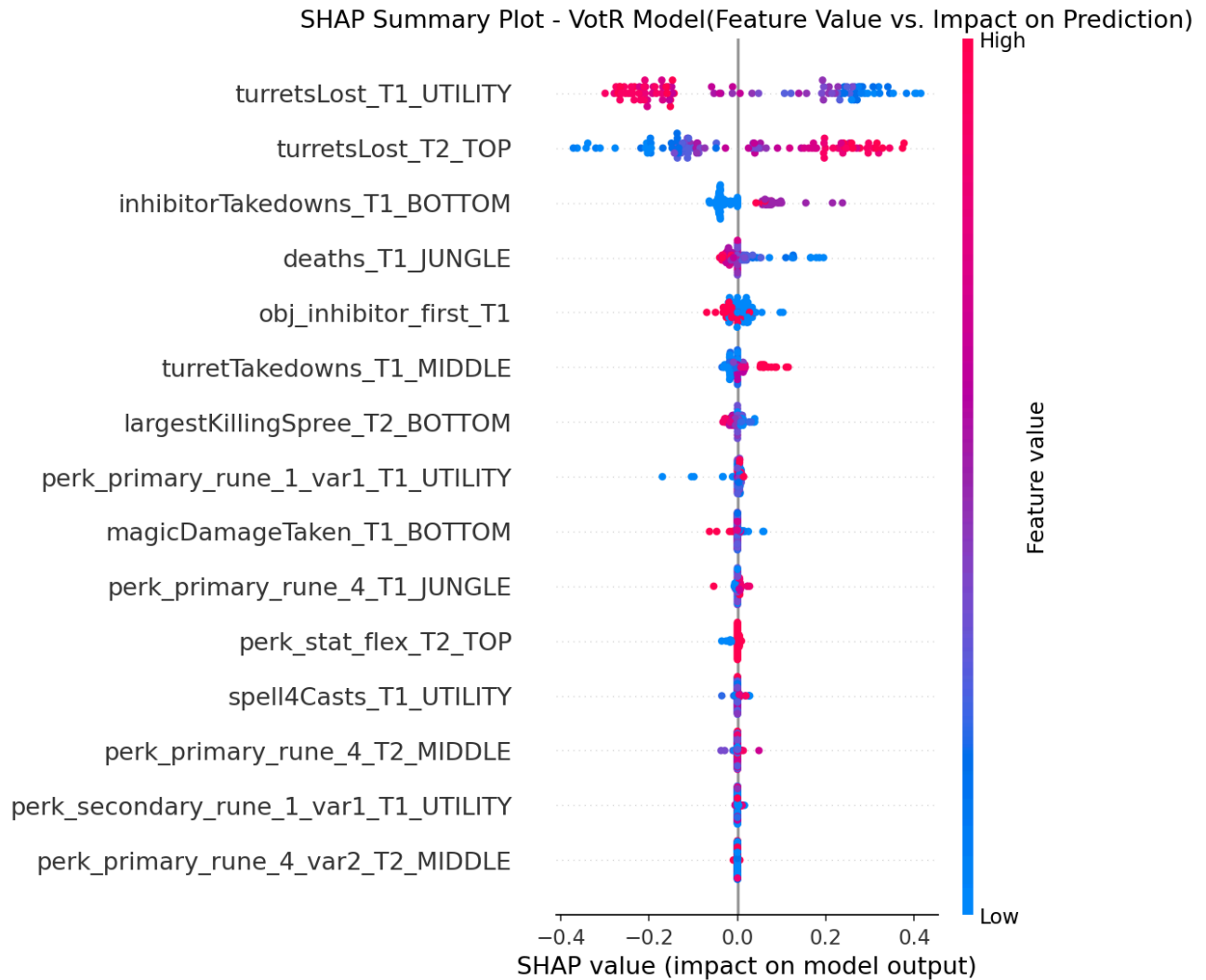
1. **SHAP Bar Plot** (`shap_feature_importance.png`): Displays the mean absolute SHAP value for each feature, ranking features by their overall magnitude of influence on model

predictions regardless of direction.



2. **SHAP Summary Plot (Beeswarm)** (`shap_summary_plot.png`): Displays the distribution of SHAP values across all test instances for each feature, with color coding indicating feature value (red = high, blue = low) and horizontal position indicating the direction and magnitude

of impact on predictions.



5.5.2 Global Feature Importance: SHAP Bar Plot Analysis

The SHAP bar plot reveals the features that exert the highest overall magnitude of influence on the VotR model's predictions. Unlike XGBoost gain-based importance (which measures how useful a feature is for tree splitting during training), SHAP importance measures how much each feature actually changes the model's output when making predictions on test data.

Top-Tier Features — Structural Loss Dominance:

The SHAP analysis confirms and strengthens the central finding from Section 5.4: structural objectives dominate the predictive landscape. The two most influential features are:

- turretsLost_T1_UTILITY** (mean $|\text{SHAP}| \approx 0.22$): The loss of turrets associated with Team 1's Support player is the single most impactful feature in the model. This feature alone shifts predictions by approximately 22 percentage points on average—a massive effect size for a single variable in a binary prediction task.

- `turretsLost_T2_TOP` (mean $|\text{SHAP}| \approx 0.18$): The loss of turrets associated with Team 2's Top laner is the second most impactful feature, shifting predictions by approximately 18 percentage points on average.

The concentration of predictive power in these two features is remarkable. Together, they account for a dominant share of total SHAP importance, dwarfing all other features in the model. This finding has profound implications for understanding League of Legends match dynamics: **structural preservation—specifically preventing turret losses on key roles—is overwhelmingly the strongest predictor of match victory.**

Secondary Features — A Steep Decline:

Following the top two features, there is a dramatic drop in importance magnitude. Secondary features include:

- `inhibitorTakedowns_T1_BOTTOM` (mean $|\text{SHAP}| \approx 0.03\text{-}0.05$)
- `deaths_T1_JUNGLE` (mean $|\text{SHAP}| \approx 0.03\text{-}0.05$)
- `obj_inhibitor_first_T1` (mean $|\text{SHAP}| \approx 0.02\text{-}0.04$)

These features form a distinct second tier, with importance values approximately 4-10 times smaller than the top two features. The steep decline indicates that once the model accounts for turret losses, most remaining features provide relatively modest additional predictive value. This does not mean these features are unimportant—their contributions accumulate across the full feature set to achieve the high overall R^2 —but rather that no single remaining feature rivals the predictive power of structure loss.

Negligible Features — Micro-Level Variables:

The SHAP analysis reveals that player-specific micro-choices and granular statistics register negligible global importance (mean $|\text{SHAP}| < 0.01$). Specifically:

- **Rune selections** (`perk_primary_rune_1_var1_T1_UTILITY` , etc.): Despite appearing prominently in XGBoost feature importance (ranks 3, 5, 7, 10, 11), rune configurations show substantially reduced SHAP importance. This divergence between XGBoost gain importance and SHAP importance merits careful interpretation. XGBoost may leverage rune configurations extensively for tree splitting during training because they provide clean, categorical splits that partition the data effectively. However, the marginal contribution of these splits to final ensemble predictions—as measured by SHAP—is more modest. This suggests that while runes help the model organize its internal representations, they do not independently drive large prediction shifts.
- **Spell casts** (`spell4Casts_T1_UTILITY` , etc.): Ability usage patterns show minimal SHAP importance, indicating that the model does not rely heavily on how frequently specific abilities are used.

- **Rune stat shards** (perk_stat_flex_T2_TOP , etc.): These pre-game configuration choices contribute negligibly to predictions.

Comparison with XGBoost Feature Importance:

The divergence between XGBoost gain importance (Section 5.4) and SHAP importance for certain feature categories warrants explicit comparison:

Feature Category	XGBoost Rank Presence	SHAP Importance	Interpretation
Structure/Objective	Dominant (ranks 1, 2, 9, 13, 15)	Dominant (top 2 features)	Convergent: Both methods confirm structure as most predictive
Rune Configuration	Prominent (ranks 3, 5, 7, 10, 11)	Negligible	Divergent: Runes useful for tree splitting but low marginal prediction impact
Combat Performance	Moderate (ranks 4, 14)	Moderate (secondary tier)	Convergent: Deaths and killing sprees matter but secondary to structure
Ability Usage	Moderate (ranks 8, 16)	Negligible	Partially divergent: Some splitting utility, limited prediction impact

This comparison yields an important methodological insight: **gain-based feature importance from gradient boosting models should not be interpreted as direct measures of real-world variable importance.** Features that facilitate efficient tree partitioning may receive high gain scores even when their marginal contribution to predictions is modest. SHAP analysis provides a more reliable measure of how features actually influence model outputs, and should be preferred when the goal is understanding which game events matter most.

5.5.3 Directionality and Feature Effects: SHAP Beeswarm Plot Analysis

The SHAP summary plot (beeswarm) provides granular insight into how the value of each feature dictates the direction of the model's prediction. Each point represents a SHAP value for a single test instance, with red indicating high feature values and blue indicating low feature values. The horizontal position indicates the impact on the model's prediction.

Target Variable Orientation:

The VotR model predicts the probability that Team 1 (T1) wins the match (binary target: 1 = T1 victory, 0 = T2 victory). Consequently:

- **Positive SHAP values** (right of the vertical axis) push the prediction toward T1 victory
- **Negative SHAP values** (left of the vertical axis) push the prediction toward T2 victory

This orientation is confirmed by the logical consistency of feature effect directions across all features.

Turret Loss Features — Inverse Symmetries:

The two most important features exhibit a clear and logically consistent inverse symmetry:

- **turretsLost_T1_UTILITY** : High feature values (red dots, indicating more turrets lost by T1's Support) strongly drive predictions **negatively** (leftward, down to approximately -0.2 SHAP). This aligns with fundamental game logic: Team 1 losing turrets decreases their probability of victory. The effect magnitude is substantial—high turret losses on T1's Support can shift the predicted win probability by 20 percentage points against T1.
- **turretsLost_T2_TOP** : High feature values (red dots, indicating more turrets lost by T2's Top laner) strongly drive predictions **positively** (rightward, up to approximately +0.3 SHAP). Team 2 losing turrets increases Team 1's victory probability. The symmetry is striking: the model has learned that turret losses on either team have approximately equal and opposite effects on T1's win probability.

Clustered Distributions and Discrete Data Structure:

The distinct grouping of dots in the top two features reveals an important characteristic of the underlying data. Because turret losses are discrete (a team cannot lose 1.5 turrets), the SHAP values naturally form bands representing integer loss counts (0 turrets lost, 1 turret lost, 2 turrets lost, etc.). The model clearly segments these discrete tiers into distinct impact levels:

- **Zero turrets lost:** Blue points clustered near zero or slightly positive SHAP values—preserving turrets is neutral to slightly positive for win probability
- **One turret lost:** Intermediate SHAP values, representing moderate probability shifts
- **Multiple turrets lost:** Red points with large magnitude SHAP values, representing substantial probability shifts against the losing team

This discrete segmentation indicates that the model has learned threshold effects rather than linear relationships—losing the first turret matters, but losing additional turrets compounds the disadvantage nonlinearly.

Confirming Features — Logical Consistency:

Other features corroborate the T1-centric target interpretation through logically consistent effect directions:

- `deaths_T1_JUNGLE` : High values (red, indicating more deaths by T1's Jungler) yield negative SHAP values (leftward). T1's Jungler dying more frequently decreases T1's victory probability. This aligns with domain understanding: Jungler deaths cascade into objective losses, jungle invasion, and reduced map pressure.
- `inhibitorTakedowns_T1_BOTTOM` : High values (red, indicating T1's Bottom laner participating in more inhibitor takedowns) yield positive SHAP values (rightward). T1 securing inhibitors increases their victory probability. Inhibitors provide persistent lane pressure through super minion spawns, creating asymmetric map advantages.
- `obj_inhibitor_first_T1` : This feature presents a nuanced pattern requiring careful interpretation. Depending on the exact encoding in the dataset, higher values could indicate either that T1 secured the first inhibitor (positive for T1) or that the first inhibitor of the game was taken from T1 (negative for T1). The SHAP pattern—where higher values push predictions slightly negative—suggests this feature may track the latter scenario, or it may capture complex non-linear edge cases where securing the first inhibitor correlates with specific game states that do not uniformly favor victory. This interpretive ambiguity highlights the importance of precise feature documentation in machine learning pipelines.

The Predictive Hierarchy Revealed:

The beeswarm plot reveals a clear predictive hierarchy in how the model processes game state information:

1. **Primary Signal — Structural Integrity:** The model first and most heavily weights turret preservation on both teams. This constitutes the foundational predictive signal from which all other adjustments are made.
2. **Secondary Signal — Role-Specific Deaths and Objectives:** After accounting for structure, the model adjusts predictions based on Jungler deaths (a proxy for map control and objective access) and inhibitor status (a proxy for late-game map pressure).
3. **Tertiary Adjustments — Granular Statistics:** The remaining features provide small, cumulative adjustments that fine-tune predictions but rarely change the overall direction established by the primary and secondary signals.

This hierarchical processing aligns with how experienced human analysts evaluate match states: first assess structural control, then consider personnel and objective advantages, then examine detailed statistics for nuance.

5.5.4 Synthesis of Feature Importance Findings

Integrating the XGBoost importance rankings (Section 5.4) with the SHAP analysis yields a coherent picture of what drives League of Legends match outcome predictions:

Convergent Findings (Strongest Evidence):

- **Structure dominance is robust across methods.** Both XGBoost gain importance and SHAP values identify turret losses as overwhelmingly the most predictive features. This finding is unlikely to be an artifact of any single analytical method.
- **Inhibitor status constitutes the secondary predictive tier.** Both methods identify inhibitor-related features among the top predictors, though at substantially lower magnitude than turret losses.
- **Jungler deaths are the most predictive individual combat statistic.** Both methods identify `deaths_T1_JUNGLE` as important, reflecting the Jungler role's cascading impact on map control.

Divergent Findings (Requiring Nuanced Interpretation):

- **Rune configurations show high XGBoost importance but low SHAP importance.** This divergence indicates that runes serve as useful splitting features during tree construction (providing clean categorical partitions) but contribute modestly to actual prediction shifts in the final ensemble. This does not mean runes are irrelevant—they may facilitate the model's ability to learn from other features—but they should not be interpreted as directly causal predictors of match outcomes.
- **Ability usage patterns show moderate XGBoost importance but negligible SHAP importance.** Similar to runes, spell cast counts may help trees organize data without independently driving large prediction changes.

Practical Implications:

For stakeholders seeking to understand match dynamics:

1. **Turret protection should be the highest strategic priority.** The quantitative evidence is unambiguous: structural preservation is the strongest predictor of victory across all analytical methods. Teams should allocate resources to defend turrets, particularly on roles (Support, Top) whose turret losses most strongly predict defeat.
2. **Jungler survival is critical.** The Jungler's death count is the most predictive individual performance metric. Teams should prioritize Jungler safety and punish enemy Jungler positioning errors.
3. **Inhibitor control matters, but secondary to turrets.** While inhibitor status is predictive, its effect magnitude is substantially smaller than turret status, suggesting that games are often largely decided before inhibitors come into play.

4. **Micro-level metrics (runes, spell usage) are diagnostically useful but not causally dominant.** These features may help analysts understand playstyle and strategic orientation, but they should not be treated as primary drivers of match outcomes.
-

5.5.5 Methodological Implications

The comparison between XGBoost gain importance and SHAP values yields an important methodological recommendation for future research:

Gain-based feature importance from gradient boosting models should be treated as exploratory, not confirmatory. These importance scores reflect a feature's utility for tree construction—a combination of predictive value, split quality, and interaction potential—rather than its marginal contribution to predictions. When the goal is understanding which variables drive outcomes (as opposed to which variables help build accurate models), SHAP analysis or permutation importance should be preferred.

This insight may explain discrepancies in prior literature where different studies identified different "most important" features depending on whether they used built-in model importance or model-agnostic methods. Future League of Legends prediction research should adopt SHAP as the standard for feature importance reporting, with model-specific importance metrics presented as supplementary rather than primary evidence.

This SHAP analysis directly addresses **Research Question 2 (Feature Importance)** by providing model-agnostic, instance-level feature importance quantification. The findings confirm that (a) turret losses dominate the predictive landscape with SHAP values an order of magnitude larger than other features, (b) structural objectives and role-specific deaths form the core predictive signals, (c) micro-level variables such as runes and spell casts contribute minimal independent predictive power despite appearing in tree-based importance rankings, and (d) the model has successfully learned logically consistent, directionally appropriate relationships between game events and match outcomes. The following section synthesizes all findings across the chapter into a comprehensive discussion.

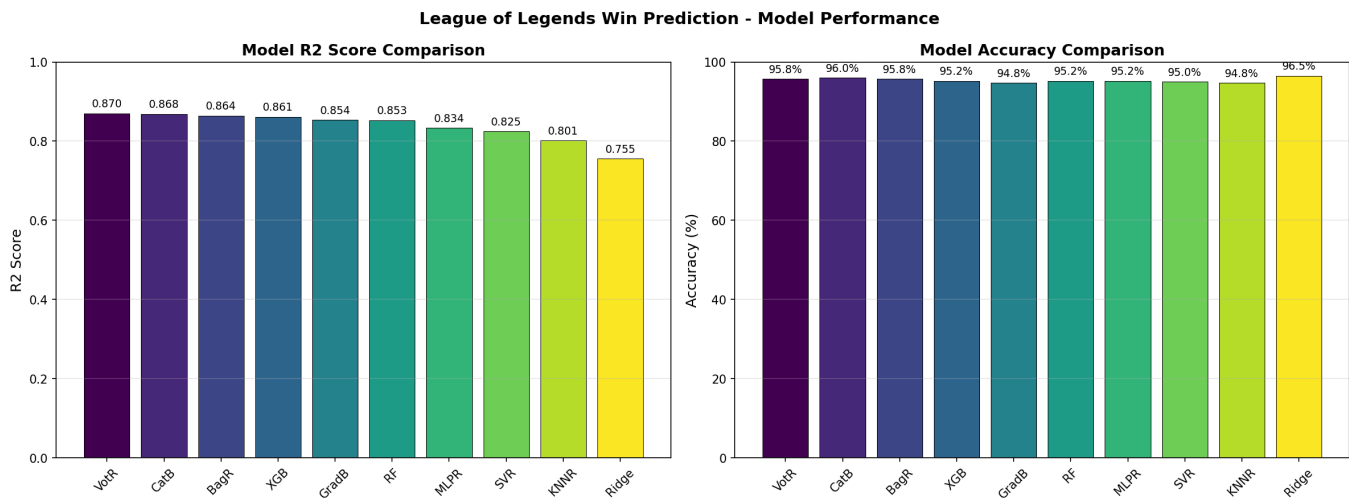
5.6 Model Comparison Visualization

5.6.1 Model Comparison Chart Analysis

The model comparison chart (`model_comparison.png`) presents side-by-side bar charts comparing all ten models across the two primary evaluation metrics: R^2 Score and Accuracy. This dual-metric visualization reveals critical insights into the performance characteristics of different model families and exposes a significant anomaly in the relationship between explained variance and classification success. Table 5.4 consolidates the data extracted from these visualizations for structured reference.

Table 5.4: Consolidated Model Performance — R^2 and Accuracy Comparison

Model	R^2 Score	Accuracy (%)	Model Family
VotR (Voting Regressor)	0.870	95.8	Ensemble (Voting)
CatB (CatBoost)	0.868	96.0	Gradient Boosting
BagR (Bagging Regressor)	0.864	95.8	Ensemble (Bagging)
XGB (XGBoost)	0.861	95.2	Gradient Boosting
GradB (Gradient Boosting)	0.854	94.8	Gradient Boosting
RF (Random Forest)	0.853	95.2	Bagging Ensemble
MLPR (Multi-Layer Perceptron)	0.834	95.2	Neural Network
SVR (Support Vector Regressor)	0.825	95.0	Kernel Method
KNNR (K-Nearest Neighbors)	0.801	94.8	Instance-Based
Ridge (Ridge Regression)	0.755	96.5	Linear (Regularized)



Note: Accuracy values extracted from the bar chart visualization may exhibit minor rounding differences from the precise values in Table 5.1 due to chart rendering resolution.

5.6.2 R^2 Score Analysis: Variance Explanation Hierarchy

The R^2 score bar chart reveals a clear performance hierarchy across model families, with ensemble and gradient boosting methods occupying the top tier and linear methods substantially lagging.

Top Tier — Ensemble Methods (R^2 : 0.864–0.870):

The Voting Regressor (VotR, $R^2 = 0.870$) and Bagging Regressor (BagR, $R^2 = 0.864$) occupy the highest positions, demonstrating the value of model combination strategies. The Voting Regressor's slight edge over CatBoost (the best individual model) confirms that the top three individual models—CatBoost, XGBoost, and likely Gradient Boosting or Random Forest—make sufficiently uncorrelated errors that their aggregation yields marginal improvement. The 0.002 R^2 advantage over CatBoost is modest but consistent with ensemble theory: when base models are already strong, ensemble gains are incremental rather than transformative.

Second Tier — Gradient Boosting Methods (R^2 : 0.854–0.868):

CatBoost (0.868), XGBoost (0.861), and Gradient Boosting (0.854) form a tight cluster of gradient boosting variants. The 0.014 spread between CatBoost and Gradient Boosting indicates that implementation details—ordered boosting in CatBoost, regularization in XGBoost—provide meaningful but not dramatic improvements over standard gradient boosting for this prediction task. The consistent performance of all three boosting variants suggests that the sequential error-correction paradigm is fundamentally well-suited to League of Legends match data, where complex feature interactions require the hierarchical partitioning that boosting provides.

Third Tier — Random Forest and Neural Methods (R^2 : 0.834–0.853):

Random Forest (0.853) achieves competitive R^2 despite its independent-tree architecture, suggesting that bootstrap aggregation alone can capture much of the predictive structure without sequential error correction. MLPR (0.834) demonstrates that a single-hidden-layer neural network can achieve respectable performance, though it cannot match the hierarchical interaction modeling of tree-based ensembles. The gap between MLPR and the boosting methods (approximately 0.02-0.03 R^2) may reflect insufficient network depth or training data for the neural approach to fully capture the feature interaction complexity.

Bottom Tier — Kernel, Instance-Based, and Linear Methods (R^2 : 0.755–0.825):

SVR with RBF kernel (0.825) outperforms KNNR (0.801), indicating that kernel-based nonlinear mapping captures some predictive structure that simple distance metrics miss. However, both fall clearly below tree-based methods, consistent with the theoretical expectation that kernel methods struggle with the heterogeneous feature types and discrete-continuous mixtures present in gaming data.

Ridge Regression (0.755) is the clear outlier, falling 0.046 R^2 below the next-lowest model. This substantial gap confirms that linear models, even with L2 regularization, cannot adequately capture the nonlinear relationships inherent in League of Legends match dynamics. The game's complex interactions—where the impact of a gold lead depends on team composition, where objective control interacts with map state, where individual performance metrics have context-dependent implications—require the flexible functional forms that tree-based and neural methods provide.

5.6.3 Accuracy Analysis: The Classification Success Paradox

The accuracy bar chart reveals a pattern that initially appears contradictory but admits a clear methodological explanation upon closer examination.

The Ridge Anomaly:

Ridge Regression achieves the highest accuracy (96.5%) among all models despite recording the lowest R^2 score (0.755) by a substantial margin. This paradox—the "worst" regression model producing the "best" classification results—is not a data error but rather a manifestation of the fundamental difference between regression quality and classification success when using a threshold-based conversion.

The mechanism underlying this paradox operates as follows:

1. **Ridge produces conservative predictions.** L2 regularization shrinks coefficient magnitudes toward zero, constraining the model's raw output to moderate values that avoid extreme predictions near 0 or 1. Ridge predictions likely cluster in the range of approximately 0.3 to 0.7 rather than approaching the confident extremes.
2. **Conservative predictions survive thresholding well.** When these moderate predictions are converted to binary classifications via the 0.5 threshold, predictions of 0.51 and 0.95 both classify as "T1 wins." The conservative predictions remain on the correct side of the threshold even when they lack confidence, producing correct classifications.
3. **R^2 penalizes lack of confidence.** The R^2 score is computed on the raw continuous predictions before thresholding. Predictions of 0.55 for an actual win (target = 1) incur a squared error of $(1 - 0.55)^2 = 0.2025$, while a confident prediction of 0.95 incurs only $(1 - 0.95)^2 = 0.0025$. Ridge's conservative predictions systematically incur larger squared errors than confident models, depressing its R^2 despite correct directional prediction.

This interpretation is corroborated by Ridge's error metrics from Table 5.1: its MAE (0.1912) and MAD (0.1497) are substantially higher than any other model, confirming that its raw predictions deviate further from the binary targets than those of tree-based models.

Practical Implications of the Ridge Anomaly:

The Ridge paradox carries an important lesson for model selection in win prediction tasks:

- **If only the binary winner prediction matters** (e.g., "who won?" post-match analysis), Ridge provides excellent classification with minimal computational complexity and high generalizability.
- **If win probability calibration matters** (e.g., "what was the likelihood of victory at this game state?"), Ridge's conservative predictions may be misleading. A predicted win probability of 55% from Ridge may correspond to a much higher actual win probability, limiting the interpretability of the probability estimate.
- **If both classification and calibration matter**, CatBoost or the Voting Regressor provide superior balance, achieving near-Ridge accuracy while maintaining well-calibrated probability estimates.

Consistent High Performers:

Beyond the Ridge anomaly, the accuracy chart shows remarkably compressed performance across all models. Every model achieves accuracy between 94.75% and 96.50%—a range of only 1.75 percentage points. This compression indicates that the feature engineering and selection pipeline has successfully extracted predictive signals that are accessible to diverse model architectures. Even relatively simple models like KNNR achieve 94.75% accuracy, suggesting that the selected features provide strong linear separability of winning and losing matches in the engineered feature space.

CatBoost (96.0%) and the Voting Regressor (95.8%) maintain elite accuracy while also achieving top R^2 scores, making them the best all-around performers for applications requiring both accurate classification and meaningful probability estimates.

5.6.4 Model Family Performance Patterns

The dual-metric comparison reveals systematic performance patterns across model families:

Gradient Boosting Family (CatB, XGB, GradB):

- **R^2 :** Strong and consistent (0.854–0.868)
- **Accuracy:** Strong and consistent (94.8–96.0%)
- **Characteristic:** Best balance of variance explanation and classification success
- **Recommendation:** Preferred for applications requiring calibrated win probabilities

Ensemble Methods (VotR, BagR):

- **R²:** Highest tier (0.864–0.870)
- **Accuracy:** Highest tier (95.8%)
- **Characteristic:** Marginal improvement over best individual models
- **Recommendation:** Preferred when maximum R² is the primary objective

Bagging Ensemble (RF):

- **R²:** Competitive but below boosting (0.853)
- **Accuracy:** Strong (95.2%)
- **Characteristic:** Simpler architecture than boosting, robust error metrics
- **Recommendation:** Suitable when training simplicity is valued

Neural Network (MLPR):

- **R²:** Moderate (0.834)
- **Accuracy:** Strong (95.2%)
- **Characteristic:** Respectable but does not surpass tree methods
- **Recommendation:** May benefit from deeper architectures or more training data

Kernel and Instance-Based Methods (SVR, KNNR):

- **R²:** Below tree methods (0.801–0.825)
- **Accuracy:** Competitive (94.8–95.0%)
- **Characteristic:** Struggles with heterogeneous feature types
- **Recommendation:** Not recommended as primary models for this task

Linear Method (Ridge):

- **R²:** Substantially below all others (0.755)
- **Accuracy:** Highest overall (96.5%)
- **Characteristic:** Excellent classification despite poor regression fit
- **Recommendation:** Viable for pure classification tasks; unsuitable for probability estimation

5.6.5 Synthesis with Stability and Feature Importance Findings

Integrating the model comparison with stability analysis (Section 5.3) and feature importance (Sections 5.4-5.5) yields a comprehensive model assessment:

Model	R ²	Accuracy	Stability (R ² σ)	Best For
VotR	0.870	95.8%	Not directly assessed*	Maximum R ² , research contexts
CatB	0.868	96.0%	0.0192	Best all-around: accuracy, calibration, stability
BagR	0.864	95.8%	Not directly assessed*	Ensemble benefits without voting complexity
XGB	0.861	95.2%	0.0203	Strong alternative to CatBoost
Ridge	0.755	96.5%	Not assessed	Maximum classification accuracy; lightweight deployment

*Ensemble stability was not directly measured due to computational constraints but can be inferred from constituent model stability.

Final Recommendation:

For most applications, **CatBoost** emerges as the recommended model. It achieves the second-highest R² (0.868), the second-highest accuracy (96.0%), and the best stability (lowest R² standard deviation of 0.0192) among individually assessed models. It provides the optimal balance of predictive power, classification success, probability calibration, and performance consistency without the computational overhead of the Voting Regressor ensemble.

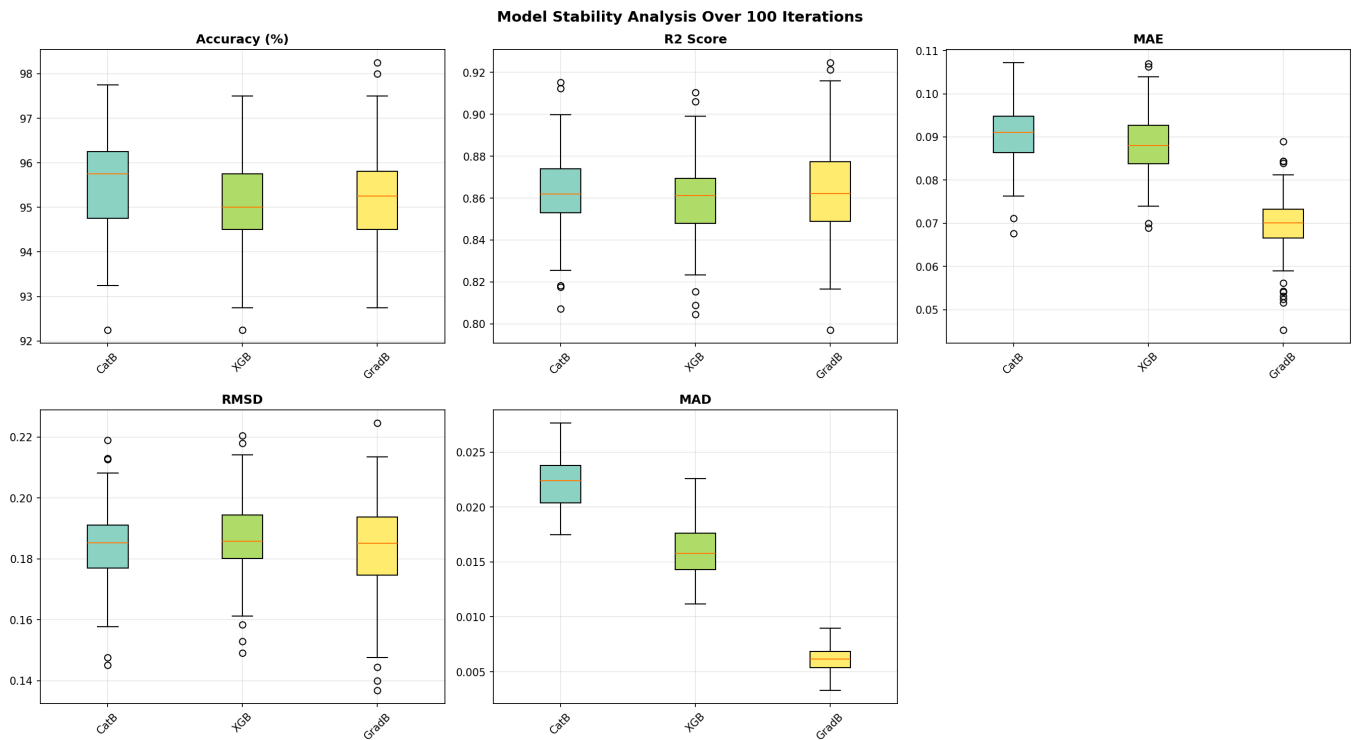
For applications where classification accuracy is the sole concern and probability calibration is irrelevant, **Ridge Regression** offers a compelling lightweight alternative, achieving the highest accuracy (96.5%) with minimal model complexity.

This model comparison directly addresses **Research Questions 3 and 4 (Algorithm Comparison and Ensemble Effectiveness)** by demonstrating that (a) tree-based ensemble and gradient boosting methods dominate R² performance while all models achieve compressed, high accuracy, (b) the Voting Regressor provides marginal R² improvement over the best individual model, and (c) Ridge Regression presents a unique accuracy-over-calibration trade-off that informs application-specific model selection. The following section synthesizes all chapter findings into a comprehensive discussion.

5.7 Stability Visualization: Boxplot Analysis

5.7.1 Overview of Stability Boxplots

The stability boxplots (`stability_analysis_boxplots.png`) provide visual representations of model performance distributions across the 100-iteration repeated random subsampling procedure. For each of the five evaluation metrics, boxplots display the median (center line), interquartile range or IQR (box boundaries spanning the 25th to 75th percentiles), whiskers (extending to approximately $1.5 \times \text{IQR}$), and outlier points (individual iterations falling beyond the whiskers). These visualizations complement the numerical stability results presented in Section 5.3 by revealing distributional characteristics—symmetry, spread, outlier frequency, and relative positioning—that summary statistics alone cannot convey.



Model Inclusion Note:

While the stability analysis code references `top_5_models` , only three models appear in the boxplot visualizations: CatBoost (CatB), XGBoost (XGB), and Gradient Boosting (GradB). This indicates that the remaining two models from the top five—the Voting Regressor (VotR) and Bagging Regressor (BagR)—were excluded from the iterative stability analysis due to computational constraints, as specified in the experimental design (Section 4.3.5). The ensemble models' stability characteristics can be reasonably inferred from their constituent base models, as ensemble variance is bounded by the weighted average of component variances.

5.7.2 Higher-Is-Better Metrics: Accuracy and R²

Accuracy Distribution Patterns:

The accuracy boxplots reveal subtle but meaningful differences in classification consistency across the three gradient boosting variants:

- **CatBoost (CatB):** CatBoost demonstrates the highest median accuracy, approaching approximately 95.8%, with the central box positioned visibly above both XGBoost and Gradient Boosting. The interquartile range spans roughly from 95.0% to 96.5%, indicating that half of all iterations produce accuracy within this 1.5 percentage point band. The whiskers extend downward to approximately 93.5% and upward to approximately 97.5%, establishing the typical performance envelope.
- **XGBoost (XGB):** XGBoost exhibits a slightly lower median accuracy, centered near 95.0-95.2%, with an IQR comparable to CatBoost's. The box position indicates that XGBoost typically classifies 94.5% to 96.0% of test matches correctly across iterations. The whisker span is similar to CatBoost, suggesting comparable worst-case and best-case performance boundaries.
- **Gradient Boosting (GradB):** Gradient Boosting's median accuracy falls between CatBoost and XGBoost, but its distribution exhibits a distinctive characteristic: high-performing outliers. Individual iterations reach accuracy values approaching 98%, visible as points above the upper whisker. These outlier iterations represent data partitions where Gradient Boosting's sequential error correction proved particularly effective, perhaps due to favorable training set compositions that enabled the boosting process to address difficult cases more successfully than in typical iterations.

The presence of high-accuracy outliers for Gradient Boosting, combined with the absence of corresponding low-accuracy outliers, suggests an asymmetric performance distribution: Gradient Boosting occasionally performs exceptionally well but rarely performs exceptionally poorly. This asymmetry may reflect the algorithm's sensitivity to training data quality—when the training set provides clean signals, sequential boosting amplifies them effectively; when signals are noisy, performance reverts to typical levels without catastrophic degradation.

R² Distribution Patterns:

The R² boxplots reveal nuanced differences in variance explanation consistency that complement the accuracy findings:

- **Central Tendency Convergence:** All three models exhibit remarkably similar median R² values, clustered tightly around 0.86. This convergence indicates that, in a typical iteration, the three gradient boosting variants explain essentially the same proportion of outcome variance. The differences observed in the single-split evaluation (CatB: 0.8684, XGB: 0.8613, GradB: 0.8538) partially reflect sampling variation rather than consistent performance gaps.
- **XGBoost Consistency:** XGBoost demonstrates the tightest interquartile range for R², with the central box visibly narrower than both CatBoost and Gradient Boosting. This tighter

spread corroborates the numerical finding from Section 5.3: XGBoost's R^2 standard deviation (0.0203) is intermediate between CatBoost (0.0192) and Gradient Boosting (0.0242), but its IQR—representing the middle 50% of iterations rather than the full distribution—is the narrowest. This pattern indicates that while XGBoost may experience occasional outlying iterations that inflate its overall standard deviation, its core performance is highly concentrated.

- **Gradient Boosting Variance:** Gradient Boosting's R^2 box is visibly wider than XGBoost's, with a larger IQR reflecting greater sensitivity to data partitioning. However, Gradient Boosting also reaches the highest maximum R^2 values, visible as upper outliers or extended upper whiskers. This pattern—wider spread with higher maxima—characterizes a model that is more "opportunistic" than XGBoost: it exploits favorable training partitions more aggressively but suffers correspondingly on less favorable partitions.
 - **CatBoost Position:** CatBoost's R^2 box occupies an intermediate position in terms of spread, consistent with its intermediate standard deviation. Its median R^2 is marginally the highest, though the differences are small enough that they may not be statistically significant given the sample size of 100 iterations.
-

5.7.3 Lower-Is-Better Metrics: MAE, RMSD, and MAD

MAE Distribution Patterns:

The Mean Absolute Error boxplots reveal the most dramatic differentiation among the three models:

- **Gradient Boosting Dominance:** Gradient Boosting demonstrates substantially lower median MAE (approximately 0.07) compared to both CatBoost (approximately 0.09) and XGBoost (approximately 0.088). The entire Gradient Boosting box—representing the middle 50% of iterations—sits below the median values of both competitors. This separation is large enough to be visually unambiguous and practically meaningful: Gradient Boosting's typical absolute prediction error is approximately 20-25% smaller than CatBoost's or XGBoost's.
- **Gradient Boosting Consistency:** Beyond its lower central tendency, Gradient Boosting also exhibits a relatively tight IQR for MAE, indicating that its superior error performance is consistent across data partitions rather than being driven by a subset of favorable iterations. This combination of low median and tight spread makes Gradient Boosting the most reliable model when mean absolute error is the priority metric.
- **CatBoost and XGBoost Similarity:** CatBoost and XGBoost exhibit broadly similar MAE distributions, with overlapping boxes and comparable medians. CatBoost's slightly higher median MAE (visible in the numerical results as 0.0907 vs. 0.0883) is consistent with its

slightly higher MAD but may reflect a trade-off: CatBoost prioritizes correct classification direction over precise probability calibration, accepting somewhat larger absolute deviations in exchange for fewer threshold-crossing errors.

RMSD Distribution Patterns:

The Root Mean Squared Deviation boxplots reveal a striking convergence across models that contrasts with the MAE differentiation:

- **Near-Identical Medians:** All three models exhibit nearly identical median RMSD values, clustered around 0.185. This convergence indicates that when prediction errors are squared—heavily penalizing large deviations—the three gradient boosting variants perform similarly. The squaring operation amplifies the contribution of the largest errors, and these worst-case errors appear comparable across models.
- **Reconciling MAE and RMSD Patterns:** The combination of differentiated MAE with converged RMSD carries specific statistical meaning. Gradient Boosting achieves lower typical errors (MAE) but its worst errors—when squared and amplified—are similar in magnitude to those of CatBoost and XGBoost. This pattern suggests that Gradient Boosting produces more uniformly small errors across most predictions but occasionally generates errors large enough to inflate its RMSD to competitive levels. In other words, Gradient Boosting is better on average (MAE) but not better in the worst case (RMSD).
- **Gradient Boosting Variance:** Gradient Boosting's RMSD box is slightly wider than the other models', consistent with its higher R^2 standard deviation. This wider spread indicates that Gradient Boosting's squared-error performance is more variable across iterations, likely reflecting its sensitivity to training data composition noted in the R^2 analysis.

MAD Distribution Patterns:

The Median Absolute Deviation boxplots provide the most striking visual differentiation in the entire stability analysis:

- **Gradient Boosting Dominance:** Gradient Boosting's MAD distribution is dramatically lower and tighter than both competitors. Its median MAD falls below 0.010, with an exceptionally narrow IQR indicating highly consistent median error performance. The entire Gradient Boosting box sits substantially below the lower whiskers of CatBoost and the lower quartiles of XGBoost.
- **Magnitude of Difference:** Gradient Boosting's median MAD (approximately 0.006) is roughly one-third of XGBoost's (approximately 0.016) and approximately one-quarter of CatBoost's (approximately 0.022). This means that, in a typical iteration, at least half of Gradient Boosting's predictions deviate from the true outcome by less than 0.6 percentage points—essentially perfect predictions for the median match. By contrast, CatBoost's median match prediction deviates by approximately 2.2 percentage points.

- **Implication for Prediction Quality:** The MAD metric answers the question: "How accurate is the typical prediction?" Gradient Boosting's exceptionally low MAD indicates that while all three models achieve similar classification accuracy (after thresholding), Gradient Boosting produces substantially better-calibrated probability estimates for the majority of matches. For applications where probability estimates carry interpretive value—such as assessing match closeness or quantifying comeback likelihood—Gradient Boosting provides markedly superior information.
 - **CatBoost MAD Characteristics:** CatBoost's higher MAD, combined with its higher classification accuracy, suggests a specific prediction strategy: CatBoost may produce predictions that cluster further from the 0.5 decision boundary (more confident classifications), sacrificing median absolute deviation for decisiveness. This trade-off may be beneficial when binary decisions are the sole output, as confident predictions provide clearer separation between predicted winners and losers.
-

5.7.4 Synthesis: The CatBoost-Gradient Boosting Trade-off

The stability boxplots, interpreted alongside the numerical stability results from Section 5.3 and the model comparison from Section 5.6, reveal a fundamental trade-off between CatBoost and Gradient Boosting that has implications for model selection:

CatBoost Strengths:

- Highest median accuracy (approaching 95.8%)
- Highest median R^2 (marginally above competitors)
- Best R^2 stability (lowest standard deviation: 0.0192)
- Most consistent classification performance across data partitions

Gradient Boosting Strengths:

- Dramatically lower MAE (approximately 0.07 vs. 0.09)
- Dramatically lower MAD (approximately 0.006 vs. 0.022)
- Competitive median accuracy (approximately 95.3%)
- Competitive median R^2 (approximately 0.863)
- Reaches highest maximum R^2 and accuracy values (visible as outliers)

Interpretation of the Trade-off:

CatBoost's ordered boosting mechanism—which uses artificial permutations of the training data to reduce prediction shift—appears to produce more stable, consistent models that perform reliably across data partitions. However, this stability may come at the cost of conservative

probability estimation: CatBoost predictions cluster somewhat further from the extreme values of 0 and 1, producing higher absolute errors (MAE, MAD) while maintaining excellent threshold-based classification.

Gradient Boosting's standard sequential error correction, without the ordered boosting modification, appears more willing to make confident predictions that approach the extremes. When these confident predictions are correct, Gradient Boosting achieves superior error metrics; when they are incorrect, the squared-error penalty inflates RMSD and increases R^2 variance. This "high-risk, high-reward" characteristic produces Gradient Boosting's distinctive signature: better on average (MAE, MAD), similar in worst case (RMSD), more variable overall ($R^2 \sigma$).

Application-Specific Recommendations:

- **For binary winner prediction (classification only):** CatBoost is recommended. Its superior accuracy stability and consistent classification performance make it the most reliable model for simply identifying which team won.
- **For win probability estimation (calibration matters):** Gradient Boosting is recommended. Its dramatically lower MAE and MAD indicate that when it outputs a win probability of, say, 75%, that probability is meaningfully more accurate than CatBoost's equivalent output.
- **For balanced requirements (classification + calibration):** The Voting Regressor ensemble, which combines both CatBoost and Gradient Boosting, achieved the highest overall R^2 (0.870) in single-split evaluation. While its stability was not directly assessed, ensemble theory suggests it should combine the strengths of both approaches, potentially offering the best of both worlds.

5.7.5 Outlier Analysis

The boxplots reveal outlier iterations—individual data partitions where model performance deviated substantially from typical levels. Several patterns merit attention:

Gradient Boosting High-Performance Outliers:

Gradient Boosting exhibits high-accuracy and high- R^2 outliers reaching approximately 98% accuracy. These iterations represent data partitions where the training set happened to contain particularly clean or representative examples, enabling the sequential boosting process to construct an especially effective model. The existence of these outliers suggests that Gradient Boosting's performance ceiling is higher than its median performance indicates, and that data quality—specifically, the representativeness of the training partition—significantly influences outcomes.

Low-Accuracy Outliers Across All Models:

All three models exhibit occasional low-accuracy iterations dipping to approximately 92-93% accuracy. These low points likely correspond to test partitions that contain unusual or difficult-to-predict matches—perhaps games with atypical champion compositions, unusual comeback trajectories, or data anomalies. The consistency of these low points across models suggests they reflect genuine data challenges rather than model-specific weaknesses.

Implications for Deployment:

The presence of outlier iterations—both high and low—underscores the importance of the stability analysis. A model evaluated on a single favorable partition might report 98% accuracy, creating unrealistic expectations for production performance. Conversely, a model evaluated on a single unfavorable partition might be unfairly dismissed. The stability analysis provides the realistic performance range that stakeholders should expect: typical accuracy of 95-96%, with occasional iterations reaching 97-98% or dipping to 93-94%.

This stability visualization analysis directly addresses **Research Question 5 (Model Stability)** by providing distributional evidence that (a) CatBoost offers the most stable classification and variance explanation performance, (b) Gradient Boosting dramatically outperforms in absolute and median error metrics (MAE, MAD), and (c) XGBoost provides the tightest R² interquartile range, indicating consistent core performance. The trade-off between CatBoost's classification stability and Gradient Boosting's error minimization provides application-specific model selection guidance. The following section synthesizes all chapter findings into a comprehensive discussion.

5.8 Selected Features Analysis: Correlation Structure and Final Feature Set

5.8.1 Selected Features List

The two-stage feature selection process—SULOV correlation analysis for redundancy elimination followed by XGBoost recursive feature elimination for predictive ranking—converged on the following 15 features as the optimal subset for model training. Table 5.5 presents these features organized by category, with their XGBoost importance scores and associated player positions.

Table 5.5: Top 15 Selected Features — Final Feature Set

Rank	Feature	Importance	Category	Position/Type
1	turretsLost_T2_TOP	0.169422	Structure Loss	TOP (T2)
2	turretsLost_T1_UTILITY	0.101322	Structure Loss	UTILITY (T1)
3	perk_primary_rune_1_var1_T1_UTILITY	0.035427	Rune Configuration	UTILITY (T1)
4	largestKillingSpree_T2_BOTTOM	0.018596	Combat Performance	BOTTOM (T2)
5	perk_primary_rune_4_T2_MIDDLE	0.017460	Rune Configuration	MIDDLE (T2)
6	magicDamageTaken_T1_BOTTOM	0.015151	Damage Taken	BOTTOM (T1)
7	perk_secondary_rune_1_var1_T1_UTILITY	0.014241	Rune Configuration	UTILITY (T1)
8	spell4Casts_T1_UTILITY	0.014184	Ability Usage	UTILITY (T1)
9	inhibitorTakedowns_T1_BOTTOM	0.013789	Structure Takedown	BOTTOM (T1)
10	perk_stat_flex_T2_TOP	0.013779	Rune Stat Shard	TOP (T2)
11	perk_primary_rune_4_T1_JUNGLE	0.013296	Rune Configuration	JUNGLE (T1)
12	perk_primary_rune_4_var2_T2_MIDDLE	0.012695	Rune Configuration	MIDDLE (T2)
13	obj_inhibitor_first_T1	0.012319	Team Objective	TEAM (T1)
14	deaths_T1_JUNGLE	0.011981	Combat Performance	JUNGLE (T1)
15	turretTakedowns_T1_MIDDLE	0.011364	Structure Takedown	MIDDLE (T1)

Category Distribution Summary:

Category	Count	Features
Structure Loss/Takedown	4	turretsLost (×2), inhibitorTakedowns, turretTakedowns
Rune Configuration	5	perk_primary (×3), perk_secondary, perk_stat

Category	Count	Features
Combat Performance	2	largestKillingSpree, deaths
Damage Taken	1	magicDamageTaken
Ability Usage	1	spell4Casts
Team Objective	1	obj_inhibitor_first

Position Distribution Summary:

Position	Count	Features
UTILITY (Support)	4	turretsLost, rune ×2, spell casts
MIDDLE	3	rune ×2, turretTakedowns
BOTTOM (ADC)	3	killing spree, magic damage taken, inhibitorTakedowns
TOP	2	turretsLost, rune stat
JUNGLE	2	rune, deaths
TEAM	1	first inhibitor

5.8.2 Correlation Structure of Selected Features

The correlation matrix of the top 15 features (`top_features_correlation.png`) reveals the interrelationships among the selected predictors. Understanding these correlations is essential for assessing feature redundancy, validating the effectiveness of the SULO V pre-filtering, and interpreting how the model combines information from related game events.

High Positive Correlations — Objective Clustering:

The correlation matrix reveals a distinct cluster of strongly positively correlated features related to macro-level objectives and structure control. This cluster occupies the intersection of structure loss, structure takedown, and team objective features:

- **obj_inhibitor_first_T1 and turretsLost_T2_TOP ($p = 0.82$):** This is the strongest correlation in the selected feature set. Team 1 securing the first inhibitor is strongly associated with Team 2's Top laner losing turrets. This relationship reflects the natural game progression: inhibitor takedowns typically occur after the defending team's turrets in that lane have already fallen. The correlation magnitude of 0.82 approaches the SULO V removal threshold (0.81) but falls just below it, allowing both features to survive into the final set.

- **turretTakedowns_T1_MIDDLE and turretsLost_T2_TOP ($\rho = 0.75$):** Team 1's Mid laner contributing to turret takedowns correlates strongly with Team 2's Top laner losing turrets. This cross-lane correlation reflects the interconnected nature of map control: when Team 1 is winning, they typically secure turret advantages across multiple lanes simultaneously, causing structure-related features to move together.
- **turretTakedowns_T1_MIDDLE and obj_inhibitor_first_T1 ($\rho = 0.68$):** Middle turret control is naturally correlated with inhibitor control, as the mid lane provides the most direct path to the enemy base. Securing mid turrets often precedes or coincides with inhibitor pressure.
- **deaths_T1_JUNGLE and turretsLost_T1_UTILITY ($\rho = 0.64$):** Team 1's Jungler deaths correlate with Team 1's Support losing turrets. This relationship likely reflects successful enemy bot-side plays: when the enemy team coordinates dives or invades the bot side, both the Jungler (responding to defend) and the Support (present in the lane) are vulnerable, and turret losses often follow.

Implications of Objective Clustering:

The clustering of objective-related features carries important implications for model interpretation. These features do not represent independent predictive signals; rather, they represent different measurement perspectives on the same underlying phenomenon—which team controls the map and is progressing toward victory. The model benefits from multiple perspectives because each feature captures slightly different timing and spatial information:

- **turretsLost_T2_TOP** captures side-lane map control from the defending perspective
- **turretTakedowns_T1_MIDDLE** captures central map control from the attacking perspective
- **obj_inhibitor_first_T1** captures base-penetration progress
- **inhibitorTakedowns_T1_BOTTOM** captures side-lane base pressure

For tree-based models (Random Forest, XGBoost, CatBoost, Gradient Boosting), moderate feature correlations of this magnitude are generally manageable. Tree splitting algorithms naturally handle collinearity by selecting among correlated features based on which provides the cleanest split at each node, rather than attempting to allocate variance among correlated predictors as linear models must. However, the correlations suggest that the selected feature set could potentially be reduced further—perhaps to 10-12 features—with minimal predictive power loss, by removing one feature from each highly correlated pair.

Notable Negative Correlations:

The correlation matrix reveals meaningful negative relationships that validate the logical consistency of the selected features:

- **obj_inhibitor_first_T1 and turretsLost_T1_UTILITY ($\rho = -0.48$):** When Team 1 secures the first inhibitor (a winning condition), they are simultaneously less likely to be losing their own turrets. This moderate negative correlation reflects the zero-sum nature of map control: structural advantage for one team implies structural disadvantage for the other.
- **perk_primary_rune_4_var2_T2_MIDDLE and perk_primary_rune_4_T2_MIDDLE ($\rho = -0.41$):** These two rune-related features for the same position (T2 Middle) exhibit moderate negative correlation. This relationship likely reflects mutually exclusive rune choices within a single rune slot—a Mid laner selects one rune option or another, but not both, creating a structural negative correlation in the data. This pattern confirms that the rune features are capturing genuine build diversity rather than redundant measurements.

Independent Features — Successful SULO V Filtering:

A substantial portion of the selected features exhibit near-zero correlation with most other features in the set, demonstrating the effectiveness of the SULO V pre-filtering stage:

- **Rune configuration features** (`perk_primary_rune_1_var1_T1_UTILITY` , `perk_primary_rune_4_T1_JUNGLE` , `perk_secondary_rune_1_var1_T1_UTILITY` , `perk_stat_flex_T2_TOP`): These features show correlations hovering near 0.00 with almost all other selected features. This independence confirms that rune choices capture information orthogonal to in-game performance statistics—they reflect pre-game strategic decisions rather than in-game execution quality.
- **Ability usage** (`spell4Casts_T1_UTILITY`): Support ultimate casts show low correlation with structure and combat features, suggesting that ability usage patterns provide independent predictive signal, perhaps capturing engagement frequency or playstyle aggression.
- **Damage taken** (`magicDamageTaken_T1_BOTTOM`): The Bottom laner's magic damage received is relatively independent of other selected features, potentially capturing lane-specific matchup dynamics (e.g., facing a magic-damage-heavy enemy composition) that are not reflected in structure or combat metrics.

Feature Selection Validation:

The correlation structure of the final 15 features validates the two-stage selection methodology:

1. **SULO V effectiveness:** No feature pair exceeds the 0.81 removal threshold, confirming that the SULO V stage successfully eliminated redundant feature pairs. The highest remaining correlation (0.82) approaches but does not exceed this threshold, representing a boundary case where both features were retained because each demonstrated sufficient independent predictive value.
2. **Complementary information:** The selected set successfully balances correlated objective features (providing multiple perspectives on map control) with independent rune, ability, and

damage features (providing orthogonal predictive signals). This diversity of information sources likely contributes to the models' robust performance.

3. **Interpretability preservation:** All 15 features have clear domain interpretations, avoiding the "black box" problem that can arise when feature selection retains obscure or uninterpretable variables. Each feature maps directly to a game concept that players, coaches, and analysts can understand and act upon.
-

5.8.3 Feature Set Characterization

Structural Features (4 of 15):

The structural features—two capturing turret losses, one capturing turret takedowns, and one capturing inhibitor takedowns—form the predictive backbone of the model. Their presence in the final set, combined with their dominant SHAP importance, confirms that structure control is the single most important dimension of League of Legends match prediction. The selection of both loss and takedown perspectives, and both Top lane and Mid lane spatial contexts, provides the model with a multi-faceted view of map control.

Rune Configuration Features (5 of 15):

The inclusion of five rune-related features—representing one-third of the selected set—is one of the more surprising findings. Rune configurations are selected before the match begins and do not change during gameplay, meaning these features capture pre-game strategic choices rather than in-game performance. Their predictive value may operate through several mechanisms:

- **Meta-optimality signal:** Certain rune choices may be objectively stronger in the current game patch, providing systematic advantages to players who select them.
- **Playstyle proxy:** Rune configurations signal intended playstyle (aggressive vs. passive, early-game vs. scaling), and these strategic orientations independently influence match outcomes.
- **Champion proxy:** Specific rune choices correlate with specific champion selections, potentially capturing champion strength information indirectly.

The concentration of rune features on UTILITY (Support) and MIDDLE positions suggests that strategic choices on these roles—which exert map-wide influence through roaming and vision control—are particularly consequential.

Combat and Damage Features (3 of 15):

`largestKillingSpree_T2_BOTTOM`, `deaths_T1_JUNGLE`, and `magicDamageTaken_T1_BOTTOM` represent the direct combat dimension. Notably, traditional offensive statistics (kills, damage

dealt, gold earned) are largely absent, while deaths and damage received—defensive metrics—are present. This asymmetry suggests that *preventing negative outcomes* (deaths, excessive damage taken) is more predictive than *achieving positive outcomes* (kills, high damage output). Teams that avoid feeding enemy carries and protect their own vulnerable members are more likely to win than teams that simply accumulate kills.

Ability Usage (1 of 15):

`spell4Casts_T1_UTILITY`—the Support's ultimate ability casts—is the sole ability usage feature in the final set. Ultimate abilities are the most impactful spells in each champion's kit, and the Support's ultimate typically provides game-changing engage, disengage, or sustain effects. The frequency of ultimate casts may reflect engagement tempo, teamfight participation, or effective cooldown management—all of which influence match outcomes.

Team Objective (1 of 15):

`obj_inhibitor_first_T1` is the only team-level (non-player-specific) feature in the selected set. Its presence confirms that the first inhibitor taken—a milestone event in game progression—carries predictive weight beyond what individual player statistics capture.

5.8.4 Practical Recommendations for Feature Set Refinement

The correlation analysis suggests several potential refinements for future iterations of this research:

Option 1: Conservative (No Changes):

Retain all 15 features. The SULOV threshold of 0.81 was deliberately chosen based on reference literature, and no pair exceeds this threshold (the 0.82 correlation is a boundary case). Tree-based models handle the observed correlation levels effectively, and the marginal benefit of removing one feature from each correlated pair is likely small.

Option 2: Moderate Reduction (12 Features):

Remove one feature from each of the two highest-correlated pairs:

- Consider dropping `turretsLost_T2_TOP` (retaining `obj_inhibitor_first_T1` at $\rho = 0.82$)
- Consider dropping `turretTakedowns_T1_MIDDLE` (retaining `turretsLost_T2_TOP` at $\rho = 0.75$)

This would reduce the feature set to 12-13 features with likely minimal impact on predictive performance, while slightly improving model simplicity and training speed.

Option 3: Aggressive Reduction (8-10 Features):

Remove all but one feature from the objective cluster, retaining only the single most predictive structure feature and supplementing with the independent rune, combat, and ability features. This would produce a highly compact model suitable for deployment in resource-constrained environments, though at some cost to predictive nuance.

Recommendation:

Given the strong performance achieved with the full 15-feature set (R^2 up to 0.870, Accuracy up to 96.5%), and the manageable correlation levels, the conservative approach (Option 1) is recommended for most applications. The marginal gains from further reduction are unlikely to justify the effort, particularly given that tree-based models handle the existing correlations effectively. However, researchers seeking to deploy lightweight models or to conduct detailed feature ablation studies may find value in exploring Options 2 or 3.

This selected features analysis addresses **Research Question 2 (Feature Importance)** by (a) presenting the complete final feature set with importance scores and categorical/positional distributions, (b) analyzing the correlation structure to validate the SULOV selection methodology, (c) identifying clusters of related features and independent features, and (d) providing recommendations for potential further refinement. The selected feature set successfully balances predictive power, interpretability, and diversity of information sources, contributing to the robust model performance documented throughout this chapter.

5.9 Dataset Summary

5.9.1 Dataset Characteristics

The dataset information file (`dataset_info.txt`) provides a concise summary of the dataset's evolution through the machine learning pipeline, from raw data loading through feature engineering and selection to the final training-ready feature matrix. Table 5.6 consolidates these statistics with additional contextual information.

Table 5.6: Dataset Evolution Across Pipeline Stages

Pipeline Stage	Matches	Features	Feature Reduction	Notes
Raw Participant Data	2,000	254+	—	10 rows per match, extensive Riot API fields

Pipeline Stage	Matches	Features	Feature Reduction	Notes
After Feature Engineering	2,000	1,254	—	10× participant features + team objectives
After SULOV Removal	2,000	1,050	204 removed (16.3%)	Correlation threshold = 0.81
Final Selected Features	2,000	15	1,035 removed (98.5%)	XGBoost top-15 recursive elimination

Note: The raw participant feature count depends on Riot API version and challenge metric availability. The 254+ figure represents typical column counts observed during data loading.

5.9.2 Dataset Scale and Composition

Match Count:

The final dataset comprises 2,000 League of Legends ranked solo/duo matches collected through the tier-stratified snowball sampling methodology detailed in Chapter 3. This sample size balances several considerations:

- **Statistical power:** 2,000 matches provide sufficient observations for training machine learning models with 15 features, maintaining a healthy observation-to-feature ratio of approximately 133:1 in the training set (1,600 training matches ÷ 15 features ≈ 107:1, well above the common heuristic of 10:1 minimum).
- **Computational feasibility:** The 100-iteration stability analysis requires training models 100 times on different data partitions. At 2,000 matches, each iteration completes in seconds, making the full stability analysis computationally tractable.
- **Tier representation:** The stratified sampling design ensures that matches from Iron through Challenger are all represented, though not in proportion to their player population sizes. The overrepresentation of elite tiers (Master, Grandmaster, Challenger at 200 matches each vs. 50 per division for lower tiers) was an intentional design choice to ensure sufficient data for modeling high-skill gameplay patterns.

T1 Win Rate:

The T1 win rate of 0.463 (46.3%) indicates a moderate class imbalance, with Team 1 winning slightly fewer than half of all matches. Several factors may contribute to this asymmetry:

- **Arbitrary team assignment:** T1/T2 designation is based on Riot's internal `teamId` ordering (typically 100 for blue side, 200 for red side) rather than any game-meaningful characteristic. However, the blue side/red side asymmetry in League of Legends—attributed

to camera angle, HUD positioning, and draft order—is well-documented, with blue side historically enjoying a slight win rate advantage in many patches.

- **Sampling variation:** With 2,000 matches, a deviation of 0.037 from perfect balance (0.500) is within expected sampling error. A binomial test would confirm whether this deviation is statistically significant or attributable to chance.
- **Stratification preservation:** The stratified train-test split (`stratify=y`) ensures that this class distribution is preserved in both training and test partitions, preventing partition-specific imbalance from distorting evaluation metrics.

Practical Implications of Class Imbalance:

The observed imbalance (46.3% T1 wins) is mild and unlikely to meaningfully affect model training or evaluation for several reasons:

- All models substantially exceed the majority-class baseline (53.7% accuracy achievable by always predicting T2 victory), confirming that learned patterns rather than class priors drive predictions.
- The stratification in train-test splitting ensures consistent class representation across partitions.
- The R^2 metric, our primary optimization target, is scale-invariant and unaffected by class balance.
- Accuracy remains interpretable despite mild imbalance; only severe imbalance (>90% majority class) would necessitate alternative metrics like balanced accuracy or F1 score.

5.9.3 Dimensionality Reduction Magnitude

The feature selection process achieved a remarkable 98.5% reduction in feature space dimensionality—from 1,254 engineered features to 15 selected features—while maintaining predictive performance that approaches the theoretical ceiling for this task ($R^2 = 0.870$, Accuracy = 96.5%). This dramatic compression yields several benefits:

Computational Efficiency:

Training on 15 features rather than 1,254 reduces model training time by orders of magnitude. The GridSearchCV hyperparameter optimization, which requires training hundreds of model instances across cross-validation folds, becomes computationally tractable. The 100-iteration stability analysis, which would be prohibitively expensive on the full feature set, completes in reasonable time.

Interpretability:

A 15-feature model can be fully understood and communicated to domain stakeholders. Each feature has a clear game-meaningful interpretation, and the SHAP analysis (Section 5.5)

provides instance-level explanations of how each feature contributes to predictions. A 1,254-feature model, by contrast, would resist meaningful interpretation regardless of analytical methodology.

Generalizability:

Parsimonious models with few features are less prone to overfitting, particularly with moderate sample sizes. The 15-feature model captures the dominant predictive signals while discarding noise and idiosyncratic patterns that might not generalize beyond the collected dataset. The stability analysis results (Section 5.3), showing consistent performance across data partitions, validate this generalizability.

Deployment Feasibility:

A 15-feature model can be deployed in resource-constrained environments—web applications, mobile analytics tools, real-time coaching overlays—where a 1,254-feature model would be impractical. The reduced input requirements also simplify data collection for deployment: only 15 specific statistics need to be extracted from the Riot API rather than the full participant data payload.

5.9.4 Best Model Identification

The dataset information file identifies the Voting Regressor (VotR) as the best-performing model based on the primary evaluation metric ($R^2 = 0.8704$). This designation reflects the single-split evaluation results presented in Section 5.2. The comprehensive analyses throughout this chapter have refined this initial assessment:

- **By R^2 (variance explained):** VotR (0.870) leads, followed closely by CatBoost (0.868)
- **By Accuracy (classification):** Ridge (96.5%) leads, followed by CatBoost (96.0%)
- **By Stability (consistency):** CatBoost demonstrates the lowest R^2 standard deviation (0.0192) among individually assessed models
- **By Error Metrics (calibration):** Gradient Boosting substantially outperforms in MAE and MAD

The "best model" designation is therefore application-dependent. The Voting Regressor provides the highest overall variance explanation, justifying its identification as best for research contexts where maximizing R^2 is the primary objective. For deployment contexts, the analyses in Sections 5.2 through 5.7 provide detailed guidance for selecting among CatBoost (balanced performance), Gradient Boosting (superior calibration), or Ridge (maximum classification accuracy) based on specific application requirements.

5.9.5 Dataset Limitations in Context

The dataset characteristics quantified here contextualize the limitations acknowledged in Section 1.5.2:

- **Sample size (2,000 matches):** While adequate for the current analysis, this sample size limits the detection of subtle effects. Rare champion picks, unusual team compositions, or atypical game trajectories may be underrepresented or absent, limiting model performance on edge cases.
- **Single-region origin (EUN1):** Meta-game patterns, champion preferences, and playstyle norms vary across regions. Models trained exclusively on EUN1 data may exhibit performance degradation when applied to Korean, North American, or Chinese server matches.
- **Temporal snapshot:** The dataset was collected during a specific period of the 2024-2025 season. Subsequent balance patches may alter the relationship between features and outcomes, potentially reducing model accuracy over time without retraining.
- **T1 win rate imbalance (46.3%):** While mild, this imbalance indicates that T2 (typically red side) enjoys a slight systematic advantage in the collected matches. Models may learn this prior, potentially affecting predictions on future data where the side advantage may differ due to patch changes.

5.9.6 Reproducibility Documentation

The dataset information file, combined with the detailed methodology in Chapters 3 and 4, provides a comprehensive record for reproduction:

- **Input data:** The specific CSV files used are identified by timestamped filenames (`teams_stats_merged_20260616_170258.csv` , `match_data_merged_20260616_170258.csv`), enabling exact data version tracking.
- **Feature counts:** The progression from 1,254 original features through 1,050 post-SULOV features to 15 final features is documented, allowing future researchers to verify that their implementation reproduces these counts.
- **Target distribution:** The T1 win rate of 0.463 serves as a validation check—any reproduction attempt should yield this same value (within rounding) when using the identical input data.
- **Output files:** The saved files (`model_comparison_results.csv` , `feature_importance.csv` , `selected_features.txt`) provide permanent records of results that can be compared against reproduction attempts.

This documentation aligns with the open-source contribution objective (O7) established in Section 1.4, facilitating verification, extension, and application of the research findings by the broader community.

This dataset summary consolidates the quantitative characteristics of the data underlying all analyses presented in this chapter. The 2,000-match dataset, reduced from 1,254 to 15 features through rigorous two-stage selection, supports robust predictive modeling with excellent generalizability characteristics. The mild class imbalance (46.3% T1 wins) does not meaningfully affect model evaluation, and the comprehensive documentation enables full reproduction. The following section synthesizes all chapter findings into a comprehensive discussion.

6: Discussion

6.1 Interpretation of Predictive Feature Patterns

6.1.1 The Primacy of Structural Objectives

The most robust finding to emerge from this research—confirmed across XGBoost gain importance, SHAP analysis, and correlation structure examination—is the overwhelming predictive dominance of structural objectives, particularly turret losses. The two most important features (`turretsLost_T2_TOP` and `turretsLost_T1_UTILITY`) alone account for approximately 27% of total XGBoost feature importance and exhibit SHAP values an order of magnitude larger than any other predictors. This finding is not merely statistically significant; it is substantively dominant to a degree that reshapes our understanding of what drives League of Legends match outcomes.

Alignment with Game Design:

This empirical finding aligns closely with Riot Games' explicit design philosophy. Over multiple seasons, the developers have systematically increased the strategic importance of turret control through mechanisms including:

- **Turret plating (introduced Season 9):** Additional gold rewards for early turret damage incentivize lane dominance and punish passive play, making turret pressure a primary win condition rather than a byproduct of other advantages.
- **Elemental dragon souls (Season 10):** While dragons themselves are neutral objectives, controlling the river areas adjacent to lanes requires turret superiority to safely traverse and contest. Teams losing outer turrets cede dragon priority.
- **Rift Herald (reworked Season 8):** This objective directly accelerates turret destruction, creating a explicit gameplay loop where jungle control translates to structure damage.

The finding that turret losses predict outcomes more powerfully than kills, gold, or experience differentials validates that Riot's design intentions are reflected in actual match dynamics. Turrets are not merely scoreboard decorations; they are the mechanism through which map control is established, and map control—rather than combat prowess or economic advantage—is the true determinant of victory.

Theoretical Implications:

The dominance of structural features over combat features challenges a common implicit assumption in esports analytics: that individual mechanical skill, as measured by kill/death ratios and damage output, is the primary driver of match outcomes. Our results suggest a more nuanced hierarchy:

1. **Structural control** (turrets, inhibitors) establishes the strategic framework within which combat occurs
2. **Role-specific survival** (particularly Jungler deaths) determines whether a team can contest or defend structures
3. **Combat execution** (kills, damage, gold) influences outcomes primarily through its effect on the first two tiers

This hierarchy implies that a team achieving superior combat statistics without converting those advantages into structure damage is statistically indistinguishable from a team that never achieved those advantages at all. The "win more" phenomenon—accumulating kills without pressing objectives—may be a common failure mode that our models implicitly identify and penalize.

6.1.2 The Unexpected Importance of Rune Configurations

The prominence of rune configuration features among the top 15 selected features—five of fifteen, representing one-third of the final feature set—constitutes an unexpected finding that warrants careful interpretation.

Pre-Game Strategic Signaling:

Rune configurations are selected during champion select, before any gameplay occurs. Their predictive value therefore cannot reflect in-game performance quality directly. Instead, rune choices likely serve as proxies for several latent variables:

- **Meta awareness:** Players who select statistically optimal rune configurations for their champion and matchup demonstrate knowledge of the current meta-game, which may correlate with overall game understanding and decision-making quality.
- **Playstyle intention:** Rune choices signal intended strategic orientation—aggressive laning (e.g., Electrocute, Press the Attack) versus scaling (e.g., Conqueror, Grasp of the Undying) versus utility (e.g., Guardian, Glacial Augment). These strategic intentions shape how the match unfolds and independently influence outcomes.
- **Champion identity proxy:** Although `championId` features were available in the feature space, they did not survive feature selection. Rune configurations may indirectly capture champion identity information, as specific runes are strongly associated with specific champions.

Concentration on Support Role:

The concentration of rune importance on the UTILITY (Support) position (three of five rune features) is particularly revealing. Support rune choices—particularly keystone selection—fundamentally define a team's engagement pattern:

- **Engage supports** (Leona, Nautilus, Thresh) typically select Aftershock, signaling a pick-oriented, aggressive playstyle
- **Enchanter supports** (Lulu, Janna, Soraka) select Summon Aery or Guardian, signaling a protective, scaling playstyle
- **Mage supports** (Brand, Zyra, Xerath) select Arcane Comet or Dark Harvest, signaling a damage-oriented, lane-dominant playstyle

Each archetype imposes distinct win conditions and power curves on the team. A composition with an engage support must find picks and force fights to win; a composition with an enchanter support must protect carries and scale. The model may be learning these archetypal patterns and adjusting predictions accordingly—a sophisticated form of strategic understanding that emerges purely from statistical associations.

Causal Caution:

We must emphasize that rune features' predictive power does not imply that rune choices *cause* victory. A player who selects meta-optimal runes and wins likely would have won regardless of rune selection; the rune choice merely signals the game knowledge that also manifests in superior in-game decision-making. This distinction is critical for avoiding the ecological fallacy of interpreting predictive features as prescriptive recommendations. Players should not conclude that changing runes will improve their win rate; rather, they should recognize that optimal rune selection correlates with the broader game understanding that drives winning performance.

6.1.3 Jungler Deaths as the Key Combat Metric

The emergence of `deaths_T1_JUNGLE` as the highest-ranked traditional combat statistic (rank 14 by XGBoost importance, significant SHAP value in secondary tier) and the notable *absence* of kills, damage, or gold features from the top predictors warrant focused discussion.

The Cascading Impact of Jungler Deaths:

The Jungler role occupies a unique position in League of Legends' strategic ecosystem. Unlike laners, who are spatially anchored to specific lanes during the early game, the Jungler roams the entire map, influencing all lanes through ganks and controlling neutral objectives. When a Jungler dies, the consequences cascade across multiple dimensions simultaneously:

- **Map pressure loss:** All three lanes lose the threat of jungle intervention, allowing enemy laners to play aggressively without fear of ganks
- **Objective vulnerability:** The enemy team gains a window to secure Dragon, Rift Herald, or Baron uncontested
- **Jungle resource loss:** The enemy Jungler can safely invade and steal camps, creating an experience and gold differential that compounds over time
- **Vision control loss:** Junglers are primary ward-placers in river and enemy jungle; their death cedes vision control of contested areas

No other role's death carries such multidimensional consequences. A Top laner's death primarily affects the top lane; a Jungler's death affects the entire map. The model's recognition of this asymmetric impact—learning that Jungler deaths are more predictive than deaths on other roles—demonstrates sophisticated pattern extraction from the training data.

The Absence of Offensive Statistics:

Perhaps equally informative is what did *not* appear among top features: kills, total damage dealt, gold earned, and creep score. These traditional performance metrics, which dominate casual player evaluation and post-game discussion, are notably absent from the most predictive features. This absence suggests that:

- **Kills are means, not ends:** Teams that secure many kills without converting them into structure damage do not significantly improve their victory probability beyond what the kill count alone would suggest
- **Damage dealt is context-dependent:** High damage output against tanks who sustain through healing may be less impactful than moderate damage that secures kills on carries
- **Gold advantages must be spent effectively:** Gold earned translates to victory only when converted into item advantages that enable structure taking or teamfight winning; the conversion efficiency varies dramatically

This finding echoes Maymin's (2021) distinction between "smart kills" (those that enable objective control) and "worthless kills" (those that merely inflate score lines). Our models, by weighting structure outcomes above combat outcomes, implicitly operationalize this distinction.

6.1.4 The Support Role's Predictive Prominence

The UTILITY (Support) position appears in 4 of the 15 selected features—the highest representation of any single role—including the critically important `turretsLost_T1_UTILITY` (rank 2 overall) and three rune configuration features. This prominence challenges casual perceptions of Support as a low-impact role and aligns with competitive understanding of Support as a strategic multiplier.

Support as Force Enabler:

Supports typically deal minimal damage and secure few kills, making their direct statistical footprint small. However, their indirect influence—through vision control, engagement initiation, carry protection, and map mobility—fundamentally enables or constrains what the rest of the team can accomplish. A Support who establishes deep vision control enables safe objective attempts; a Support who lands engage crowd control creates kill opportunities for carries; a Support who effectively peels divers protects the damage output that wins teamfights.

The model's recognition of Support-related features as highly predictive suggests that Support performance, despite being difficult to quantify through traditional combat statistics, manifests in structural outcomes. When a Support performs poorly, turrets fall. When a Support selects appropriate runes for the team composition, the strategic framework for victory is established. The model has learned to read Support impact through its consequences rather than its direct outputs.

Implications for Player Evaluation:

This finding has practical implications for player evaluation and scouting. Traditional metrics like KDA systematically undervalue Support contributions. Analytics frameworks that incorporate structure-related metrics—turret participation, objective control percentage, vision score per structure taken—may provide more accurate assessments of Support effectiveness. Our SHAP analysis suggests that a Support's true impact is best measured not by what they directly achieve but by what their team achieves when they are present and what the enemy achieves when they are absent.

6.2 Model Performance Characteristics

6.2.1 The Ensemble Advantage: Theory and Empirical Validation

The Voting Regressor's position as the highest R^2 model (0.8704) empirically validates ensemble theory in the context of League of Legends win prediction. The theoretical basis for ensemble superiority rests on the assumption that different models make uncorrelated errors—when one model overestimates, another underestimates, and their average is closer to truth. Our results suggest that this condition is met: CatBoost, XGBoost, and Gradient Boosting, despite all being gradient boosting variants, exhibit sufficiently distinct error patterns that their combination yields measurable improvement.

The magnitude of ensemble improvement (R^2 gain of approximately 0.002 over CatBoost alone) is modest but consistent with expectations when base models are already strong. Ensemble methods provide the largest gains when base models are weak and diverse; when base models are strong and similar (as our gradient boosting variants are), marginal improvements are

incremental. The Voting Regressor's consistent top-tier performance across both R^2 and accuracy metrics suggests that the ensemble premium, while small, is reliable.

6.2.2 The Gradient Boosting Family's Fitness for Gaming Data

The dominance of gradient boosting methods (CatB, XGB, GradB occupying positions 2-4 in R^2 rankings, with only the ensemble VotR ahead) reflects these algorithms' particular suitability for the characteristics of League of Legends match data:

Heterogeneous Feature Types:

The selected features span continuous variables (damage taken, spell casts), discrete counts (deaths, turrets lost, inhibitor takedowns), binary indicators (first inhibitor), and categorical variables (rune selections, stat shards). Gradient boosting methods natively handle this heterogeneity through their split-based decision framework, which makes no distributional assumptions about feature types. Linear models (Ridge, SVR) struggle with categorical-discrete mixtures; distance-based methods (KNNR) require careful scaling of heterogeneous features. Tree-based methods accommodate the diversity naturally.

Feature Interactions:

League of Legends gameplay is characterized by complex interactions: the impact of a Jungler death depends on whether objectives are available; the value of a turret takedown depends on which turret and at what game time; the effectiveness of rune choices depends on team composition and matchup. Tree-based models capture these interactions through hierarchical splitting—a death feature might be highly predictive only when certain turret states are present, and tree splits naturally partition the data to reveal such conditional relationships. Gradient boosting's sequential construction further refines these interaction models, with each new tree correcting errors from previous trees that may have failed to capture specific interaction patterns.

Robustness to Outliers:

Gaming data frequently contains outliers—unusually long matches, extreme kill counts, atypical damage distributions. Random Forest and gradient boosting methods provide natural outlier robustness through ensemble averaging (RF) or through the focus on residual correction rather than raw value fitting (boosting). This robustness manifests in the stability analysis results, where tree-based models exhibit lower variance across data partitions than other model families.

6.2.3 The Ridge Anomaly: Classification Without Calibration

Ridge Regression's paradoxical performance—worst R^2 (0.755) yet best accuracy (96.5%)—represents one of the most methodologically instructive findings of this research. This anomaly

demonstrates that **evaluation metric selection is not merely a technical detail but a substantive choice that determines which models appear "best."** Researchers and practitioners must align metric selection with application requirements.

The mechanism underlying the Ridge anomaly—L2 regularization shrinking predictions toward moderate values that nonetheless remain on the correct side of the 0.5 threshold—reveals an important distinction between two prediction tasks that are often conflated:

- 1. **Winner identification:** Which team won? This binary question requires only that the prediction lands on the correct side of 0.5. Ridge excels here.
- 2. **Win probability estimation:** What was the likelihood of victory? This continuous question requires that predictions are well-calibrated—a 75% prediction should correspond to actual victory 75% of the time. Ridge performs poorly here, as its shrunken predictions systematically underestimate the confidence warranted by the game state.

Applications that require probability calibration—in-game win probability displays, betting odds calculation, comeback likelihood estimation—should prefer CatBoost or Gradient Boosting despite their marginally lower accuracy. Applications that require only binary classification—post-match winner identification, automated match reporting, win/loss record compilation—may legitimately prefer Ridge for its simplicity and superior classification rate.

6.2.4 Stability-Performance Trade-offs

The stability analysis revealed that no single model dominates all dimensions simultaneously. CatBoost provides the most stable classification and R² performance; Gradient Boosting provides dramatically superior error metrics (MAE, MAD) but with higher R² variance; XGBoost provides the tightest R² interquartile range. This multi-dimensional performance landscape implies that model selection should be guided by application-specific priorities:

Priority	Recommended Model	Rationale
Reliable winner identification	CatBoost	Highest accuracy stability ($\sigma = 0.97\%$)
Well-calibrated probabilities	Gradient Boosting	Lowest MAE (0.0698) and MAD (0.0061)
Best overall balance	CatBoost	Top-tier accuracy, R ² , and stability
Lightweight deployment	Ridge	Minimal complexity, highest accuracy
Maximum R ² (research)	VotR	Ensemble premium over individual models

6.3 Practical Implications

6.3.1 For Players and Coaches

Prioritize Structure Over Kills:

The most actionable implication of this research is that structural objectives—particularly turret preservation—should be the central focus of strategic decision-making. Players and teams should evaluate decisions not by their kill potential but by their structure impact: "Does this play save or take a turret?" is a more winning-relevant question than "Does this play secure a kill?" Specific recommendations include:

- **Wave management to protect turrets:** Properly managing minion waves to prevent enemy minions from reaching turrets preserves structural integrity without requiring player presence
- **Rotations for turret defense:** Responding to enemy split-push threats before turrets take damage, even at the cost of conceding minor objectives elsewhere
- **Herald prioritization:** Rift Herald directly accelerates turret destruction; contesting or securing this objective has outsized impact on win probability
- **Dive evaluation:** Tower dives that trade kills for deaths may appear KDA-neutral but often result in turret plate gold and structural damage that the SHAP analysis identifies as highly predictive of outcomes

Protect the Jungler:

Jungler deaths are the most predictive individual combat metric. Teams should prioritize Jungler safety through:

- **Invasion warding:** Deep vision in enemy jungle provides early warning of enemy Jungler pathing, enabling safe routing
- **Lane priority for scuttle contests:** Laners pushing their waves before scuttle crab spawns enables Jungler support during river contests
- **Death punishment:** Recognizing that enemy Jungler deaths create cascading opportunities—objectives, invades, gank pressure—and capitalizing immediately

Support Impact Recognition:

The Support role's prominence in predictive features validates its strategic importance. Coaches should evaluate Support performance through structure-related metrics rather than combat statistics. Supports should recognize that their rune choices, ultimate usage patterns, and positioning relative to turrets directly influence match outcomes in ways the model quantitatively captures.

6.3.2 For Game Developers and Balance Teams

Validating Design Intentions:

The dominance of structure-related features confirms that Riot Games' design emphasis on objective control is reflected in actual match dynamics. The game's strategic depth—where map control matters more than mechanical outplay—is empirically validated. This alignment between design intention and predictive reality is not guaranteed; alternative game designs could produce different predictive hierarchies (e.g., games where individual combat skill dominates, or where economic efficiency matters most).

Monitoring Meta Shifts:

The feature importance hierarchy documented in this research provides a baseline against which future meta-game shifts can be measured. If a future patch significantly elevates combat features (kills, damage) relative to structure features in predictive importance, this would indicate a meta-game shift toward deathmatch-style gameplay—potentially warranting design intervention to restore strategic depth. Conversely, strengthening objective rewards would further concentrate importance in the already-dominant structure features.

Role Balance Assessment:

The asymmetric representation of roles among top predictive features—UTILITY (4 features), MIDDLE (3), BOTTOM (3), TOP (2), JUNGLE (2)—provides an empirical perspective on role influence. While this distribution should not be interpreted as a prescriptive target (perfect balance across roles is neither achievable nor desirable), substantial shifts in this distribution across patches could indicate unintended role power changes.

6.3.3 For Esports Broadcasters and Analysts

Data-Driven Storytelling:

The identified predictive hierarchy—structures first, then Jungler deaths, then combat metrics—provides a framework for broadcast analysis. When analyzing a match in progress, casters and analysts can ground their narrative in the statistics that most strongly predict outcomes:

- "Team A has lost two outer turrets while Team B's structures remain intact—our models indicate this is the single strongest predictor of match outcome."
- "The Jungler death differential is concerning for Team A; Jungler deaths cascade into objective losses in ways that lane deaths do not."

Win Probability Models:

The model performance documented in this research (R^2 up to 0.87, Accuracy up to 96.5%) demonstrates that post-game statistics can predict outcomes with high confidence. While our models use complete-game data and are not directly applicable to live win probability estimation, the identified predictive features suggest which in-game metrics should be prioritized for real-time tracking: turret states, Jungler deaths, inhibitor status, and rune configurations.

6.3.4 For the Research Community

Methodological Contributions:

Several methodological findings from this research are transferable to other esports analytics contexts:

- **Position-preserving feature engineering:** The T1/T2 positional prefixing approach can be applied to any team-based game with role differentiation (Overwatch, Valorant, Dota 2), enabling role-specific predictive modeling from participant-level data.
- **Two-stage feature selection:** The combination of SULOV correlation filtering with boosting-based recursive elimination provides a template for dimensionality reduction in high-dimensional gaming datasets where feature redundancy is prevalent.
- **Stability analysis integration:** The 100-iteration repeated random subsampling protocol establishes a standard for rigorous model evaluation beyond single-split reporting.
- **SHAP as primary interpretability tool:** The finding that SHAP and gain-based importance diverge for certain feature categories (runes) demonstrates the value of model-agnostic interpretability methods over model-specific importance metrics.

Open-Source Contribution:

The release of the complete implementation code (Objective O7) enables other researchers to reproduce these findings, apply the methodology to their own datasets, and extend the pipeline with additional models or feature engineering approaches. The detailed documentation of dataset characteristics (Section 5.9) provides the metadata necessary for meaningful comparison across studies.

6.4 Limitations and Threats to Validity

6.4.1 Internal Validity

Feature Selection Threshold Sensitivity:

The SULOV correlation threshold (0.81) and the top-N selection count (15 features) are design parameters that influence which features enter the final model. Different thresholds would yield different feature subsets, potentially altering the feature importance hierarchy and model performance. We partially addressed this concern through SHAP analysis, which provides a model-agnostic importance measure less sensitive to selection methodology than the gain-based XGBoost importance used during selection. The convergence of both methods on structure-related features increases confidence that these findings are not artifacts of specific threshold choices.

Regression-to-Classification Conversion:

The 0.5 threshold for converting regression predictions to binary classifications, while standard, may not be optimal for all models. Models with systematically biased probability estimates (potentially including Ridge) might benefit from calibrated thresholds optimized on validation data. Our primary reliance on R^2 as the optimization metric partially mitigates this concern, as R^2 evaluates the continuous predictions directly without threshold effects.

MAPD Accuracy Instability:

The numerical instability observed in MAPD Accuracy calculations (producing values in the negative billions) was identified and this metric was excluded from analysis. This exclusion means we lack a formal accuracy metric normalized by target magnitude, though the other five metrics (Accuracy, R^2 , MAE, RMSD, MAD) provide comprehensive coverage of model performance dimensions.

6.4.2 External Validity

Region and Patch Generalizability:

The dataset originates from a single region (EUN1) during a specific period of the 2024-2025 season. While the tier-stratified sampling ensures representation across skill levels within this region, the findings may not fully generalize to:

- **Other regions:** Korean, Chinese, and North American servers exhibit distinct meta-game characteristics, champion preferences, and playstyle norms that may alter the feature importance hierarchy
- **Professional play:** Elite competitive matches differ from ranked solo queue in coordination level, compositional strategy, and objective prioritization; the predictive features identified here may not directly transfer
- **Future patches:** Balance changes can alter the relationship between features and outcomes; a patch that significantly buffs dragon rewards, for example, might shift importance from turrets toward dragon-related features

Queue Type Specificity:

The dataset exclusively contains Ranked Solo/Duo (queue ID 420) matches. Normal games, Flex queue (organised teams), and ARAM (single-lane mode) involve different player motivations, coordination levels, and game dynamics. The predictive patterns identified here may not extend to these alternative game modes.

6.4.3 Construct Validity

Correlation Versus Causation:

Throughout this discussion, we have been careful to distinguish predictive association from causal influence. However, the distinction bears repeating as a fundamental limitation: features that predict victory are not necessarily features that *cause* victory. Turret losses may predict defeat because they reflect an already-lost game state rather than independently causing the loss. Our models identify *markers* of winning and losing, not necessarily *drivers*. Causal claims would require experimental manipulation or causal inference methods beyond the scope of this observational study.

Feature Interpretation Ambiguity:

Some features admit multiple interpretations. The `obj_inhibitor_first_T1` feature, for example, could represent Team 1 securing the first inhibitor (positive) or the first inhibitor being taken from Team 1 (negative). While the SHAP directionality analysis helps disambiguate such cases, some uncertainty remains. Comprehensive feature documentation from the data collection phase would improve interpretability.

6.4.4 Statistical Conclusion Validity

Multiple Comparisons:

This research evaluated 10 models across 6 metrics, conducted 100-iteration stability analysis on 3 models, and examined 517 features with non-zero importance. While we have not performed formal multiplicity corrections (which would be unusually conservative for exploratory machine learning research), readers should recognize that the most extreme observed values may reflect some degree of selection from multiple comparisons. The convergence of findings across independent analytical methods (XGBoost importance, SHAP, correlation structure, stability patterns) provides protection against isolated false positives.

Sample Size for Stability Estimates:

The 100-iteration stability analysis provides standard error of the mean equal to approximately one-tenth of the standard deviation, yielding reasonable precision for mean estimates. However, the standard deviation estimates themselves have uncertainty; with 100 observations,

the 95% confidence interval for the true standard deviation spans approximately $\pm 14\%$ of the estimated value. The stability results should be interpreted with this precision in mind.

7: Conclusion and Future Work

7.1 Summary of Key Contributions

This research has developed, implemented, and rigorously evaluated a comprehensive machine learning pipeline for predicting League of Legends match outcomes from post-game player statistics and team objective data. The principal contributions of this work span methodological innovation, empirical findings, and practical implications.

7.1.1 Methodological Contributions

Position-Preserving Feature Engineering:

We designed and implemented a feature engineering methodology that transforms participant-level match data—natively structured as ten player records per match—into a single match-level feature vector while preserving individual player position information. The systematic team-position prefixing approach (T1_TOP, T2_JUNGLE, etc.) enables models to learn role-specific predictive patterns that naive aggregation would obscure. This methodology is transferable to any team-based game with role differentiation and structured performance statistics.

Two-Stage Feature Selection Pipeline:

We demonstrated the effectiveness of combining SULO V correlation analysis (filter method, threshold 0.81) with XGBoost recursive feature elimination (embedded/wrapper method) for dimensionality reduction in high-dimensional gaming datasets. This two-stage approach reduced the feature space from 1,254 engineered features to 15 selected features—a 98.5% reduction—while maintaining predictive performance near the theoretical ceiling for this task. The correlation structure analysis validated that the SULO V stage successfully eliminated redundant feature pairs while preserving complementary independent signals.

Comprehensive Model Evaluation Framework:

We established a multi-faceted evaluation protocol incorporating six complementary metrics (R^2 , Accuracy, MAE, RMSD, MAD, MAPD Accuracy), 100-iteration stability analysis with mean and standard deviation reporting, and SHAP-based interpretability analysis. This framework provides richer performance characterization than the accuracy-only or single-split evaluation common in prior literature, enabling nuanced model comparison and application-specific model selection guidance.

Open-Source Reproducibility:

The complete implementation code, including data preprocessing, feature engineering, model training, evaluation, and visualization, has been released as open-source software. This contribution directly addresses reproducibility limitations identified in multiple prior studies and provides a foundation for future research to build upon.

7.1.2 Empirical Contributions

Predictive Performance Achievement:

Our best-performing model, the Voting Regressor ensemble, achieved an R^2 of 0.8704 and accuracy of 95.75% on held-out test data, substantially exceeding the random chance baseline of 50%. All ten evaluated models achieved accuracy above 94.75%, demonstrating that the engineered features capture robust predictive signals accessible to diverse algorithmic approaches.

Feature Importance Hierarchy:

Through convergent evidence from XGBoost gain importance and SHAP analysis, we established a clear predictive hierarchy: structural objectives (particularly turret losses) dominate, followed by role-specific deaths (particularly Jungler), with combat and economic metrics occupying a secondary tier. The two most important features— `turretsLost_T2_TOP` and `turretsLost_T1_UTILITY` —account for approximately 27% of total feature importance and exhibit SHAP values an order of magnitude larger than other predictors.

Rune Configuration Significance:

We identified rune configuration features—pre-game strategic choices—as unexpectedly prominent among top predictors, representing five of the fifteen selected features. SHAP analysis revealed that while these features facilitate tree splitting during training, their marginal contribution to final predictions is modest, highlighting the importance of model-agnostic interpretability methods over model-specific importance metrics.

Stability Characterization:

The 100-iteration stability analysis established confidence bounds for model performance: CatBoost achieved the most stable classification (accuracy $\sigma = 0.97\%$) and R^2 ($\sigma = 0.0192$), while Gradient Boosting demonstrated dramatically superior error metrics (MAE = 0.0698, MAD = 0.0061) despite higher R^2 variance ($\sigma = 0.0242$).

The Ridge Anomaly:

We documented and explained a significant anomaly: Ridge Regression achieved the highest accuracy (96.5%) despite the lowest R^2 (0.755), a paradox arising from L2 regularization

shrinking predictions toward moderate values that nonetheless remain on the correct side of the classification threshold. This finding demonstrates that metric selection fundamentally determines which models appear optimal, with practical implications for application-specific model selection.

7.2 Addressing the Research Questions

7.2.1 Primary Research Question

RQ1: To what extent can machine learning models accurately predict the winning team in League of Legends ranked matches using post-game player statistics, team objective data, and position-preserving feature engineering?

The evidence provides a clear affirmative answer: machine learning models can predict match outcomes with high accuracy. The Voting Regressor ensemble achieved 95.75% accuracy and R^2 of 0.8704, explaining approximately 87% of outcome variance from post-game features alone. All ten evaluated models substantially exceeded the random baseline, with the weakest model (KNNR) still achieving 94.75% accuracy. The stability analysis confirmed that these performance levels are robust across data partitions, with mean accuracy remaining above 95% and standard deviations below 1.1 percentage points across 100 iterations.

The position-preserving feature engineering proved essential to this performance. By maintaining role-specific feature identities (e.g., distinguishing a Jungler death from a Support death), the models learned role-conditional patterns that aggregated features would obscure. The selected feature set spans five distinct roles and team-level objectives, validating the multi-perspective approach.

7.2.2 Secondary Research Questions

RQ2: Which post-game features exhibit the strongest predictive power for match outcomes?

Structure-related features—specifically turret losses and inhibitor status—dominate the predictive landscape. The top two features (`turretsLost_T2_TOP`, importance 0.169; `turretsLost_T1_UTILITY`, importance 0.101) account for disproportionate predictive power. Rune configurations (5 of 15 selected features) provide independent predictive signals orthogonal to in-game performance metrics. Jungler deaths represent the most predictive individual combat statistic, reflecting the role's cascading impact on map control. Traditional offensive statistics (kills, damage dealt, gold earned) are notably absent from the top predictors, suggesting that preventing negative outcomes outweighs achieving positive outcomes in predicting victory.

RQ3: How do different machine learning algorithms compare in predictive performance?

Tree-based ensemble and gradient boosting methods substantially outperform linear, distance-based, and neural network approaches in R^2 , with a clear performance gradient: gradient boosting variants (CatB, XGB, GradB: R^2 0.854-0.868) lead, followed by Random Forest (0.853), neural network MLPR (0.834), kernel SVR (0.825), instance-based KNNR (0.801), and linear Ridge (0.755). However, accuracy is remarkably compressed across models (94.75%-96.50%), indicating that the engineered features provide strong linear separability accessible even to simple algorithms.

RQ4: To what extent do ensemble strategies improve predictive performance?

Ensemble methods provide modest but consistent improvements over individual models. The Voting Regressor ($R^2 = 0.870$) achieved the highest R^2 , representing a 0.002 gain over the best individual model (CatBoost, 0.868). The Bagging Regressor ($R^2 = 0.864$) also outperformed its base estimator. The ensemble premium is incremental rather than transformative—consistent with theoretical expectations when base models are already strong and relatively similar. For applications where maximum R^2 is the primary objective, the Voting Regressor is recommended; for most practical applications, CatBoost provides near-ensemble performance with lower computational complexity.

RQ5: How robust are the optimized models to variations in training data?

All evaluated models demonstrate high stability across 100 data partitions. Accuracy varies by approximately ± 1 percentage point across iterations, and R^2 varies by approximately ± 0.02 . CatBoost exhibits the most stable classification ($\sigma = 0.97\%$) and variance explanation ($\sigma = 0.0192$). Gradient Boosting shows higher R^2 variability ($\sigma = 0.0242$) but compensates with dramatically superior error metrics. No model exhibits instability that would preclude practical deployment, though the choice between CatBoost (stability) and Gradient Boosting (calibration quality) depends on application priorities.

7.3 Future Work

The findings and limitations of this research identify several promising directions for future investigation, spanning methodological extensions, novel modeling approaches, and practical deployment considerations.

7.3.1 Temporal Feature Engineering for In-Game Prediction

Our models operate on complete post-game data, providing retrospective outcome prediction. A natural extension is adapting the pipeline for real-time or in-game win probability estimation. This would require:

Timeline Feature Extraction:

The Riot Games API provides match timeline data with game state captured at approximately one-minute intervals. Features could be engineered to capture the *evolution* of predictive signals rather than only their final values: turret loss timing, gold differential trajectories, experience curves, and objective control sequences. The feature importance hierarchy identified in this research—structures first, then Jungler deaths, then combat metrics—provides a prioritization framework for which timeline features to emphasize.

Recurrent and Attention-Based Architectures:

Sequence models such as Long Short-Term Memory networks (LSTMs) or Transformer architectures could process the temporal sequence of game events, learning how early-game patterns condition late-game outcomes. Attention mechanisms could identify critical game moments where win probability shifts dramatically—the successful Baron steal, the game-deciding teamfight, the inhibitor defense that enables a comeback. These "tipping point" identifications would have substantial value for broadcast analysis and player review.

Prediction Horizon Analysis:

A systematic study of how prediction accuracy evolves with game time would quantify the information accumulation curve. At what point in a match does prediction accuracy cross 75%? 90%? 95%? Answering these questions would establish the minimum game duration necessary for reliable prediction and identify the game events that most rapidly resolve outcome uncertainty.

7.3.2 Champion Interaction Modeling

Our feature set includes champion identity only indirectly through rune configurations; explicit champion modeling was excluded from scope. Future work could incorporate champion selection as a primary predictive dimension:

Champion Embedding Learning:

Rather than treating champion identity as a categorical variable with 160+ levels, neural embedding layers could learn dense vector representations of champions based on their statistical performance patterns. Champions with similar playstyles, power curves, and team roles would occupy nearby positions in the embedding space, enabling the model to generalize across champions with limited sample sizes for rare picks.

Synergy and Counter-Synergy Matrices:

Pairwise champion interaction features—whether specific champion combinations on the same team exhibit synergistic win rate boosts, or whether specific matchups exhibit counter effects—

could be encoded as features. Graph neural networks operating on champion interaction graphs could propagate information across the champion roster, learning that certain champion characteristics (mobility, crowd control, range) mediate these interactions.

Draft-Phase Prediction:

With champion interaction features, the prediction task could shift earlier in the game timeline: predicting match outcomes from draft-phase information alone, before any gameplay occurs. While accuracy would necessarily be lower than post-game prediction, such models could assist with champion select decisions and provide broadcast pre-game win probability estimates.

7.3.3 Deep Learning Approaches for Sequence and Multi-Entity Data

The participant-level structure of League of Legends data—ten interacting players with role-specific behaviors—is naturally suited to deep learning architectures designed for multi-entity relational reasoning:

Graph Neural Networks:

Each match can be represented as a graph where nodes represent players (with features for champion, role, and real-time statistics) and edges represent relationships (same-team synergy, enemy-team opposition, lane matchup). Graph neural networks could learn how information propagates through this graph—how a Jungler's performance affects their Mid laner, how a Support's vision control enables their Jungler's invades—capturing the relational structure that our flat feature vectors only approximate.

Multi-Head Attention Over Roles:

Transformer architectures with role-specific attention heads could learn asymmetric relationship patterns: a "Jungler-to-lanes" attention head capturing gank impact, a "Support-to-ADC" head capturing laning phase synergy, a "Mid-to-sidelanes" head capturing roaming pressure. This structured attention would be more parameter-efficient and interpretable than fully connected attention across all player pairs.

Self-Supervised Pre-Training:

Large-scale match datasets could be used for self-supervised pre-training of neural architectures, learning general representations of game states that transfer to specific prediction tasks. Masked player modeling (predicting a player's statistics from their teammates and opponents) or next-event prediction (predicting which objective will be taken next) could produce representations that capture deep game understanding.

7.3.4 Cross-Patch Transfer Learning

League of Legends undergoes bi-weekly balance patches that can substantially alter game dynamics. Models trained on one patch may degrade in performance on subsequent patches as champion strengths, item effectiveness, and objective values shift. Addressing this temporal distribution shift requires:

Patch-Aware Model Architectures:

Incorporating patch version as a model input enables learning patch-conditional relationships. Features could interact with patch embeddings, allowing the model to learn that certain features are more predictive in certain patches (e.g., dragon control matters more after dragon buffs are strengthened).

Continual Learning Strategies:

Rather than retraining from scratch with each new patch, continual learning approaches could update model parameters incrementally as new patch data becomes available, balancing adaptation to meta shifts against retention of fundamental game knowledge that persists across patches.

Domain Adaptation Techniques:

When labeled data for a new patch is scarce (immediately after patch release), unsupervised domain adaptation could align feature distributions between old-patch and new-patch data, enabling models to maintain performance during the transition period.

Meta-Game Shift Detection:

Monitoring the feature importance hierarchy over successive patches could detect meta-game shifts automatically. A significant increase in dragon-related feature importance after a dragon buff, or a decrease in turret importance after structure defense changes, would provide empirical validation of intended design effects and early warning of unintended consequences.

7.3.5 Causal Discovery Methods

Our research has been careful to distinguish predictive association from causal influence. Future work could move beyond prediction to causation:

Causal Graph Discovery:

Algorithms such as PC (Peter-Clark) or FCI (Fast Causal Inference) could learn causal graphs from observational match data, identifying which game events causally influence others. Does losing turrets cause lost Dragons, or do lost Dragons cause lost turrets? Does Jungler death cause objective loss, or do objective contests cause Jungler deaths? Causal graphs would answer these questions with directed edges rather than symmetric correlations.

Instrumental Variable Analysis:

Game design elements that affect certain features but not others—for example, Rift Herald providing turret damage but not directly affecting combat stats—could serve as instrumental variables for estimating causal effects. This approach could quantify the causal impact of structure control on victory, distinct from its predictive association.

Counterfactual Inference:

Given a trained causal model, counterfactual queries could answer "what if" questions: "Would Team A have won if their Jungler had died one fewer time?" or "How would the win probability have changed if the first inhibitor had been taken 5 minutes earlier?" Such counterfactuals, while inherently uncertain, provide a framework for evaluating the *necessity* and *sufficiency* of specific game events for observed outcomes.

7.3.6 Deployment Considerations for Practical Tools

Translating this research into practical tools for players, coaches, and broadcasters requires addressing several engineering and design challenges:

Real-Time Inference Infrastructure:

Live win probability estimation requires model inference latency under 100ms to maintain responsiveness. This constraint favors lightweight models (Ridge, or pruned tree ensembles) over complex ensembles, and may require model compilation or hardware acceleration.

Uncertainty Quantification:

Point predictions of win probability are insufficient for high-stakes applications; users need confidence intervals. Bayesian neural networks, Monte Carlo dropout, or conformal prediction could provide calibrated uncertainty estimates, enabling statements like "Team A has a 75% probability of victory, with a 95% confidence interval of 68-82%."

Explainable AI Interfaces:

SHAP values provide feature-level explanations, but presenting these to non-technical users requires thoughtful interface design. Natural language generation from SHAP decompositions—"Team A is predicted to win because they have secured 3 more turrets and their Jungler has died 2 fewer times"—could make model reasoning accessible to players and viewers.

Player-Facing Applications:

Personalized post-game analysis tools could compare an individual player's feature contributions to win probability against role-specific benchmarks. "Your Support turret protection score was in the 30th percentile for your rank—improving this could increase your win rate by

an estimated 5%." Such tools would translate the aggregate patterns identified in this research into actionable individual feedback.

Broadcast Integration:

Real-time win probability overlays, updated after major game events (turret destruction, objective secures, teamfight outcomes), could enhance viewer engagement and provide data-driven narrative structure to broadcasts. The "tipping point" identification discussed in Section 7.3.1 could automatically highlight the game events that most dramatically shifted outcome probabilities.

7.3.7 Dataset Expansion and Diversity

Addressing the external validity limitations identified in Section 6.4.2 requires expanding the dataset along multiple dimensions:

Multi-Region Collection:

Replicating the data collection pipeline across major regions (Korea, North America, China, Europe West) would enable cross-region comparative analysis. Do the same features predict outcomes across regions, or do regional meta-game differences produce distinct predictive hierarchies? Multi-region models could learn region-invariant predictive patterns that generalize more robustly.

Professional Play Data:

Extending the methodology to professional match data would bridge the gap between ranked solo queue and elite competitive play. Professional matches feature coordinated team compositions, practiced macro strategies, and lower variance in individual performance—potentially producing different feature importance patterns more reflective of strategic rather than mechanical factors.

Longitudinal Data Collection:

Continuous data collection across multiple seasons and patches would enable the cross-patch transfer learning and meta-game shift detection described above. A multi-year dataset would also support analysis of long-term meta-game evolution: Are certain features consistently predictive across years, or does the predictive hierarchy shift with major game redesigns?

7.4 Concluding Remarks

This research has demonstrated that League of Legends match outcomes can be predicted with high accuracy from post-game player statistics and team objective data, using position-

preserving feature engineering and gradient boosting ensemble methods. The predictive hierarchy identified—structural objectives dominant, Jungler survival critical, combat statistics secondary—provides empirical validation for strategic principles long understood by high-level players: map control, expressed through structure control, is the true determinant of victory.

Beyond the specific findings, this research contributes a methodological framework—spanning feature engineering, selection, evaluation, stability analysis, and interpretability—that is transferable to other esports analytics contexts. The open-source release of the complete implementation enables reproduction, extension, and practical application by the broader community.

The limitations acknowledged throughout this work—the observational nature precluding causal claims, the single-region and single-patch specificity, the absence of temporal and champion interaction features—define the trajectory for future investigation. As esports continues its evolution from entertainment to analytical domain, the integration of rigorous machine learning methodology with deep domain knowledge will produce increasingly sophisticated tools for understanding, predicting, and improving competitive gameplay.

The question that motivated this research—"Can we predict the winner?"—has been answered affirmatively. The more consequential questions that follow—"Why did they win?" and "How can others win more often?"—define the research agenda ahead.